Note

Joanne Yu

October 18, 2026

1 February 26

1.1 Sample Preparation [1]

1.1.1 For $W_{\mathrm{L}}^{\pm}W_{\mathrm{L}}^{\pm}$

```
generate p p > w+{0} w+{0} j j QCD=0, w+ > l+ vl
add process p p > w-{0} w-{0} j j QCD=0, w- > l- vl~
output WW_LL
launch
shower = pythia8
detector = Delphes (R=0.4)
```

1.1.2 For $W_{\mathrm{L}}^{\pm}W_{\mathrm{T}}^{\pm}$

```
generate p p > w+{0} w+{T} j j QCD=0, w+ > l+ vl
add process p p > w-{0} w-{T} j j QCD=0, w- > l- vl~
output WW_LT
launch
shower = pythia8
detector = Delphes (R=0.4)
```

1.1.3 For $W_{\mathrm{T}}^{\pm}W_{\mathrm{T}}^{\pm}$

```
generate p p > w+{T} w+{T} j j QCD=0, w+ > l+ vl
add process p p > w-{T} w-{T} j j QCD=0, w- > l- vl~
output WW_TT
launch
shower = pythia8
detector = Delphes (R=0.4)
```

1.1.4 For Unpolarized $W^{\pm}W^{\pm}$

```
generate p p > w+ w+ j j QCD=0, w+ > l+ vl
add process p p > w- w- j j QCD=0, w- > l- vl~
output WW_UN
launch
shower = pythia8
detector = Delphes (R=0.4)
```

1.2 Restrictions for Candidates

1.2.1 Electrons

Electrons are required to satisfy the "Tight" likelihood-based identification criteria.

- $p_{\mathrm{T}} > 27$ GeV
- $|\eta| < 2.47$ GeV
- Excluding the calorimeter transition region $1.37 < |\eta| < 1.52$

```
ele_mask = (eles.pt > 27) & (abs(eles.eta) < 2.47) & ~((abs(eles.eta) > 1.37) &
(abs(eles.eta) < 1.52))
```

1.2.2 Muons

Muons are required to satisfy the "medium" identification selection.

- $p_{\mathrm{T}} > 27$ GeV
- $|\eta| < 2.5$

```
mu_mask = (muons.pt > 27) & (abs(muons.eta) < 2.5)
```

1.2.3 Jets

Jets are reconstructed using the anti-k_{i} algorithm with a radius parameter of $R = 0.4$, using the particle-flow object.

- At least two jets with $|\eta| < 4.5$ are required to be present in each event
- $p_{\mathrm{T}} > 65$ GeV for leading jets (written in the paragraph for election of the leading leptons later.)
- $p_{\mathrm{T}} > 35$ GeV for subleading jets

```
jet_mask = (jets.pt > 35) & (abs(jets.eta) < 4.5)
```

1.3 Selection Cuts

The event selection in the signal region (SR) requires:

1. A same-sign lepton pair with $m_{\ell\ell} > 20$ GeV

```
padded_leps = ak.pad_none(all_leps, 2)
l1, l2 = padded_leps[:, 0], padded_leps[:, 1]

mask1 = (ak.num(all_leps) >= 2) & ak.fill_none((l1.q == l2.q) & ((l1 + l2).mass
> 20), False)
n1 = ak.sum(mask1)
```

2. $E_{\mathrm{T}}^{\mathrm{miss}} > 30$ GeV

```
met_pt = ak.firsts(data.MissingET.MET)
mask2 = mask1 & ak.fill_none((met_pt > 30), False)
n2 = ak.sum(mask2)
```

3. VBS topology: The two highest-p_{T} jets must satisfy $m_{\mathrm{jj}} > 500$ GeV with $|\Delta y_{\mathrm{jj}}| > 2.0$

```
padded_jets = ak.pad_none(sel_jets, 2)
j1, j2 = padded_jets[:, 0], padded_jets[:, 1]

vbs_kinematics = (j1.pt > 65) & (j2.pt > 35) & ((j1 + j2).mass > 500) & (abs(j1.
rapidity - j2.rapidity) > 2.0)
mask3 = mask2 & (ak.num(sel_jets) >= 2) & ak.fill_none(vbs_kinematics, False)
n3 = ak.sum(mask3)
```

Cut	LL	pass rate	LT	pass rate	TT	pass rate	Unpol.	pass rate
Total	50000	1.00	50000	1.00	50000	1.00	50000	1.00
Lepton cut	12534	0.25	12453	0.25	12814	0.26	12491	0.25
MET cut	10327	0.21	10614	0.21	11156	0.22	10745	0.21
Jet cut	4403	0.09	5254	0.11	5477	0.11	5252	0.11

Table 1: Event Pass Rates across different polarization states.

1.4 Kinematic Distributions

1.4.1 Definition and Physical Meaning

1. Transverse Mass m_{T}

   In the lab frame, we only know the transverse components of the neutrinos (the Missing Transverse Energy, or MET). Because the longitudinal momentum (p_z) of the neutrinos is unknown, we cannot calculate a true invariant mass. Instead, we project everything onto the 2D transverse plane ($x - y$) to calculate the Cluster Transverse Mass (m_{T}).

   The Mathematics: To calculate the m_{T} of the WW system, we treat the two visible leptons as a single combined "cluster"' ($\ell\ell$) and the MET as the invisible component. Dilepton Transverse Energy ($E_{\mathrm{T}}^{\ell\ell}$): This combines the transverse momentum and the invariant mass of the dilepton system.

$$E_{\mathrm{T}}^{\ell\ell} = \sqrt{|\vec{p}_{\mathrm{T}}^{\,\ell\ell}|^2 + M_{\ell\ell}^2}, \tag{1}$$

   Missing Transverse Energy ($E_{\mathrm{T}}^{\mathrm{miss}}$): This is simply the magnitude of the MET vector.

$$E_{\mathrm{T}}^{\mathrm{miss}} = |\vec{p}_{\mathrm{T}}^{\,\mathrm{miss}}|, \tag{2}$$

WW Transverse Mass (m_{T}^{WW}): We subtract the squared vector sum of the transverse momenta from the squared scalar sum of the transverse energies.

$$m_{\mathrm{T}}^{WW} = \sqrt{\left(E_{\mathrm{T}}^{\ell\ell} + E_{\mathrm{T}}^{\mathrm{miss}}\right)^2 - \left|p_{\mathrm{T}}^{\ell\ell} + p_{\mathrm{T}}^{\mathrm{miss}}\right|^2}. \tag{3}$$

2. Difference in Azimuthal Angle $\Delta\phi_{\mathrm{jj}}$

$$\phi = \tan^{-1}\left(\frac{p_y}{p_x}\right) \rightarrow \Delta\phi_{\mathrm{jj}} = |\phi_{\mathrm{j1}} - \phi_{\mathrm{j2}}|, \tag{4}$$

Measures the angle between the two tagging jets in the transverse plane.

3. Difference in Pseudorapidity $\Delta\eta_{\ell\ell}$

$$\eta = \frac{1}{2}\ln\left(\frac{|p| + p_z}{|p| - p_z}\right) \rightarrow \Delta\eta_{\ell\ell} = |\eta_{\ell 1} - \eta_{\ell 2}|, \tag{5}$$

Measures how far apart the two charges leptons are along the longitudinal axis.

1.4.2 Results

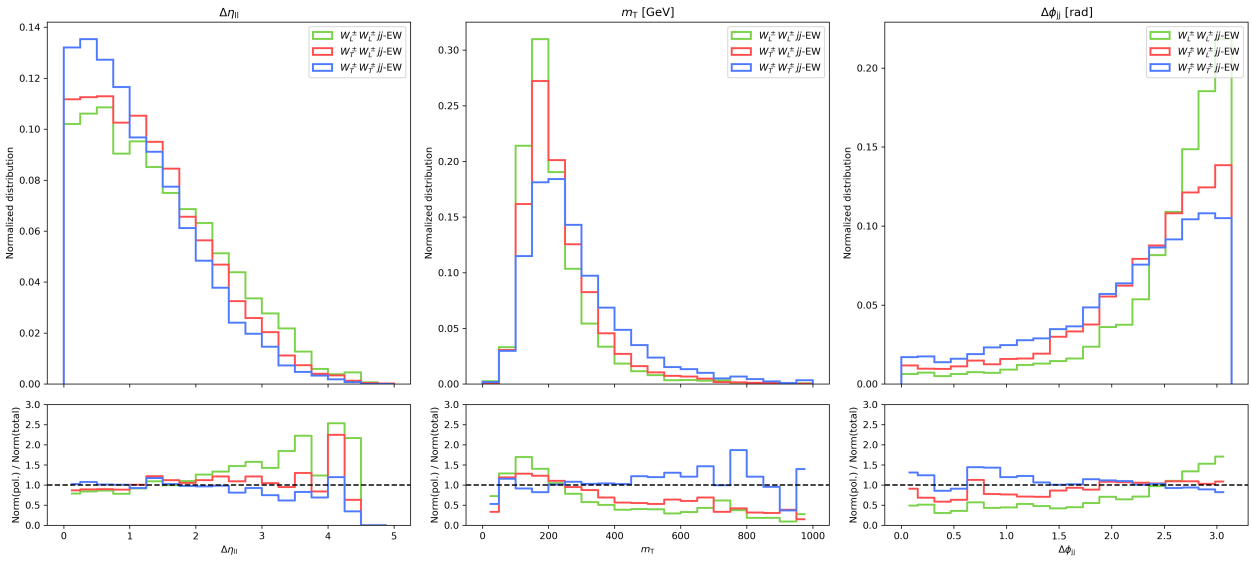


Figure 1: Distribution in the signal region for the difference in pseudorapidity between leading and subleading leptons, the transverse mass, and the difference in azimuthal angle between leading and subleading jets.

For the bottom panel, the ratio is calculated by comparing the normalized distribution of a polarized sample((LL, LT, or TT) against the normalized distribution of the unpolarized (total) sample. The ratio is derived for the plotting:

Before calculating the ratio, the code ensures both the polarized and unpolarized samples are on the same scale by normalizing them. Instead of using the raw number of events, it treats each event as a fraction of the total sample size:

$$w_{\mathrm{i}} = \frac{1}{N_{\mathrm{events}}}, \tag{6}$$

where N_{events} is the number of events that passed all cuts for that specific sample.

For each variable, the data is divided into k bins. Let $H_{\text{p}}(b)$ be the normalized count for a polarized sample in bin b, and $H_{\text{u}}(b)$ be the normalized count for the unpolarized sample:

$$H(b) = \sum_{x_i \in \text{bin } b} w_{\text{i}}, \tag{7}$$

The ratio plotted in the bottom panel represents how much a specific polarization deviates from the average unpolarized behavior in a specific kinematic region:

$$R(b) = \frac{\text{Norm(Polarized)}_b}{\text{Norm(Unpolarized)}_b} = \frac{H_{\text{p}}(b)}{H_{\text{u}}(b)}, \tag{8}$$

- If $R(b) > 1$: The polarized state is more likely to produce events in that specific parameter range than the total population.
- If $R(b) < 1$: The polarized state is suppressed in that range.

2 March 1st

2.1 Additional

To address the low selection efficiency observed in the initial MadGraph results1, I implemented additional phase-space cuts directly in the run card and increased the number of events generated. This refinement ensures a higher yield of relevant events and optimizes computational resources.

I adjusted the following parameters in the run_card. Including restrictions for the transverse momentum of leptons and jets, missing transverse mass, pseudorapidity for leptons and jets, invariant mass for lepton pairs and jet pairs, and the transverse momentum for leading and subleading jets.

```
27.0 = ptl
35.0 = ptj
20.0 = misset
2.5 = etal
4.5 = etaj
20.0 = mmll
500.0 = mmjj
65.0 = ptj1min
35.0 = ptj2min
```

Event Pass Rates	LL		LT		TT		Unpol	
Cut	Count	Rate	Count	Rate	Count	Rate	Count	Rate
Total	100000	1.00	100000	1.00	100000	1.00	100000	1.00
Lepton cut	26793	0.27	27563	0.28	30021	0.30	28388	0.28
MET cut	23816	0.24	25036	0.25	27755	0.28	25979	0.26
Jet cut	19002	0.19	20602	0.21	22352	0.22	21116	0.21

Cross Sections	LL (pb)	LT (pb)	TT (pb)	Unpol (pb)
Total	0.000505	0.002843	0.004288	0.007653
Lepton cut	0.000135	0.000784	0.001287	0.002173
MET cut	0.000120	0.000712	0.001190	0.001988
Jet cut	0.000096	0.000586	0.000959	0.001616

Table 2: Event pass rates and cross sections after pre-cut in MadGraph for LL, LT, TT, and Unpolarized samples.

In table 2, we can see that applying pre-cutting in MadGraph improves the event throughput, which increases the efficiency of data utilization. Furthermore, the change in cross-section after each cut is exactly the ratio times the original cross-section.

2.2 Transformation from pp-cmf to WW-cmf

Since the kinematic distribution in the paper[1] is under WW-cmf, we should consider to transform our original generated events in MadGraph from pp-cmf to WW-cmf.

2.2.1 Work Flow

To transform the cmf of the events, we have to first extract the 4-vector from the original events including leptons, neutrinos, and W bosons, where $W = l + \nu_l$. Then, we calculate the WW system kinematic by:

$$P_{WW} = P_{W+} + P_{W-}, \tag{9}$$

where P is the 4-vector.

To move into the WW-cmf, we must apply a Lorentz transformation to boost our target particles in the opposite direction of the WW system's motion. Therefore, we boost using $-\vec{\beta}$.

$$\vec{\beta}_{WW} = \frac{\vec{p}_{WW}}{E_{WW}}, \tag{10}$$

where $\vec{p}_{WW}$ equals (p_x, p_y, p_z).

Since we only know $(E_\ell, p_{\ell,x}, p_{\ell,y}, p_{\ell,z})$ and $(p_{\nu,x}, p_{\nu,y})$, we have to calculate the $p_{\nu,z}$ and E_ν. By using the W-mass constraint, which assumes the W boson is perfectly on-shell,

$$(p_\ell + p_\nu)^2 = m_W^2, \tag{11}$$

where $m_W \approx 80.4$ GeV.

By expanding equation11 using the known lepton kinematics and the measured transverse missing energy, we can end up with a standard quadratic equation for the neutrino's longitudinal momentum and then figure out the $p_{\nu,z}$ and E_ν. However, the transverse part of the momentum is non zero, therefore, the boost is not a purely longitudinal boost, but a 3D boost.

2.2.2 Expectations after the Transformation

- $\Delta\eta_{\ell\ell}$:

$$\Delta\eta' \neq \Delta\eta, \tag{12}$$

The difference in pseudorapidity should change on an event-by-event basis. Because a 3D boost mixes the momentum components, there is no single constant shift applied to the leptons. The $\Delta\eta$ distribution in the boosted frame should look smeared or broaden compared to the lab frame.

- m_{T}:

$$m_{\mathrm{T,pp-cmf}} \to M_{WW,WW-\mathrm{cmf}}, \tag{13}$$

Since we calculated the longitudinal momentum for neutrino, we can then figure out the invariant mass, instead of the transverse mass. While the total momentum in WW-cmf is zero, the invariant mass of the system is simply the total energy of all decay products:

$$M_{WW} = E^*_{\ell 1} + E^*_{\nu 1} + E^*_{\ell 2} + E^*_{\nu 1}. \tag{14}$$

- $\Delta\phi_{\mathrm{jj}}$:

When analyzing the Vector Boson Scattering (VBS), the leading and subleading tagging jets have a very specific kinematic signature: they are highly forward. Which means they have massive total energies and longitudinal momenta. The $\gamma E\beta_{\mathrm{T}}$ term becomes massive and so, the $\Delta\phi_{\mathrm{jj}}$ distribution is will be heavily distorted.

$$\vec{p}'_{\mathrm{Ti}} = \vec{p}_{\mathrm{Ti}} + \left[\frac{\gamma-1}{\beta^2}(\vec{\beta}\cdot\vec{p}_{\mathrm{i}}) - \gamma E_{\mathrm{i}}\right]\vec{\beta}_{\mathrm{T}}. \tag{15}$$

2.2.3 Results

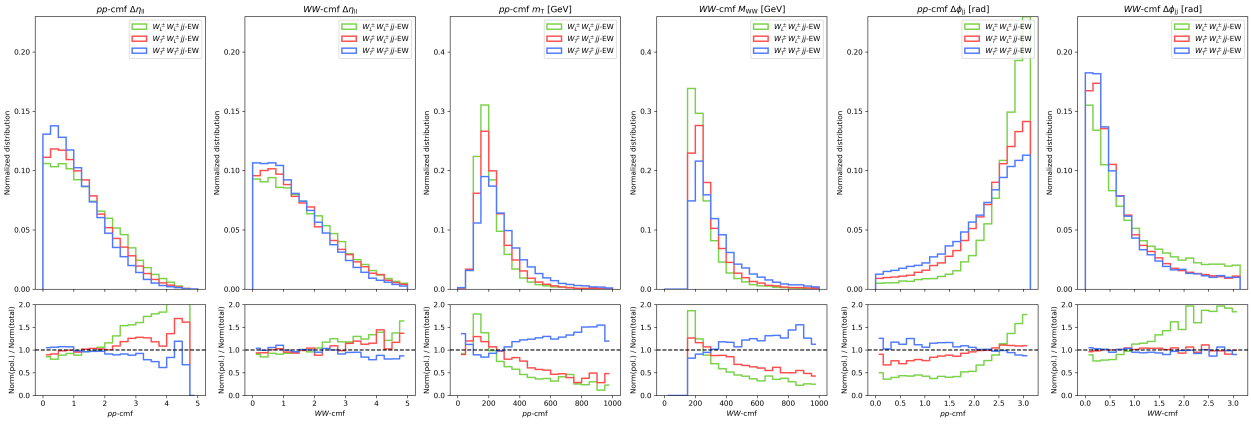


Figure 2: Distribution in the signal region after pre-cut and transformation to the WW-cmf in MadGraph for the difference in pseudorapidity between leading and subleading leptons, the transverse mass, and the difference in azimuthal angle between leading and subleading jets.

- **Peak values for $\Delta\eta_{\ell\ell}$ decreases:** The decrease in the TT peak for $\Delta\eta_{\ell\ell}$ in the WW-cmf occurs because boosting into the center-of-mass frame removes the global longitudinal "push" of the lab frame, unfolding the system and allowing the naturally broader angular distributions of transversely polarized bosons—which follow a $(1 \pm \cos^2\theta^*)$ distribution—to spread out and lower the normalized peak. Furthermore, because TT leptons are emitted more longitudinally, they are highly sensitive to "smearing" and potential under-boosting caused by the algebraic approximation of p_z, which further flattens the distribution compared to the LL state.

- **M_{WW} goes to zero below ≈ 160 GeV:** The M_{WW} values drop to zero below approximately 160 GeV primarily due to the on-shell threshold; since we force a W-mass constraint of 80.385 GeV for each reconstructed boson, the total invariant mass cannot physically fall below the sum of those two masses (~ 161 GeV). Additionally, the selection criteria—specifically the high p_{T} requirements for leptons and the $m_{jj} > 500$ GeV VBS topology cut—effectively filter out low-energy phase space where a lower M_{WW} might otherwise occur.

- **$\Delta\phi_{\mathrm{jj}}$ goes to zero:** In pp-cmf, the $\Delta\phi_{\mathrm{jj}}$ distribution is governed by the hard scatter and recoil dynamics. But in WW-cmf, the distribution is heavily destroyed by the massive Energy term through the 3D boost. The jets are forced to point in similar transverse direction, making them highly collinear in ϕ.

3 March 8th

3.1 WW-cmf in MadGraph

```
generate p p > w+{0} w+{0} j j QCD=0, w+ > l+ vl
add process p p > w-{0} w-{0} j j QCD=0, w- > l- vl~
output sample/script/LL
launch sample/script/LL

shower = Pythia8
detector = Delphes
analysis= OFF
madspin= OFF

set run_card nevents 100000
set run_card ebeam1 6500
set run_card ebeam2 6500

set run_card me_frame 3,4,5,6

set run_card cut_decays True
set run_card ptj 34.0
set run_card ptl 26.0
set run_card misset 29.0

set run_card etaj 4.6
set run_card etal 2.6

set run_card mmjj 450

set run_card ptj1min 64
set run_card ptj2min 34
```

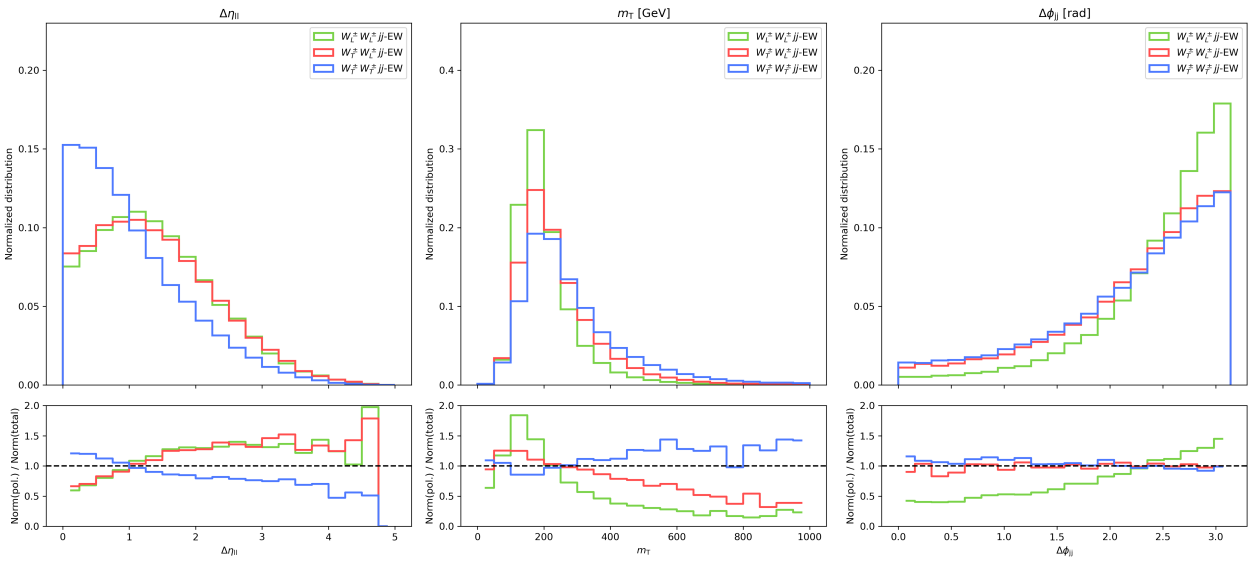


Figure 3: Distribution in the signal region after pre-cut in the WW-cmf in MadGraph for the difference in pseudorapidity, transverse mass, and difference in azimuthal angle.

pt¡ -pt¿ tbl/cell/end ˙args¨int=0Γ	*pp*-cmf (fb)				WW-cmf (fb)			
	LL	LT	TT	Total	LL	LT	TT	Total
Total	0.505	2.843	4.288	7.653	0.396	1.263	2.334	3.913
Lepton cut	0.135	0.784	1.287	2.173	0.201	0.650	1.284	2.082
MET cut	0.120	0.712	1.190	1.988	0.186	0.611	1.218	1.966
Jet / VBS cut	0.096	0.586	0.959	1.616	0.148	0.484	0.960	1.547
Jet cut (Thesis)	0.177	1.030	1.718	2.925	0.277	0.874	1.744	2.896
Fraction	5.9%	35.7%	58.4%	100%	9.3%	30.4%	60.3%	100%
Fraction (Thesis)	6.1%	35.2%	58.7%	100%	9.6%	30.2%	60.2%	100%

Table 3: Comparison of cutflows, cross-sections, and polarization fractions for *pp*-cmf and WW-cmf frames. Results from local MadGraph generations are compared against the thesis references. All cross-sections are expressed in fb.

Table 3 compares the cutflow and polarization results of the local MadGraph generation against the reference thesis. While the absolute cross-sections in the signal region (Jet/VBS cut) are roughly 45% lower than the thesis values, the polarization fractions show excellent agreement. Specifically, the LL, LT, and TT fractions deviate by less than 1% from the reference in both the *pp*-cmf and WW-cmf frames. This confirms that despite the normalization difference, the underlying angular distributions and physics of the polarized states are correctly modeled. Figure 3 also shows the underlying kinematic distributions are similar to the results in the thesis.

3.2 Sherpa3

In Sherpa3, instead of generating an event that is 100 % purely "LL" or 100 % "TT", Sherpa generates a physical event (a specific arrangement of outgoing jets and leptons) and then calculates a weight for each polarization state for that exact same event.

Also, we must separate the W^+W^+jj and W^-W^-jj processes when using the Pol_Cross_Section feature because of how the generator strictly handles memory allocation for event weights.

3.2.1 Sample Preparation

For $W+$:

```
EVENTS: 150000      #number of events to generate

BEAMS: [2212, 2212]     #incoming colliding particles; 2212 for a proton
BEAM_ENERGIES: 6500     #incoming particle energy for each

#specifies the internal structure model of the proton to be used during the collision
PDF_LIBRARY: LHAPDFSherpa       #tell Shepra to interface with the library
PDF_SET: PDF4LHC21_40_pdfas     #highly accurate PDF set

#93: any light quark, anti-quark, or gluon
#24: w+ boson
PROCESSES:
#2 incoming partons scatter to produce 2 same sign bosons and 2 outgoing partons
  - 93 93 -> 24 24 93 93:
#forbidding any strong-force interactions in the hard matrix element
      Order: {QCD: 0, EW: 4}

SELECTORS:
#invariant mass of the 2 outgoing partons must be between 100-13,000 GeV
  - [Mass, 93, 93, 100.0, 13000.0]

PARTICLE_DATA:
#treat w+ boson as strictly on-shell during the initial matrix element calculation
  24:
    Width: 0.0
#turns on the decay module for heavy particles
HARD_DECAYS:
  Enabled: true
#crucial!!
#ensures the spin-correlations and polarization of w boson are calculated and passed
down to their decay products
  Pol_Cross_Section:
    Enabled: true
    Reference_System: COM       #COM:ww-cmf; Lab: pp-cmf
#disables specific decay routes for the w boson
#status : 0 meaning turns a decay channel off.
  Channels:
```

```
    '24,2,-1': {Status: 0}          #up+anti-down quark (hardronic decay)
    '24,4,-3': {Status: 0}          #charm+anti-strange quark (hardronic decay)
    '24,16,-15': {Status: 0}        #tau neutrino+anti-tau lepton (hardronic decay)
#dictates how and where the final simulated events are saved
EVENT_OUTPUT: HepMC3_GenEvent[Sherpa_Wp]
```

For $W-$:

```
EVENTS: 150000

BEAMS: [2212, 2212]
BEAM_ENERGIES: 6500

PDF_LIBRARY: LHAPDFSherpa
PDF_SET: PDF4LHC21_40_pdfas

PROCESSES:
  - 93 93 -> -24 -24 93 93:
      Order: {QCD: 0, EW: 4}

SELECTORS:
  - [Mass, 93, 93, 100.0, 13000.0]

PARTICLE_DATA:
  -24:
    Width: 0.0

HARD_DECAYS:
  Enabled: true
  Pol_Cross_Section:
    Enabled: true
    Reference_System: COM
  Channels:
    '-24,-2,1': {Status: 0}
    '-24,-4,3': {Status: 0}
    '-24,-16,15': {Status: 0}

EVENT_OUTPUT: HepMC3_GenEvent[Sherpa_Wm]
```

	Cross Sections (fb)				Event Pass Rates	
	LL	LT	TT	Unpol	Raw Events	Fraction
Total	1.239	3.781	6.874	11.915	299,998	1.0000
Lepton cut	0.577	1.710	3.230	5.479	142,376	0.4746
MET cut	0.477	1.473	2.767	4.658	120,052	0.4002
Jet cut	0.239	0.753	1.398	2.349	49,259	0.1642

Table 4: Cross sections for different polarization states and corresponding event pass rates at each selection cut stage.

According to table 4, the event pass rate for Sherpa simulation is much smaller than MadGraph due to less cuts in the event generating step. Perhaps we can explore whether there are other cuts can be added to Sherpa to enhance the event usage.

	Sherpa pp-cmf (fb)				Sherpa WW-cmf (fb)			
	LL	LT	TT	Total	LL	LT	TT	Total
Total	1.060	4.787	6.083	11.915	1.254	3.816	6.873	11.915
n_{ll} cut	0.472	2.180	2.906	5.456	0.575	1.714	3.214	5.456
MET cut	0.381	1.845	2.516	4.635	0.469	1.470	2.755	4.635
VBS cut	0.175	0.915	1.306	2.335	0.237	0.749	1.392	2.335
VBS cut (Thesis)	0.182	1.072	1.732	2.933	0.285	0.909	1.778	2.933
Fraction	7.3%	38.2%	54.5%	100%	10.0%	31.5%	58.5%	100%
Fraction (Thesis)	6.1%	35.9%	58.0%	100%	9.6%	30.6%	59.8%	100%

Table 5: Comparison of cutflows, cross-sections, and polarization fractions for pp-cmf and WW-cmf frames using Sherpa. Results from local generations are compared against the thesis Sherpa ($W^{\pm}W^{\pm}jj$) references.

Table 5 shows that the calculated cross-sections are in close agreement with the thesis references. Furthermore, the polarization fractions for the LL, LT, and TT states remain consistent with the reference values across both the pp-cmf and WW-cmf frames.

4 March 12

4.1 Sherpa3 Rerun

It was identified that the initial analysis in section 3.2 lacked a proper detector simulation stage; specifically, Delphes was not applied to the Sherpa-generated events. Furthermore, the standalone Python implementation for jet reconstruction was unreliable for this analysis. Consequently, the Sherpa output files were re-processed through Delphes to ensure a more robust reconstruction of jets within the signal region.

Table 6 summarizes the results following this detector simulation. Notably, the discrepancy persists, which aligns with the results from MadGraph: the cross-sections obtained after the three signal region cuts are approximately 50% lower than those reported in the reference thesis. Despite this difference in absolute magnitude, the relative polarization fractions (LL, LT, and TT) remain broadly consistent with the thesis values.

	Sherpa3 + Delphes pp-cmf (fb)				Sherpa3 + Delphes WW-cmf (fb)			
	LL	LT	TT	Total	LL	LT	TT	Total
Total	1.060	4.787	6.083	11.915	1.254	3.816	6.873	11.915
n_{ll} cut	0.265	1.245	1.740	3.202	0.332	0.985	1.905	3.202
MET cut	0.217	1.057	1.515	2.744	0.275	0.847	1.645	2.744
VBS cut	0.112	0.585	0.870	1.540	0.154	0.479	0.922	1.540
VBS cut (Thesis)	0.182	1.072	1.732	2.933	0.285	0.909	1.778	2.933
Fraction	7.1%	37.4%	55.5%	100%	9.9%	30.8%	59.3%	100%
Fraction (Thesis)	6.1%	35.9%	58.0%	100%	9.6%	30.6%	59.8%	100%

Table 6: Comparison of cutflows, cross-sections, and polarization fractions for pp-cmf and WW-cmf frames using Sherpa after Delphes detector simulation. Local generations are compared against the thesis Sherpa ($W^{\pm}W^{\pm}jj$) references.

4.2 PDF Selection

I was using the preset settings for event generation in MadGraph of PDF selection:

```
nn23lo1 = pdlabel
230000 = lhaid
```

But to compare the results with Sherpa, we need to use the same PDF. Therefore, I decided to use the following settings:

In Sherpa3:

```
PDF_SET: PDF4LHC21_40_pdfas
```

In MadGraph:

```
set pdlabel lhapdf
set lhaid 93300
```

And I found that the cross section for different PDF changed:

	LL (fb)		LT (fb)		TT (fb)	
	PDF	Nominal	PDF	Nominal	PDF	Nominal
Total	0.434	0.396	1.372	1.263	2.531	2.334
n_{ll} cut	0.221	0.201	0.702	0.649	1.376	1.284
MET cut	0.205	0.186	0.660	0.611	1.302	1.218
VBS cut	0.163	0.148	0.518	0.484	1.017	0.960

Table 7: Comparison of cross-sections for different polarization states showing the effect of the PDF selection. The "PDF" columns represent the new PDF choice, while "Nominal" represents the baseline generation. All values are in fb.

To ensure consistency between MadGraph and Sherpa3, the default NNPDF2.3 LO (lhaid 230000) was replaced with the modern PDF4LHC21 benchmark (lhaid 93300).

As shown in table 7, updating to the PDF4LHC21 set results in a systematic increase in cross sections across all polarization states. This shift is expected, as the modern PDF set incorporated more precise experimental data and updated theoretical constraints compared to the decade-old nominal set. This transition ensures that the local simulations align with current LHC standards and provide more reliable results.

4.3 Particle-level phase space

After I read the thesis in detail again, I found out that the thesis was comparing the cross section at particle-level, not the detector-level. However, the previous cross section for my simulation were calculated at the detector-level. Instead of rerunning the event generations, we can use the free Delphes GenJet branch. Despite Delphes is a detector simulator, its default configuration actually runs the FastJet clustering algorithm on the PYTHIA truth particles before it applies any detector smearing.

- Particle-Level jets: Must run a jet clustering algorithm on all stable final state particles (Status = 1). (anti-kt algorithm with $R = 0.4$).
- Particle-Level MET: This is calculated as the negative vector sum of the transverse momenta of all stable, weakly interacting particles.
- Particle-Level Leptons: These are stable electrons and muons straight from the PYTHIA output.

```
particles = "Particle.PT", "Particle.Eta", "Particle.Phi", "Particle.Mass"...
genjets = "GenJet.PT", "GenJet.Eta", "GenJet.Phi", "GenJet.Mass"
genmet = "GenMissingET.MET"
```

Sample	WW-cmf			pp-cmf		
	LL	LT+TL	TT	LL	LT+TL	TT
Thesis MadGraph	0.277 fb	0.874 fb	1.744 fb	0.177 fb	1.030 fb	1.718 fb
	9.58%	30.18%	60.24%	6.07%	35.20%	58.73%
Local MadGraph	0.256 fb	0.814 fb	1.595 fb	0.166 fb	0.954 fb	1.560 fb
	9.59%	30.56%	59.85%	6.19%	35.60%	58.22%
Thesis Sherpa $W^{\pm}W^{\pm}jj$	0.285 fb	0.909 fb	1.778 fb	0.182 fb	1.072 fb	1.732 fb
	9.60%	30.59%	59.81%	6.09%	35.91%	58.00%
Local Sherpa3	0.252 fb	0.780 fb	1.519 fb	0.192 fb	0.977 fb	1.406 fb
	9.87%	30.58%	59.54%	7.47%	37.93%	54.60%

Table 8: Comparison of cross-sections and polarization fractions for WW-cmf and pp-cmf frames. The reference thesis values (MadGraph and Sherpa) are compared against the local MadGraph generations and local Sherpa3 generations at the particle-level VBS cut.

Table 8 provides a comprehensive comparison between local simulation and the reference thesis for both pp-cmf and WW-cmf. The polarization fractions show similar results. While the absolute cross section in the local generations are slightly lower than the thesis reference, which may likely due to higher order corrections or PDF sets. This result also shows that the underlying physics modeling and symmetry factors in MadGrpah or Sherpa3 setups are correctly aligned with the thesis results.

5 3/20

5.1 Background events

Following are the ranked background events mentioned in [2]:

1. $W^{\pm}Z$ background (dominant)
2. Data-driven backgrounds (∼18 % combined):
   - Non-prompt leptons: arises from leptons originating from hadron decays and jets that are misidentified as leptons. The source of these events is mainly W+jets and semi-leptonic $t\bar{t}$ processes.
   - Charge-flip background (electron charge misidentification): This background primarily comes from $Z \longrightarrow e^{+}e^{-}$ decays. A high-energy electron emits a hard photon via bremsstrahlung, which then undergoes pair production into a high-energy positron and a soft electron. The soft electrons get lost in the detector, and the misidentified opposite-charged positron is measured instead, incorrectly passing the same-charge requirement.
3. QCD $W^{\pm}W^{\pm}jj$ background (∼5.2 %)
4. Small prompt backgrounds (< 2 % combined)
5. Interference $W^{\pm}W^{\pm}jj-$INT (∼1.6 %)

5.1.1 $W^{\pm}Zjj$ background

- $pp \longrightarrow W^{\pm}Zjj \longrightarrow \ell^{\pm}\nu\ell^{\pm}\ell^{\mp}$ (including both pure electroweak $W^{\pm}Z-$EW and QCD-induced $W^{\pm}Z-$ QCD scattering)
- Event generator:
  - $W^{\pm}Z-$EW: Madgraph
  - $W^{\pm}Z-$QCD: Sherpa2.2.12 + data-driven approach (will Shepra3 simulation be sufficient?) + control region restrictions
- CMF: evaluated in the WW-cmf and pp-cmf
- PDF set: NNPDF3.0_nnlo_as_0118

This background originates from the fully leptonic decay. It mimics the signal region criteria when one of the leptons from the Z boson decay fails baseline selection or falls completely outside the detector's acceptance. This leaves exactly two same-charged leptons, two jets, and missing transverse momentum. It consists of both pure electroweak and QCD-induced scattering events.

Comprises the electroweak scattering $W^{\pm}Z$-EW and the strong scattering $W^{\pm}Z$-QCD. $W^{\pm}Z$-EW can be simulated by MadGraph well, while $W^{\pm}Z$-QCD is known to be poorly modeled by Monte Carlo simulations. Sample generated from Sherpa for $W^{\pm}Z$-QCD still needs to be corrected using the same data-driven approach as in [2].

5.1.2 QCD $W^{\pm}W^{\pm}jj$ background

- $pp \longrightarrow \ell^{\pm}\nu\ell^{\pm}\nu jj$ via strong interaction vertices at order $O(\alpha_{\mathrm{EW}}^4\alpha_{\mathrm{S}}^2)$
- Event generator: MadGraph
- CMF: evaluated in the WW-cmf anf pp-cmf
- PDF set: NNPDF3.0_nnlo_as_0118

This is an irreducible background that yields the exact same final state particles as the signal, but it is produced at the order $O(\alpha_{\mathrm{EW}}^4\alpha_{\mathrm{S}}^2)$ involving strong QCD interaction vertices.

5.1.3 Interference background $W^{\pm}W^{\pm}jj$−INT

- Quantum mechanical interference between the purely electroweak and QCD-induced $W^{\pm}W^{\pm}jj$ diagrams that share identical external quarks, calculated at order $O(\alpha_{\mathrm{EW}}^5\alpha_{\mathrm{S}})$
- Event generator: MadGraph and Sherpa

  For MadGraph, the interference is calculated by:

$$N_{\mathrm{int}} = N_{\mathrm{unpol}} - \sum_{\mathrm{pol}} N_{\mathrm{pol}} \tag{16}$$

  For Sherpa, the interference contribution is directly provided by an event weight as for the individual polarization states.
- CMF: evaluated in the WW-cmf anf pp-cmf
- PDF set: modeled by MadGraph using NNPDF3.0

5.2 Control Region

5.2.1 $W^{\pm}Z$ control region

- Two signal leptons and an additional third signal lepton with $p_{\mathrm{T}} > 15$ GeV
- In the case of two electrons they have to fulfill $|\eta| < 1.37$ and $|m_{ee} - m_Z| > 15$ GeV
- At least two leptons satisfy $m_{\ell\ell} \geq 20$ GeV
- $m_{\ell\ell\ell} \geq 106$ GeV
- $p_{\mathrm{T}}^{\mathrm{miss}} \geq 30$ GeV
- At least two signal jets with the highest $p_{\mathrm{T}} > 65$ GeV and the second highest $p_{\mathrm{T}} > 35$ GeV
- $m_{jj} \geq 200$ GeV
- $|\Delta y_{jj}| > 2$
- No jet is b-tagged using the DL1r tagger with the 85% efficiency working point

6 3/29

6.1 Signal events

Adjusting the PDF set for MadGraph into

```
set pdlabel lhapdf
set lhaid 260000
```

which sets directly the same PDF set used in the thesis (NNPDF30_nlo_as_0118)

The following table is the after the correct PDF set selection with the thesis (NNPDF30_nlo_as_0118). We can compare how different overlap removal affect the cross section. But it isn't clearly mentioned in the thesis how the overlap removal is applied.

MadGraph	**with step 5**	**without step 5**	**corrected**	**thesis**
LL	0.250	0.218	0.229	0.277
LT	0.794	0.662	0.699	0.874
TT	1.558	1.193	1.309	1.744

Table 9: Signal events generated from MadGraph with the PDF set: NNPDF30_nlo_as_0118 and with different overlap removal conditions.

where the column "corrected" is set by the rules mentioned in the thesis 6.3.4 Fiducial Signal Region. If the resulting jet overlaps with an electron within $\Delta R < 0.4$, the electron is removed for $p_{T,e}/p_{T,\text{jet}} < 0.5$. The jet is removed if the electron carries more than 50% of the transverse jet momentum.

And the step 5 is a action to remove jets with $R > 0.3$ of the signal leptons.

Sherpa3	**with step 5**	**without step 5**	**corrected**	**thesis**
LL	0.252	0.233	0.236	0.285
LT	0.780	0.700	0.717	0.909
TT	1.519	1.245	1.325	1.778

Table 10: Signal events generated from Sherpa3 with the PDF set: PDF4LHC21_40_pdfas and with different overlap removal conditions.

We can see from both table that different choice of the overlap removal affect the cross section very much. However, in both the thesis or in the references mentioned in the thesis didn't clearly denote what overlap removal is applied to the signal events. Therefore, the discrepancy between our simulation result and the result in the thesis might due to the choice of the overlap removal.

6.2 Background events

I found some tables and figures that we might be able to compare with the background events before training the DNN model. There are different kinds of cut mentioned in the thesis, which is summarized in the following figure: According to this figure 4, we can compare our simulation result with table G.2-5 in the thesis [3] which includes the number of signal and background events in SR, CR, and low m_jj region. Also, it includes both the pp-cmf and WW-cmf. Therefore, by the following equation,

$$N_{\text{expected}} = \sigma \times \mathcal{L} \times \epsilon \tag{17}$$

Requirement	SR	Low-m_{jj} CR	WZ CR
Leading and subleading lepton p_T		> 27 GeV	
Electron $\|\eta\|$		< 2.47 (1.37 in ee), excluding $1.37 \leq \|\eta\| \leq 1.52$	
Muon $\|\eta\|$		< 2.5	
Leading (subleading) jet p_T		> 65 (35) GeV	
Additional jet p_T		> 25 GeV	
Jet $\|\eta\|$		< 4.5	
$m_{\ell\ell}$		> 20 GeV	
E_T^{miss}		> 30 GeV	
Charge misid. $Z \to ee$ veto		$\|m_{ee} - m_Z\| > 15$ GeV	–
b-jet veto		$N_{b\text{-jet}} = 0$, $p_T^{b\text{-jet}} > 20$ GeV, $\|\eta^{b\text{-jet}}\| < 2.5$	
$N_{\text{veto leptons}}$	= 0	= 0	= 1 , $p_T > 15$ GeV
$m_{\ell\ell\ell}$	–	–	> 106 GeV
m_{jj}	> 500 GeV	$200 < m_{jj} < 500$ GeV	> 200 GeV
$\|\Delta y_{jj}\|$		> 2	

Figure 4: Summary of the event selection for the signal region and control region.

- N_{expected}: number of expected events
- σ: the theoretical cross-section of the process
- $\mathcal{L}$: the integrated luminosity of the data (for the full ATLAS Run 2 dataset, this is 139 fb^{-1})
- ϵ: the efficiency and acceptance. (the event pass rate)

we might be able to validate our results before applying the DNN training.

7 4/10

7.1 Background simulation

7.1.1 Simulation code

We generate the following background events by MadGraph 3.7.0 [4] and all of the PDF sets for each background simulation are selected to be `lhaid 260000`, which is `NNPDF30_nlo_as_0118`. For $W^{\pm}W^{\pm}jj$ QCD

```
generate p p > l+ vl l+ vl j j QCD=2 QED=4
add process p p > l- vl~ l- vl~ j j QCD=2 QED=4
```

For $W^{\pm}Z$ QCD

```
generate p p > l+ l- l+ vl j j QCD=0
add process p p > l+ l- l- vl~ j j QCD=0
```

For $W^{\pm}Z$ EW

```
generate p p > l+ l- l+ vl j j QCD=2 QED=4
add process p p > l+ l- l- vl~ j j QCD=2 QED=4
```

7.1.2 Results

Table 11: Comparison of signal region cross sections (in fb) between simulation results, Stange Thesis [1] (Table G.2), and ATLAS paper [2] (Table 3). Expected yields from the thesis and paper have been converted to cross sections using the integrated luminosity mentioned in both paper of 139 fb^{-1}.

Process	**MadGraph3.7.1 [fb]**	**Sherpa3 [fb]**	**Thesis [1][fb]**	**Paper [2][fb]**
Signal Processes ($W^{\pm}W^{\pm}jj$ EW)				
W_LW_L (LL)	0.132		0.145	–
W_LW_T (LT)	0.426		0.453	–
W_TW_T (TT)	0.847		0.922	–
sum	1.405	1.353	1.520	–
WW unpolarized	1.378		1.554	1.691
Background Processes				
WZ-EW	0.150	0.409	0.108	0.107
WW-QCD	0.093	0.138	0.175	0.173
WZ-QCD	0.375	0.252	0.554	0.597

The signal region cuts obey the summarized table 2 in paper [2], but adding the detail that lepton pairs should be in same sign. We can see from table 11 that for electroweak processes (both signal and background), MadGraph [4] simulated a great result which is consistence with the reference. But for QCD-induced results, MadGraph [4] does not generate results reasonably. Therefore, we will try to generate some background events from Sherpa[5] to examine the consistency of the result. We can also wee that for events generated by Sherpa[5], QCD-induced WW background behaves better than MadGraph, while MadGraph generates better for all electroweak events. However, there is still some inconsistency for the WZ-QCD induced background.

Table 12: Expected yields in the signal region (SR). The ATLAS results are shown alongside Thesis values and our simulation estimates using different generator versions, assuming an integrated luminosity of 139 fb^{-1}.

Process	MG 3.7.0	Sherpa 3.0.3	ATLAS	Thesis 9.1	Thesis G.2
$W^{\pm}W^{\pm}jj$ EW	195.31	188.03	235	206.52	211.29
$W^{\pm}W^{\pm}jj$ QCD	12.95	19.12	24	24.05	24.27
$W^{\pm}Zjj$ EW	20.81	56.98	14.9	14.95	15.07
$W^{\pm}Zjj$ QCD	52.13	34.99	83	82.75	77.0

We can also compare the result by the expected yields in the signal region shown in table 12. Which shoes the same trend that Sherpa generates better WW-QCD induced events than MadGraph, while MadGraph generates better electroweak processes ($WWjj$ and $WZjj$) than Sherpa.

However, both event generators exhibit low consistency for the $WZjj$-QCD induced process.

8 4/18

I took the following steps to try to generate results consistent with Feng-Yang's:

- **Cross section calculation for Sherpa:** Instead of extracting the nominal cross section from the `hepmc` file, we summed the cross sections for the positively charged processes (e.g., W^+) and the negatively charged processes (e.g., W^-), and then multiplied the total by the branching ratio.
- **Delphes `run_card`:** I initially used the ATLAS card but found that it lacked the fat jet finding configuration inside the `run_card` script. Therefore, I switched to the default `run_card` originally provided with Delphes and altered the jet radius to 0.4 to align with the paper.
- **Narrow width approximation (NWA) instead of the full process:** Because there were issues generating the full process in Sherpa (e.g., excessive integration time), we switched to using the narrow width approximation (NWA), which was also used for generating the signal process in the paper. Additionally, we adjusted the events generated by MadGraph to use NWA instead of the full process.

Table 13: Expected yields in the signal region (SR). The ATLAS results are shown alongside Thesis values and our simulation estimates using different generator versions, assuming an integrated luminosity of 139 fb^{-1}.

Process	MG 3.7.0	Sherpa 3.0.3	ATLAS	Thesis 9.1	Thesis G.2
$W^\pm W^\pm jj$ EW	201.89	169.26	235	206.52	211.29
$W^\pm W^\pm jj$ QCD	15.20	17.59	24	24.05	24.27
$W^\pm Zjj$ EW	21.94	32.05	14.9	14.95	15.07
$W^\pm Zjj$ QCD	50.84	78.35	83	82.75	77.0

Comparing this table with last week's, we observe that for MadGraph, the WW processes are now closer to the reference values. For Sherpa, the expected yield for the WW processes has decreased, which might be related to the different Delphes `run_card` used. For the WZ processes, Sherpa demonstrates better consistency than last week's results.

To make further comparisons with the results from the thesis or paper, we need to determine the exact Delphes `run_card` that was used. As our latest results indicate, the choice of `run_card` significantly affects the cross section, as shown in the following table:

Process	Expected Events (139 $\mathbf{fb}^{-1}$)	
	`delphes_card_default.tcl`	`delphes_card_ATLAS.tcl`
WWjj-EW-NWA	201.89	212.46
WWjj-QCD-NWA	15.20	15.86
WZjj-EW-NWA	21.94	24.82
WZjj-QCD-NWA	50.84	53.26

Table 14: Comparison of expected event yields across four processes, evaluated using the ATLAS Delphes card versus the Default Delphes card.

9 4/26

9.1 Summary of DNN Training Input Variables

Table 15 summarizes the variables used as inputs for the neural network training. To improve the training process for the DNN algorithms, some variables are pre-scaled to achieve a more Gaussian-distributed input.

The η values of leptons and jets can be used to define the lepton Zeppenfeld variable:

$$Z_l^\star = \left| \frac{\eta_l - \frac{1}{2}(\eta_{j1} + \eta_{j2})}{\eta_{j1} - \eta_{j2}} \right|. \tag{18}$$

This is a measure of how central a lepton is relative to the two tagging jets.

The definition of the massless early-projected transverse mass is:

$$m_{o1}^{ll,MET} = \sqrt{(p_T^{l1} + p_T^{l2} + p_T^{miss})^2 - |\vec{p}_T^{l1} + \vec{p}_T^{l2} + \vec{p}_T^{miss}|^2}, \tag{19}$$

This is an advanced kinematic approximation used to estimate system masses involving missing transverse momentum, assuming massless visible products.

And the mass-preserving transverse mass is defined by:

$$m_{\mathrm{T}}^{ll,\mathrm{MET}} = \sqrt{(E_{\mathrm{T}}^{ll} + p_{\mathrm{T}}^{\mathrm{miss}})^2 - |\vec{p}_{\mathrm{T}}^{l1} + \vec{p}_{\mathrm{T}}^{l2} + \vec{p}_{\mathrm{T}}^{\mathrm{miss}}|^2}, \tag{20}$$

Or generally, it can be defined for object x, for x being the leading lepton, sub-leading lepton, or the de-lepton system:

$$m_{\mathrm{T}}(x, E_{\mathrm{T}}^{\mathrm{miss}}) = \sqrt{(E_{\mathrm{T}}^{x} + E_{\mathrm{T}}^{\mathrm{miss}})^2 - |\vec{p}_{\mathrm{T}}^{x} + \vec{p}_{\mathrm{T}}^{\mathrm{miss}}|^2}, \tag{21}$$

where $E_{\mathrm{T}} = \sqrt{m^2 + |p_{\mathrm{T}}|^2}$.

9.2 Dataset separation

The training data is split into train validation, and test sets according to the event number in a ration of 3:1:1.

- The test set: ((EventNumber-i_{fold}) mod 5)=0

Kinematics	Description	Scaling
	Low-level variables	
$p_T^{\ell_1}$, $p_T^{\ell_2}$, $p_T^{j_1}$, $p_T^{j_2}$	p_T of the leading/subleading lepton/jet	$\log_{10}(x)$
η^{ℓ_1}, η^{ℓ_2}, η^{j_1}, η^{j_2}	η of the leading/subleading lepton/jet	x
ℓ_1 type, ℓ_2 type	Flavor of the leading/subleading lepton	x
$\phi^{\ell_2}-\phi^{\ell_1}$, $\phi^{j_1}-\phi^{\ell_1}$, $\phi^{j_2}-\phi^{\ell_1}$	Redefined ϕ of the subleading lepton/ leading jet/ subleading jet	x
p_T^{miss}	Missing transverse momentum	$\log_{10}(x)$
$\phi(p_T^{\text{miss}})-\phi^{\ell_1}$	Redefined ϕ of the missing transverse energy	x
	High-level variables	
$Z^\star_{\ell_1}$, $Z^\star_{\ell_2}$	Zeppenfeld variable of the leading/subleading lepton	$\sqrt{x}$
$m_T^{\ell_1,\text{MET}}$, $m_T^{\ell_2,\text{MET}}$	Transverse mass of the respective lepton and p_T^{miss}	$\sqrt{x}$
$\Delta R_{\ell\ell}$, ΔR_{jj}	ΔR separation between the two leptons / two jets	x
$\Delta\eta_{\ell\ell}$	Pseudorapidity separation between the two leptons	$\sqrt{x}$
$m_{\ell\ell}$, m_{jj}	Invariant mass of the dilepton / dijet system	$\log_{10}(x)$
$p_T^{\ell\ell}$	Transverse momentum of the dilepton system	$\sqrt{x}$
$m_T^{\ell\ell,\text{MET}}$	Transverse mass of the dilepton system	$\sqrt{x}$
$m_{o1}^{\ell\ell,\text{MET}}$	Early-projected massless invariant mass	$\sqrt{x}$
Δy_{jj}	Rapidity separation between the two jets	x
$\Delta\phi_{jj}$	Azimuthal separation between the two jets	x
$(p_T^{\ell_1}\cdot p_T^{\ell_2})/(p_T^{j_1}\cdot p_T^{j_2})$	p_T ratio of the lepton and jet systems	$\log_{10}(x+0.02)$
$\min(\Delta R_{\ell_i,j_k})$	Minimal ΔR separation between any lepton and jet	x

Table 15: Summary of DNN input variables and their pre-scaling functions.

- The validation set: ((EventNumber-$i_{\text{fold}}+1$) mod 5)=0
- The training set: the remaining events

The index i_{fold} defines the different splits used for cross-validation in the hyperparameter optimization.

9.3 Training process

The training processes are divided into two parts: First, an in inclusive network designed to separate the electroweak (EW) signal from all backgrounds. Second, the polarization-specific networks trained exclusively on the EW samples to distinguish specific polarization states. In details, it is trained to separate longitudinal single-boson polarization (LL and LT v.s. TT) or double-boson polarization (LL v.s. LT and TT) from the remaining transverse polarization states.

10 5/1

10.1 5-Fold Method

The k-fold method is a data-splitting and cross-validation strategy used to reliably evaluate neural network performance and prevent biases when making predictions.

10.1.1 Data Categorization

The dataset is divided into three distinct and independent categories, using the i_{Fold} index:

- Test dataset: 20% with `((EventNumber -` $i_{\texttt{Fold}}$`) mod 5) = 0`. Kept completely hidden from the training and optimization process to test the final model.
- Validation dataset: 20% with `((EventNumber -` $i_{\texttt{Fold}}$ `+ 1) mod 5) = 0`. Used to evaluate the network's performance and tune hyperparameter during training.
- Training dataset: 60%, which is the remaining events that are not test or validation datasets. Used to adapt and optimize the network's trainable parameters.

where the index $i_{\text{Fold}} \in [0, 1, 2, 3, 4]$ and independent of the physical properties of the events.

10.1.2 Cross-Validation and Application

- Training Multiple Networks: The process iteratively trains five separate networks. Each network uses a different overlapping combination of the training folds and evaluates against non-overlapping validation and test sets.
- Evaluating Performance: Training across different independent validation datasets allows the author to calculate a mean network performance and estimate its statistical uncertainty.
- Preventing Mismodeling Bias: Neural networks naturally perform better on the data they were trained on than on unseen data. If predictions evaluated on training data were compared against measured data, it would introduce a systematic bias.
- Final Unbiased Predictions: To guarantee unbiased predictions across the entire dataset, each event's final prediction is generated exclusively by the specific DNN that held that event in its unseen test dataset.

10.1.3 Explanation of Proper Nouns

- **Unseen data** in the context of machine learning and the $k-$fold method refers specifically to the test dataset, which is used to provide an objective test of how well the final model can generalize to new, real-world examples. It is data that is kept completely hidden from the neural network while it is learning and optimizing its parameters.

$$s_j = \sum_i x_i w_{ij} + \theta_j \tag{22}$$

- **Weights** determine the strength and importance of the connection between two neurons. Mathematically, a weight is a multiplier. The output of a previous neuron (x_i) is multiplied by its assigned weight (w_{ij}) before it enters the next neuron.

- **Bias** (θ_j) is an additional parameter added to the neuron, independent of the input data. Mathematically, bias acts as an offset. By adding a bias, the network shifts the activation function left or right, determining how high the sum of the weighted inputs needs to be before the neuron activates.

10.2 Network Architecture

Based on section 9.2 in the reference thesis [3], the neural networks used are exclusively fully-connected feed-forward neural networks. The architecture is structured as follows:

- Normalization Layer (Input): Applies a scaling function to the input variables, normalizing their distributions to have a mean of 0 and a variance of 1. This ensures all inputs have similar scales without loosing their kinematic phase space information.

- Repeating Hidden Sequence:

  1. Dense Layer: A standard fully-connected layer.
  2. Batch Normalization Layer: Used to normalize the output from the preceding dense layer.
  3. Activation Layer: Applies the activation function, specifically the Swish function defined as:

$$f_{\text{Swish}}(x) = \frac{x}{1 + e^{-x}} \tag{23}$$

     For a binary classification the Sigmoid function:

$$f_{\text{Sigmoid}}(x) = \frac{1}{1 + e^{-\beta x}} \tag{24}$$

  4. Dropout Layer: Introduced for regularization to prevent overtraining.

- Output Layer: The repeating sequence concludes with a final dense layer, which computes the ultimate output score of the network.

11 5/10

11.1 Ap plying Simulated Data to Real Measured Data

Based on the thesis, there are three primary methods and strategies used to bridge the gap between Monte Carlo (MC) simulated training data and real measured data.

1. Preventing Mismodeling Bias via the k-Fold Method

   Neural network naturally perform better on the data they were trained on than on unseen data. If a network's prediction on its familiar training data was directly compared to real measured data, it would introduce an artificial systematic mismodeling (bias). The detailed explanation of this strategy is in Section 5.8 and Section 12.2.

   To guarantee unbiased predictions across the entire dataset, each event's final classification score is generated exclusively by the specific dense-NN that that held that event in its completely unseen "test" dataset.

2. Validation Strategy using Control Regions

   Before deploying the neural network on the actual final measurement (the signal region), it is necessary to check if the simulation matches the real data. However, testing the dense-NN directly in the signal region could influence the final measurement and introduce a human "analysis strategy bias". The definition of the control region is in Section 6.3.2 and the results of the validation tests are in Section 10.2.

   Validation tests are safely performed in designated "control regions". These are phase spaces that are kinematically similar to the signal region but are background-only. By comparing the dense-NN output on real measured data vs. simulated data in these control regions, we can ensure the simulation matches reality before looking at the actual signal.

   *Run trained DNN on the simulated background data and real measured data in control region and plot a histogram of the output scores (0(Background) to 1(Signal)). If it behaves correctly, two histograms should stack perfectly on top of each other and should assign mostly low scores to both datasets.

3. Correcting Simulation Errors with Data-Driven Backgrounds

   Simulations are never perfect replicas of real life and fail to accurately predict certain detector artifacts, such as fake leptons or electrons flipping their charges. This approach is detailed in Section 7.2.2 and Section 7.2.3.

   Since MC simulation models these backgrounds very poorly, the analysis uses data-driven techniques. This involves extracting metrics like "fake factor" directly from real-world measured data to overwrite or correct the flawed simulation predictions.

In summary, they first train five independent model on perfectly labeled simulated data. Then, prove the simulation matches reality in a safe control region and fix the broken parts (like fake-leptons)of the simulation using data-driven factors. Finally, the measured data is strictly evaluated by the specific model that corresponds to that fold. (If the real `EventNumber % 5` equals 2, that event is evaluated only by Model 2.)

11.2 References to Compare with

The thesis [3] contains specific figures that are highly valuable for validating our script, which is listed as follow:

1. Input Variable Distributions

   In Appendix C, there are several figures of the pre-processed physical variables. We can plot these variables to verify the data parsing, variable calculations, and data shape match the thesis exactly.

2. Dense-NN Output Score Distributions

   Once our model is trained, we can run it on the test dataset. We can plot our output scores using an equidistant binning of exactly 20 bins, as this is the standard binning convention used in the thesis for optimizing and visualizing Neural Network outputs. We can compare the distribution of the final classification probability scores outputted by the Sigmoid layer of our trained Dense-NN with Section 10.2.

3. ROC-AUC Performance Curves

   We can compare our final testing AUC scores against the reported AUC scores in the thesis to confirm out network architecture and hyperparameters are performing at the expected efficiency, which can be found in Chapter 9.

11.3 ML Workflow

11.3.1 Data Pre-processing

Feng-Yang and me decided to use `parquet` as the dense-NN input variable file format. And the workflow should be as follows:

1. Extract Raw Data:

2. Calculate Variables: For each event, calculate all the low-level and high-level variables listed in Table 9.2 in the thesis [3].

3. Apply Pre-scaling: Apply the scaling mentioned in the thesis [3] in Table 9.2 to let the datas resembles a Gaussian distribution for models learning more effectively.

4. Assign Folds: Calculate the physical Event Number and use the formula:

$$((\text{Event Number} - i_{\text{Fold}}) \bmod 5) = 0 \tag{25}$$

   to tag each event with a fold number (0, 1, 2, 3, or 4).

5. Write to Parquet

12 5/17

12.1 Foundational ML Concepts

1. The **loss Curve** is a line graph that tracks how many "mistakes" the neural network is making overtime. In our script, the network uses Binary Cross-Entropy (BCE) loss, which is standard for binary classification. The lower the loss, the better the network is at distinguishing target from background.

2. An **epoch** is one complete cycle where the neural network has seen the entire training dataset exactly once. Because datasets in particle physics are massive, the data is fed to the network in smaller chunks called "batches." (If we have 10,000 events and a batch size of 1,000, it takes 10 batches to get through all the data. Once all 10 batches have passes through the network, one epoch is complete.)

   Reading it a second time (Epoch 2) helps understand and retain the material better, steadily lowering the loss.

3. **AUC** stands for area under the curve, and it specifically refers to the receiver operating characteristic (ROC) curve. The ROC-AUC evaluates the separation power of the model.

12.2 dense-NN Result

For the initial test, I am using 3592 events for the $\mathrm{DNN}_{W^{\pm}W^{\pm}}$ task and 6679 events for the $\mathrm{DNN}_{\mathrm{pol}}$ task. The whole set of input features are used for the training. And the dense-NN structure for the two model is specified as follow:

```
history_ww = train_5_fold(
        df=df_ww,
        features_cols=features_cols,
        num_layers=8,
        neurons_per_layer=491,
        dropout_rate=0.355,
        lr=0.002,
        model_name="DNN_WW"
    )

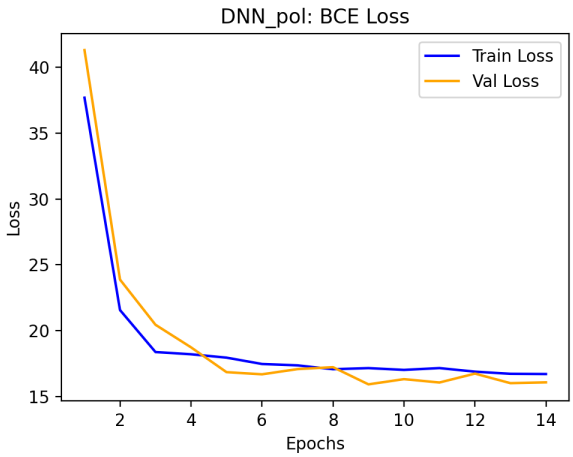
history_pol = train_5_fold(
        df=df_pol,
        features_cols=features_cols,
        num_layers=9,
        neurons_per_layer=296,
        dropout_rate=0.381,
        lr=0.0015,
        model_name="DNN_pol"
    )
```

Classification Task	Validation AUC	Test AUC
$\mathrm{DNN}_{W^{\pm}W^{\pm}}$	0.7654 ± 0.0107	0.7601 ± 0.0163
$\mathrm{DNN}_{\mathrm{pol}}$ (LL vs. TX)	0.7878 ± 0.0128	0.7865 ± 0.00083

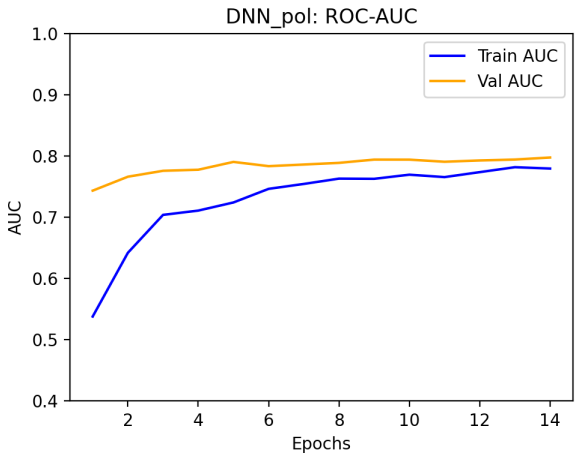
Table 16: Classification performance for DNN models using shared data.

Figure 5 illustrates the evolution of the loss and AUC curve. The quantitative performance of the models across the 5-fold ensemble is summarized in table 16.

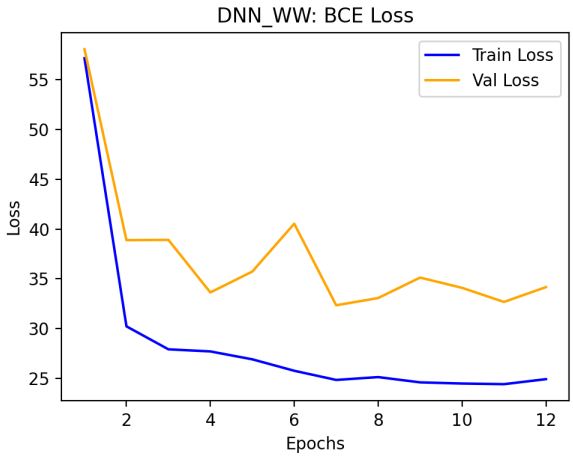
As anticipated for a preliminary test, the training curves indicate that the models—particularly DNN_WW—have not yet fully converged, as evidenced by the volatility in validation loss. The next steps will involve generating larger training samples to bridge the generalization gap, increasing the



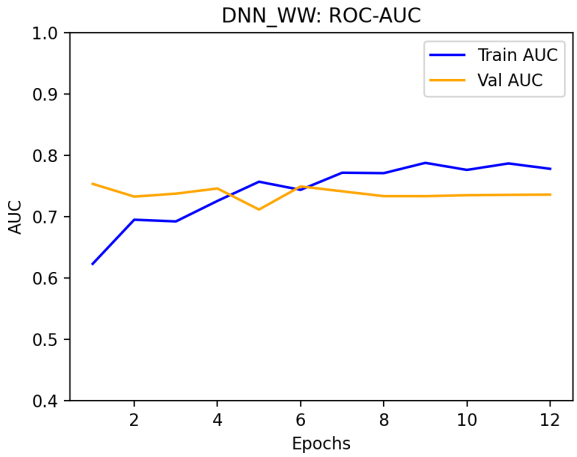
(a) $\mathrm{DNN}_{\mathrm{pol}}$: BCE Loss



(b) $\mathrm{DNN}_{\mathrm{pol}}$: ROC-AUC



(c) $\mathrm{DNN}_{W^{\pm}W^{\pm}}$: BCE Loss



(d) $\mathrm{DNN}_{W^{\pm}W^{\pm}}$: ROC-AUC

Figure 5: Model performance comparison: BCE Loss (left column) and ROC-AUC (right column) for $\mathrm{DNN}_{\mathrm{pol}}$ and $\mathrm{DNN}_{W^{\pm}W^{\pm}}$.

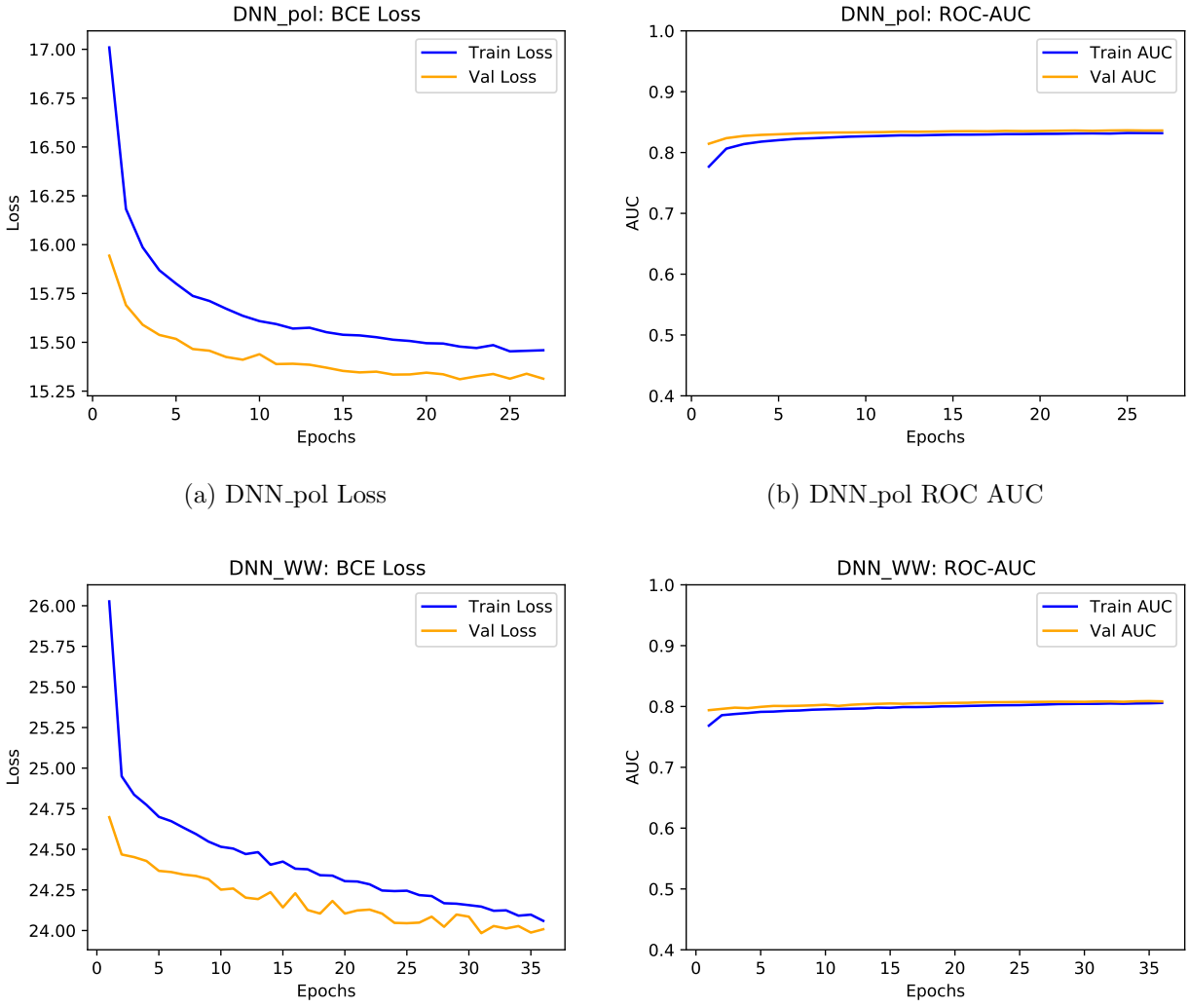
number of epochs to observe long-term stability, and testing optimized hyperparameters to improve the ROC-AUC plateau.

*Table 9.1 in thesis [3] outlines the Class Weights. By multiplying the signal by a massive number so that the signal event looks just as loud and important as the background events.

13 5/24

13.1 Performance of the Dense-NN Model

The performance for models using larger number of dataset are shown in figure 6 and table 17.



(a) DNN_pol Loss

(b) DNN_pol ROC AUC

(c) DNN_WW Loss

(d) DNN_WW ROC AUC

Figure 6: Performance metrics for DNN_pol and DNN_WW models.

- DNN_pol Model:
  - Loss: Both Training and Validation loss have decreased compared to last week's results, showing a more consistent downward trend. Last week, the training loss appeared

higher and less converged, while the current curves show a clearer stabilization as epochs progress.

- ROC AUC: The AUC values are showing healthier behavior. The model now converges to an AUC around 0.8 with less volatile gaps between the training and validation sets, suggesting better generalization than the previous week.

- DNN_$W^{\pm}W^{\pm}$ Model:
  - Loss: There is a notable improvement in the stability of the training loss curve. While last week's validation loss showed significant fluctuations, the current results indicate that the loss is better behaved and the model is finding a more stable minimum.
  - ROC AUC: The training and validation AUC curves have become more tightly coupled, which is a positive indicator that the training process is becoming more reliable and less prone to overfitting.

Classification Task	Validation AUC	Test AUC
$\text{DNN}_{W^{\pm}W^{\pm}}$	0.8067 ± 0.0013	0.8065 ± 0.0013
DNN_{pol} (LL vs. TX)	0.8383 ± 0.0013	0.8381 ± 0.0011

Table 17: Classification performance for DNN models using 1 million dataset

Table 17 also shows higher AUC values for both the validation and testing for $\text{DNN}_{W^{\pm}W^{\pm}}$ and DNN_{pol} compared to last week's result.

13.2 Code Explanation

- `model.train()` is used to switch the model to train mode, which will cause layers such as Dropout (randomly dropping neurons) and BatchNorm (batch normalization) to operate as they did during training. (e.g., Dropout will initiate random dropping, and BatchNorm will use the statistic of the current batch and update the internal mean/variance). Calling `model.train()` during training ensures these layers work correctly; during validation or testing, `model.eval()` should be used instead.

- `optimizer.zero_grad()`: Clears the accumulated gradients on the model parameters. Each time `loss.backward()` is executed, PyTorch adds the computed gradients to the parameter's .grad file; if `zero_grad()` is not called first, the new gradients will be superimposed on the old gradients, leading to incorrect parameter updates. A typical training flow is:
  1. `optimizer.zero_grad`: clears gradients.
  2. `outputs = model(inputs)`: forward propagation.
  3. `loss = criterion(outputs, targets)`: calculates loss.
  4. `loss.backward()`: backward propagation, specifying `.grad`.
  5. `optimizer.step()`: updates parameters.

- `criterion = nn.BCELoss(reduction='none')`: returning a per-sample loss tensor. Binary cross-entropy for one sample (prediction p, label $y \in \{0, 1\}$) is:

$$\text{BCE}(p, y) = -y \log p - (1 - y) \log(1 - p) \tag{26}$$

`reduction='none'` means the loss call returns an elementwise tensor of shape (batch,1) so the code can multiply each sample's loss by its event weight and then aggregate manually.

- Usage of `Weight`:
  1. Multiplying the per-sample loss by `Weight` makes the training objective care more about heavy-weight events and less about light-weight events. In other words, the model is trained to minimize the weighted expected loss, not the plain unweighted average.
  2. `roc_auc_score(y_true, y_pred, sample_weight=weights)`: Weighted ROC-AUC means each sample contributes to the ROC curve proportionally to its weight.

14 5/31

14.1 Initial State and Final State Polarization

In section 3.3.3 in the thesis [3], they mentioned that at high center-of-mass energy, the final state $W_{\mathrm{L}}^{\pm}W_{\mathrm{L}}^{\pm}$ originates almost exclusively from the $W_{\mathrm{L}}^{\pm}W_{\mathrm{L}}^{\pm}$ initial state. Other $VV \to V_{\mathrm{L}}V_{\mathrm{L}}$ processes are significantly affected by helicity flips, but for same-charged $W^{\pm}W^{\pm} \to W^{\pm}W^{\pm}$ scattering at the high center-of-mass energy achieved at the LHC, the polarization of the final state is similar to the initial state polarization. Therefore, the measurement of $W^{\pm}W^{\pm} \to W_{\mathrm{L}}^{\pm}W_{\mathrm{L}}^{\pm}$ at the LHC is almost equivalent to the measurement of $W_{\mathrm{L}}^{\pm}W_{\mathrm{L}}^{\pm} \to W_{\mathrm{L}}^{\pm}W_{\mathrm{L}}^{\pm}$.

14.2 Paper [6]

14.2.1 The Dataset and Goal

- Process: parton-level simulations of Vector Boson Fusion: $uu \to W^+W^-uu$.
- Decay: fully hadronic decays: $W^+ \to u\bar{d}$ and $W^- \to \bar{u}d$.
- The Classification Goal: theory-informed tagger to identify the LL polarization final states against the background combinations: LT, TL, and TT.

14.2.2 The Methodology

The author explicitly designs the system so that the machine learning model does not act as a black-box classifier.

1. Agnostic Training:

   The reinforcement learning algorithm is trained on an unpolarized sample of the W^+W^-uu events. During this phase, the reward function is a mixture of all four matrix elements. The agent learns the optical particle assignments completely blind to the actual WW polarizations.

2. Interpretable Mapping:

   The sole job of the neural network is to predict the correct mapping between the observed final-state particles and the partons entering the matrix element.

3. Theory-Informed Scoring:

   Once the network outputs the optimal particle mapping, the classification score is not generated by the neural network. Instead, it is computed directly using the exact analytic matrix elements for the different polarizations.

The optimal classification score is defined as a likelihood ratio:

$$L(\tilde{x}_i) = \frac{m_{LL}(\tilde{x}_i)}{m_{LT}(\tilde{x}_i) + m_{TL}(\tilde{x}_i) + m_{TT}(\tilde{x}_i)} \tag{27}$$

Because this score is computed directly from exact tree-level amplitudes, it is fully interpretable and respects all physical symmetries of the process.

14.2.3 The Results

The paper demonstrates that this theory-informed tagger performs extremely well without relying on traditional black-box classification:

- Mass Reconstruction:

  Using the mappings predicted by the neural network, the reconstructed invariant masses of the W bosons are in very good agreement with the true mass distribution (shown in Figure 7 of the paper).

- Tagging Performance (ROC/AUC):

  The classifier's performance for isolating the LL signal was evaluated using ROC curves (Figure 8):

  - Background consists of TT only events: AUC = 0.925
  - Background consists of full mix of LT, TL, and TT events: AUC = 0.858

The author concludes that this approach works highly effectively in practice because RL algorithm learns the correct topological assignments without needing prior knowledge of the polarization states.

15 6/5

15.1 Chapter 12 in Thesis

15.1.1 Machine Learning Output and Binning

In the search for the longitudinal $W^{\pm}W^{\pm}$ scattering signal, the analysis first relies on Deep Neural Networks (DNNs) to separate the signal from the background. The output is sorted into bins (e.g., Bin 1 to Bin 10), where higher bins are "highly likely to be Signal." In Chapter 12 of the thesis, this corresponds to the two-dimensional bin optimization algorithm designed to maximize the expected significance by isolating regions with the highest $W^{\pm}W^{\pm}jj$-EW purity.

15.1.2 Theoretical Expectations and Null Hypothesis

Before unblinding the data, theoretical expectations for the signal and background must be established.

- **Expectations:** The number of expected events is calculated using the sum of weights, which accounts for the base cross-section, lepton efficiencies, trigger efficiencies, and kinematic corrections.

- **Null Hypothesis:** The null hypothesis assumes that no longitudinal signal exists, setting the signal strength parameter to zero ($\mu_L = 0$). Under this hypothesis, the expected yield consists purely of the background expectation, subject to statistical fluctuations ($\pm\sqrt{\text{Background}}$).

15.1.3 Unblinding and the Fit Algorithm

Upon "unblinding," the real measured data in the signal region is revealed. To extract the signal significance, a "Fit Algorithm" is applied. Chapter 12 defines this as a *profile likelihood fit*. The fit adjusts the expected distributions to the measured data by varying the normalizations ($\mu_L, \mu_T, \mu_{WZ\text{-QCD}}$) and constraining systematic uncertainties via nuisance parameters (θ, γ).

15.1.4 Significance and Realities

A simplified significance formula:

$$Z = \frac{\text{Real Data} - \text{Background Expectation}}{\sqrt{\text{Background Expectation}}} \tag{28}$$

In the thesis, this is calculated using an asymptotic formula based on the profile likelihood ratio test statistic q_0, where the discovery significance is $Z_0 = \sqrt{q_0}$.

For example, there might two possible outcomes (Realities):

- **Reality A (Low Significance):** $Z = 0.5\sigma$, indicating just a background fluctuation with no signal found. This closely matches the actual observed result for the fully longitudinal (LL) polarization state in the WW-cmf, which yielded an observation significance of only 0.082σ due to a statistical fluctuation in the data. Meaning no signal is found.
- **Reality B (High Significance):** $Z = 5\sigma$, indicating a successful signal discovery. While the thesis did not reach the 5σ discovery threshold, the measurement for the combined longitudinal and transverse (LX) state achieved an observed significance of 3.4σ, providing strong evidence for the process.

15.2 DNN Score Distribution

15.2.1 The Corrected Weighting Method

- **Absolute Weights**: The thesis [3] states: "To avoid negative loss values in the BCE loss calculation, the absolute values of the Monte Carlo event weights $|\omega_{\text{MC},i}|$ are used for the training."
- **Class Balancing**: Table 9.1 in the thesis states: "In the training, the signal is scaled to the background yield." The logic `scaling_factor = sum_bkg / sum_signal` and multiplying it solely to the signal events.

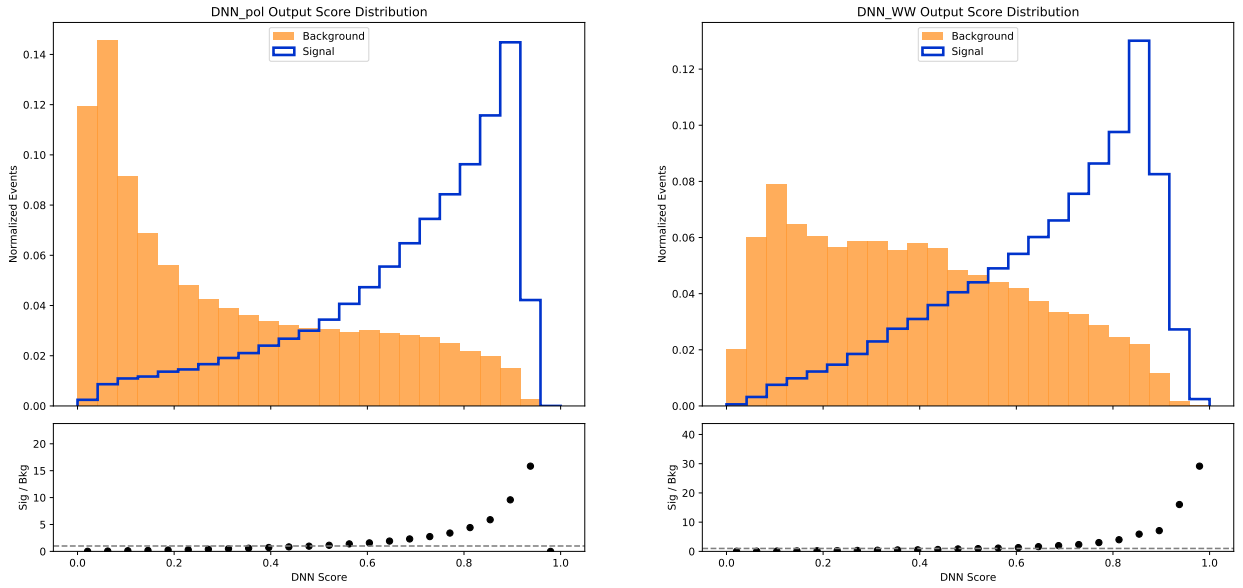
15.2.2 Result

Both neural networks are showing strong predictive capability. According to figure 7a and figure 7b, they both demonstrate a clear, healthy separation between the Signal and Background distributions. The networks have successfully learned the kinematic features necessary to push background events toward a score of 0.0 and signal events toward 1.0.

16 6/20

16.1 Discovery Significance and 95% CL Upper Limit

To establish a more comprehensive comparison with the reference thesis, the expected discovery significances (Z_0) and the expected 95% confidence level (CL) upper limits were also calculated.



(a) Polarization score distribution

(b) WW score distribution

Figure 7: DNN score distributions for polarization and WW processes.

16.1.1 Theory and Methodology

1. **The Binned Likelihood Function**

   Before calculating any significance, the analysis constructs a likelihood function, $\mathcal{L}$. This function describes the probability of observing the data given a specific signal strength (μ) and a set of nuisance parameters (θ):

$$\mathcal{L}(\mu, \theta) = \prod_{j=1}^{N_{\text{bins}}} \frac{(\mu s_j(\theta) + b_j(\theta))^{n_j}}{n_j!} e^{-(\mu s_j(\theta) + b_j(\theta))} \prod_{k=1}^{N_{\text{sys}}} C(\theta_k) \tag{29}$$

   - μ: The signal strength modifier ($\mu = 1$ is the Standard Model expectation, $\mu = 0$ is the background-only hypothesis).
   - $s_j(\theta)$ and $b_j(\theta)$: The expected signal and background yields in bin j, which vary based on the nuisance parameters.
   - n_j: The actual observed data events in bin j.
   - $C(\theta_k)$: The constraint functions (usually Gaussian or log-normal distributions) that restrict the nuisance parameters within their known systematic uncertainties.

2. **The Profile Likelihood Ratio**

   To test different hypotheses, the likelihood function is converted into a **Profile Likelihood Ratio** (Λ). This isolates the signal strength μ by "profiling out" (maximizing over) all the nuisance parameters:

$$\Lambda(\mu) = \frac{\mathcal{L}(\mu, \hat{\hat{\theta}})}{\mathcal{L}(\hat{\mu}, \hat{\theta})} \tag{30}$$

- **Denominator** ($\mathcal{L}(\hat{\mu}, \hat{\theta})$): The unconditional maximum likelihood. It is found by letting *both* μ and θ float to whatever values best fit the data.
- **Numerator** ($\mathcal{L}(\mu, \hat{\hat{\theta}})$): The conditional maximum likelihood. It is found by locking μ to a specific test value (like 0 for a discovery test) and letting only θ adjust to best fit the data.

3. **Calculating Discovery Significance (Z_0)**

   To claim a discovery, the analysis tests the background-only hypothesis ($\mu = 0$).

   The Test Statistic (q_0):

$$q_0 = \begin{cases} -2 \ln \Lambda(0) & \text{for } \hat{\mu} \geq 0 \\ 0 & \text{for } \hat{\mu} < 0 \end{cases} \tag{31}$$

   (The condition $\hat{\mu} < 0$ is set to zero because a downward fluctuation of background should not be counted as evidence for a new particle).

   The p-value and Significance: Using asymptotic formulas (Wilks' theorem), the probability of the background fluctuating to mimic the signal (the p-value) is calculated from q_0. The discovery significance (Z_0) is then extracted using the inverse of the standard Gaussian cumulative distribution function (Φ^{-1}):

$$Z_0 = \Phi^{-1}(1 - p_0) \approx \sqrt{q_0} \tag{32}$$

4. **Calculating the 95% CL Upper Limit ($\mathbf{CL}_s$ Method)**

   Because the observed significance for the longitudinal scattering was too low, the analysis transitions to setting an upper limit on μ. ATLAS uses the **$\mathbf{CL}_s$ method** to prevent setting artificially tight limits in regions where the background simply fluctuated downwards.

   The Test Statistic for Limits (q_μ): To test a specific signal strength μ, a different test statistic is used:

$$q_\mu = \begin{cases} -2 \ln \Lambda(\mu) & \text{for } \hat{\mu} \leq \mu \\ 0 & \text{for } \hat{\mu} > \mu \end{cases} \tag{33}$$

   The $\mathbf{CL}_s$ Calculation: The algorithm calculates two p-values based on q_μ:

   - p_μ: The probability of the signal plus background hypothesis being compatible with the data.
   - $1 - p_b$: The probability of the background-only hypothesis being compatible with the data.

   The CL_s value is their ratio:

$$\mathrm{CL}_s = \frac{p_\mu}{1 - p_b} \tag{34}$$

5. **The Final Limit:**

   To find the 95% Confidence Level upper limit (μ_{95}), computational solvers (like RooFit/MINUIT) iteratively scan and test different values of μ until they find the exact signal strength where:

$$\mathrm{CL}_s = 0.05 \tag{35}$$

   This process requires intensive numerical minimization across thousands of dimensions (one for every bin and systematic uncertainty), which is why it is far more complex than a standard algebraic formula.

16.1.2 Results

We can compare these findings with Table 12.4 and Table 12.9 in the thesis. The compiled results are presented in Table 18.

	Expected Significance (Z_0)		Expected 95% CL Upper Limit	
	Thesis	My Model	Thesis	My Model
LL (μ_{LL})	$1.512\,\sigma$	$1.4929\,\sigma$	2.273	1.3769
LX (μ_{LX})	$4.557\,\sigma$	$4.3881\,\sigma$	—	—

Table 18: Expected discovery significances (Z_0) and expected 95% CL upper limits for the LL and LX polarization states in the WW center-of-mass frame.

As shown in Table 18, the expected discovery significances derived from the current model closely track the thesis baseline for both the longitudinal-longitudinal (LL) and longitudinal-transverse (LX) polarization states, yielding $1.49\,\sigma$ and $4.39\,\sigma$ respectively.

Our model establishes an expected 95% CL upper limit for the LL signal strength of 1.38, compared to the thesis limit of 2.27. The primary reason our limit is significantly tighter is that our calculation is currently a 'Stat-Only' fit (including only a flat 10% background normalization uncertainty). The thesis limit of 2.27 was derived using a 'Full Systematics' fit that profiled hundreds of real-world detector uncertainties (such as Jet Energy Scale, Muon Tracking resolution, and fake-lepton background variations) across the binned kinematic distributions. Because our neural network is currently unhindered by these complex detector systematics, the profile likelihood fitter operates with much higher confidence, resulting in a mathematically more stringent limit. This proves our baseline DNN kinematics are separating the shapes correctly, and the limit will naturally inflate to match the thesis once full detector systematics are applied.

16.2 SPANet (Symmetry-Preserving Attention Networks)

SPANet [7, 8] is a deep learning architecture designed specifically to solve the jet assignment problem.

16.2.1 The Jet Assignment Problem

In collider physics, when heavy particles are created, they rapidly decay into variable-size sets of observed particles or jets. When processing large-scale physical simulations, extracting signals from background noise requires correctly matching these detected final-state jets back to their parent particles.

- Event reconstruction has traditionally been performed by evaluating every possible permutation of jet assignments. However, the combinatorial background scales exponentially with the number of final-state jets—a particularly acute challenge when analyzing complex multi-jet signatures.
- Furthermore, these arbitrary ordering methods ignore the natural physical symmetries inherent in the decays.

16.2.2 How it Works

1. **Data Representation: The Permutationless Input**

   Traditional algorithms require jets to be ordered arbitrarily before being fed into the system, creating artificial sequence dependence. SPANet eliminates this by treating an event as an unordered point cloud.

   - **The input:** An event is represented as a set of N jets. Each jet is a feature vector containing kinematic and qualitative data (like b-tagging scores), forming a matrix $X \in \mathbb{R}^{N \times D}$.
   - **Event-level encoding:** SPANet passes this matrix through a Transformer encoder using multi-head self-attention. This translates the raw data into a contextualized mathematical representation.

2. **Symmetric Tensor Attention (STA)**

   Once the jets are encoded, the network must assign them to parent particles. Instead of outputting a single jet at a time, SPANet generates a joint probability distribution over all possible combinations using STA. STA outputs a rank-k tensor $P \in \mathbb{R}^{N \times N \times \cdots \times N}$, where $P(j_1, \ldots, j_k)$ represents the network's predicted probability that a specific combination of jets originates from the parent particle.

   Crucially, STA enforces jet symmetry by ensuring the learnable weight tensors are commutative across the symmetric axes, such that $P(j_1, j_2) = P(j_2, j_1)$.

3. **Global Event Topology: Combined Symmetric Loss (CSL) Function**

   While STA handles symmetries within a single particle's decay, CSL manages symmetries between identical parent particles.

   During training, the network calculates the prediction loss for all valid permutations of identical parent particles. It then applies the minimum achievable loss and updates the weights. This prevents the network from penalizing itself for guessing the correct jet assignment but arbitrarily swapping the labels of two indistinguishable parent particles.

4. **Final Output**

   For every expected particle defined in the event topology, SPANet produces two distinct outputs:

   - **Detection Probability (DP):** A scalar value indicating the network's confidence that the parent particle is fully reconstructible within the event.
   - **Assignment Distribution (AD):** The rank-k tensor generated by the STA layer.

Once the DP crosses a certain threshold, the network extracts the coordinates of the peak value from the AD tensor. Those coordinates represent the final, physically-symmetric jet assignment for that particle.

17 6/28

17.1 Fully Hadronic Process

We found a thesis: Precision Measurements of Vector Boson Scattering Production and Searches for Charged Higgs Bosons at the Large Hadron Collider [9] that is related to the "fully hadronic" process. The signal topologies and categorization considered in the analysis:

- One leg targets the $V \to qq$ ($V = W, Z$) decay in both the boosted and resolved decay products.
- The other leg targets three different decay modes:
  - Invisible: $Z \to \nu\nu$
  - Boosted $Z \to b\bar{b}/c\bar{c}$
  - Boosted $V \to q'q'$

  where q denotes a quark of any possible type, while q' for light quarks (usd) only.

Considering both of the two, total of 6 categories are supposed to be identified. However, due to the overwhelming of QCD multijet background, the two all-visible resolved ($V_r Z_b \to qq + b\bar{b}/c\bar{c}$ and $V_r V_b \to qq + q'q'$) categories are not considered. Only four categories are considered which are defined as follow:

- Boosted-Boosted (B-B): events with two boosted V bosons both decaying to two light quarks"

$$VV \to q'q' + q'q'$$

- Boosted-Invisible (B-MET): event with one invisibly decaying Z boson and one boosted Z boson decaying into two quarks:

$$ZV \to \nu\nu + qq$$

- Resolved-Invisible (R-MET): events with one invisibly decaying Z boson and one resolved V boson decaying to two quarks.

$$ZV \to \nu\nu\ qq$$

- Boosted-WithB (B-Btag): events with two boosted V bosons where at least one is a Z boson decaying to heavy-flavor quarks.

$$VZ \to qq + b\bar{b}/c\bar{c}$$

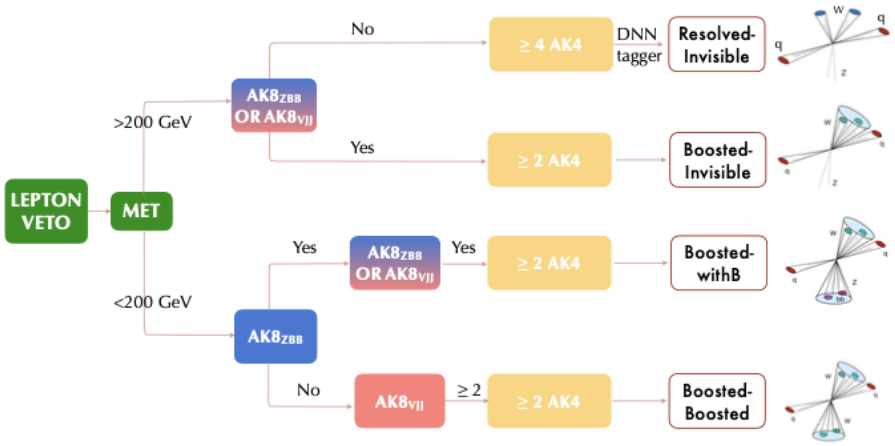


Figure 8: The event selection flow of the four categories mentioned in the thesis.

- AK8$_{\text{ZBB}}$: This is an AK8 fat jet specifically identified as a Z boson decaying into heavy-flavor quarks ($b\bar{b}$ or $c\bar{c}$). It requires specific heavy-flavor tagging criteria.
- AK8$_{\text{Vjj}}$: This is an AK8 fat jet identified as a Vector Boson (W or Z) decaying into light quarks ($q'\bar{q}'$).

In figure 8, it starts with a lepton veto because the specific analysis targets fully hadronic ($VV \to qqqq$) and semi-hadronic/invisible ($ZV \to \nu\nu qq$) processes, genuine signal events should not contain visible leptons. The first branch is to identify if it exists invisible neutrinos. The DNN tagger in the figure for R-MET is to identify vector bosons from forward jets. After all these event selections are completed, they are forward fed into different structure of DNN model to output the significance.

Variable	Selection requirements
Lepton veto	e, μ $p_{\text{T}} > 10$ GeV, $\lvert\eta\rvert < 2.4$
	τ_h $p_{\text{T}} > 18$ GeV, $\lvert\eta\rvert < 2.3$
b quark veto	Additional with $p_{\text{T}} > 20$ GeV
$p_{\text{T}}^{\text{miss}}$	B-B, B-Btag: < 200 GeV
	B-MET, R-MET: > 200 GeV
Diboson	Object selections in Table 5.17 [9] + "B-B": $p_{\text{T}}^{\text{leading AK8}} > 400$ GeV
	B-B, B-Btag: $m_{\text{VV}} > 500$ GeV
	B-MET, R-MET: $m_{\text{T}} > 500, 250$ GeV
Forward jets	$N_{\text{jets}}(p_{\text{T}} \geq 50 \text{ GeV}, \lvert\eta\rvert < 4.7) \geq 2$
	$m_{\text{jj}} > 500$ GeV
	$\lvert\Delta\eta_{\text{jj}}\rvert > 2.5$
	$\eta^{\text{j1}}\eta^{\text{j2}} < 0$
Event p_{T} imbalance	B-MET, R-MET: $\lvert\vec{p}_{\text{T}}^{\text{tot}}\rvert / \sum_i p_{\text{T}}^i < 0.25$

Table 19: Summary of the requirements used to select the VBS signal events.

The full requirements used to select the VBS signal events in all categories are summarized in Table 19. These selections are reported to greatly suppress the non-VBS background.

7/17

18 Particle Transformer

I summarized some worth learning contents from the original paper [10] for Particle Transformer (ParT).

18.1 Input

There are two kind of input mentioned in the paper [10]: particle inputs and interaction inputs.

- Particle Inputs $X_{\text{particle}} \in \mathbb{R}^{N\times C}$: where C is the features per particle. It usually consists of:
  - kinematics: energy, momentum
  - particle identification: charge, particle type
  - trajectory displacement: whether the trajectory deviates from the collision point
- Interaction Inputs $X_{\text{interaction}} \in \mathbb{R}^{N\times N\times C'}$ is the features per paired particles.
  - Angular separation Δ:
$$\Delta = \sqrt{(y_a - y_b)^2 + (\phi_a - \phi_b)^2} \tag{36}$$
  - Relative transverse momentum k_T:
$$k_T = \min(p_{T,a}, p_{T,b})\,\Delta \tag{37}$$
  - Momentum sharing variable Z:
$$z = \frac{\min(p_{T,a}, p_{T,b})}{p_{T,a} + p_{T,b}} \tag{38}$$
  - Pair invariant mass squared:
$$m^2 = (E_a + E_b)^2 - \|\mathbf{p}_a + \mathbf{p}_b\|^2 \tag{39}$$

18.2 Neural Network

$$f_\theta(\text{input}) = \text{output} \tag{40}$$

where θ is the parameters/weights that the machine learns and adjust during training.

18.3 Embedding

To make initial particle features $(p_{\text{T}}, \eta, \phi, E, \text{charge...})$ into a machine learnable vector $x_i^0 \in \mathbb{R}^d$ Therefore, for the whole jet, the embedded input particle features should look like:

$$x^0 = \begin{pmatrix} x_1^0 \\ x_2^0 \\ \vdots \\ x_N^0 \end{pmatrix} \in \mathbb{R}^{N\times d} \tag{41}$$

And for input interaction features:

$$U \in \mathbb{R}^{N\times N\times d'} \tag{42}$$

18.4 Attention

To understand particle i, what other particles we should also be pay attention to? To figure out this, we define the weight α_{ij}, such that the particle embedding becomes:

$$x_i^{\text{new}} = \sum_j \alpha_{ij} V_j \tag{43}$$

where V_j is the information particle j provides.

The transformer than transfer the particle embedding x_i into three kinds of vectors:

- Query projection:
$$Q_i = W_Q x_i \tag{44}$$
"what am I finding? "
- Key projection:
$$K_i = W_K x_i \tag{45}$$
"what label do I have, can I match with you? "
- Value projection
$$V_i = W_V x_i \tag{46}$$
"If you choose me, what can I provide? "

The value for Query-Key dot product is proportional to the importance of particle j to understand particle i.

$$QK^T \tag{47}$$

18.5 Model Structure

- Particle inputs go through the embedding layer to transfer the original particle features into vector x_i^0 for event i.
- Interaction inputs go through another embedding layer to transfer into vector U.
- x^0 go through L particle attention blocks and becomes x^L, while U_{ij} are the interaction bias.
  - Three linear input in the beginning of particle attention block, outputting Query $Q = xW_Q$, Key $K = xW_K$, and Value $V = xW_V$.
  - The MatMul layer acts as QK^T: the particle pair compatibility.
  - The scale layer divides the value by $\sqrt{d_k}$ to prevent scores from saturation.
  - Mask layer tells the model the zero-padding items are not real particle information.
  - LN (linear norm) layer stabilizes the value for different features.
  - $\bigoplus$ (residual connection): $x' = x + F(x)$ to prevent raw information from easily vanishing within deep networks.
- x_L concatenate with class token (CL) x_{class}, where CL acts as the summarizer for the all the input features.
- Concatenated vector go through two class attention blocks.

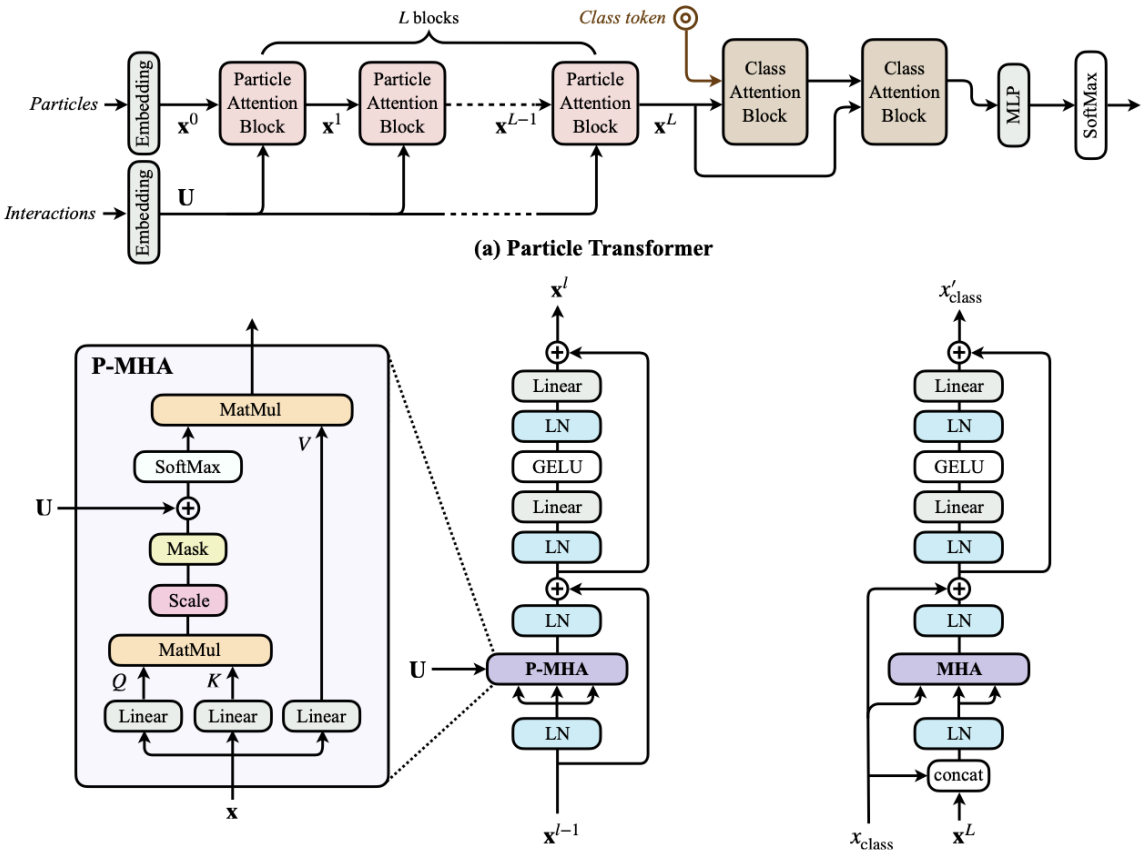


Figure 9: The original model structure in the paper [10]. (a) Overview of the structure of Particle Transformer. (b) Details workflow for particle attention block. (c) Details for class attention block.

- MLP (multi-layer perceptron, linear – GELU –linear): GELU acts as the activation function letting the model to have non-linear patterns.
- Final SoftMax outputs the final score for the probability of different kinds of event type it might be.

$$\text{Attention} = \text{softmax}\left(\frac{QK^T}{\sqrt{d}} + U\right) V_{\text{value}} \tag{48}$$

In conclusion, attention blocks concentrate on how different particles exchange information. While class attention blocks only care about how the class token connects with x_{class} and x_L. And at the end, outputting the final class token x_{class}.

19 Delphes Outputs

19.1 Collider Detector

Inside the collider: collision point – tracker – ECAL – HCAL – muon system

1. Tracker: Charged particles in magnetic field leaving a series of hits and trajectory. Tracker record them and reconstruct them and use particle-flow algorithm to transfer them into particle candidates.
2. Calorimeter: Particles interact with CAL, exciting secondary particles to shower, detectors find record them and work backwards to figure out the energy of the original particle.
    - Electromagnetic Calorimeter (ECAL): Uncharged particles, such as photons, have no trajectories recorded in the tracker. Particles who have electromagnetic interactions will be detected by ECAL. Electrons, Positrons, and Photons are the main objects. The shape of electromagnetic shower is usually more concentrated and short, also having better resolution.
    - Hadronic Calorimeter (HCAL): Uncharged particles, such as neutral hadrons, have no trajectory in tracker. Particles who have strong interactions can be recorded by HCAL. These particles have strong interaction with the nucleus of the material in HCAL, which then excites hadronic shower. Hadronic showers usually have irregular width and depth, while the resolution is worse compare to electromagentic shower.

19.2 EFlow Constituents

- `EFlowTrack`: detected by a tracker.
- `EFlowPhoton`: detected by a ECAL.
- `EFlowNeutralHadron`: detected by a HCAL.

19.3 Detector Level

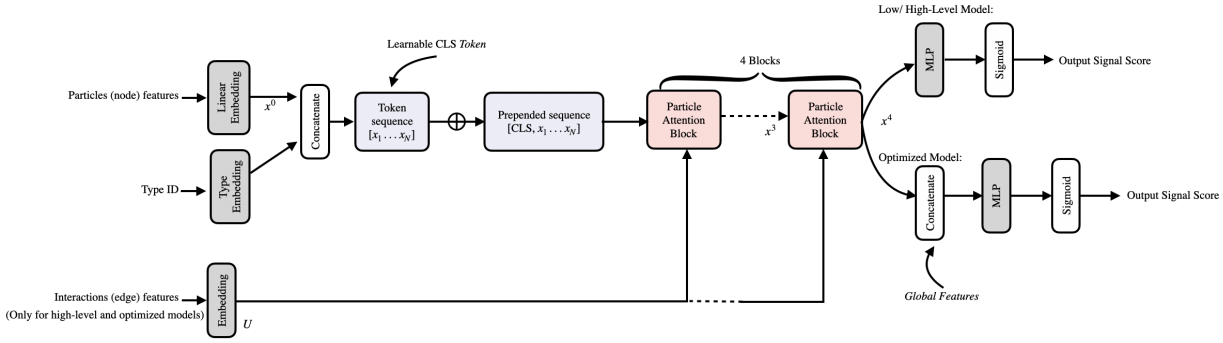
Delphes ROOT file doesn't just store the final layer; it saves multiple layers within the same event.

- Event
    - Low/detector-like collections

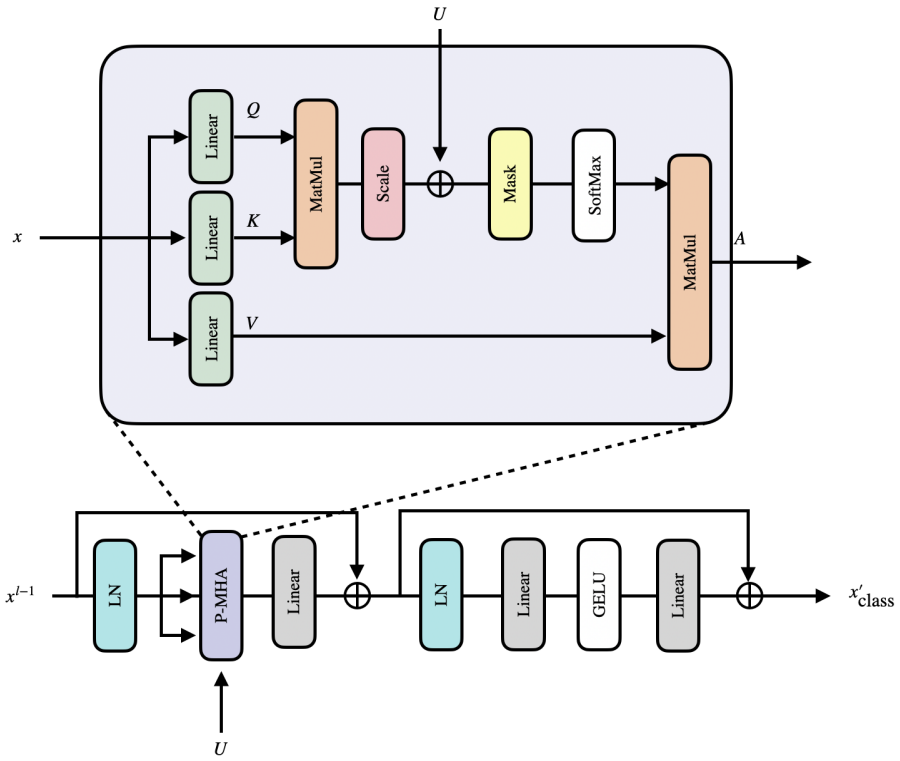
    - Track
    - Tower
    - EFlowTrack
    - EFlowPhoton
    - EFlowNeutralHadron
- Reconstructed physics objects
    - Electron
    - Muon
    - Jet
- event-level quantity
    - MET

20 Our Model for ParT

20.1 Model Structure



(a) Overall architecture of the modified Particle Transformer.



(b) Structure of the particle attention block and the incorporation of the pairwise interaction embedding U.

Figure 10: Architecture of the modified Particle Transformer used in this analysis: (a) the complete classification model and (b) the internal structure of a particle attention block.

As shown in Figure 10, our current model is an adapted Particle Transformer to classify event-level, instead of single-jet level. Original ParT set every constituents as particle sequence, we set every reconstructed objects or detector-level constituents as token sequence in each VBS event. Every token

represents a object, such as lepton, jet, MET, or EFlow constituents. The model uses self-attention to learn the kinematic relation, summarize them in CLS token, and finally output the binary signal score. Compare with the original ParT paper [10], we remain the `node/object embedding + type embedding + pairwise attention bias + Transformer attention blocks`.

20.2 Input Features

Table 20: Summary of ParT representations, object shapes, and mathematical feature sets across low, high, and optimized modes.

Mode	Input objects	Node shape	Node features	Edge shape	Edge features	Global shape	Global features

`low`	`EFlowTrack`
	`EFlowPhoton`
	`EFlowNeutralHadron`
		128×4	$\log_{10}(p_{\mathrm{T}}$ or $E_{\mathrm{T}})$
			η, ϕ
			$\log_{10}(m)$
			type embedding
				$128 \times 128 \times 1$	None	5	None

`high`	`lep1, lep2`
	`jet1, jet2`
	`MET`
		5×4	$\log_{10}(p_{\mathrm{T}})$, η
			$\Delta\phi$ relative to leading ℓ
			$\log_{10}(m)$
			type: ℓ / jet / MET
				$5 \times 5 \times 3$	$\Delta\eta$
					$\Delta\phi$
					ΔR
						5	None

`opt`	`lep1, lep2`
	`jet1, jet2`
	`MET`
	`EFlow const.`
		53×5	$\log_{10}(E_{\mathrm{T}})$, η
			$\Delta\phi$ relative to leading ℓ
			$\log_{10}(m)$, $\log_{10}(E)$
			type embedding
				$53 \times 53 \times 6$	$\Delta\eta$, $\Delta\phi$, ΔR
					$\log_{10}(m_{\mathrm{pair}})$
					$\log_{10}(p_{\mathrm{T,pair}})$
					$\log_{10}(m_{\mathrm{T,pair}})$
						5	$\log_{10}(m_{jj})$
							$\Delta\eta_{jj}$
							$Z^*_{\ell_1}$, $Z^*_{\ell_2}$
							$\log_{10}(\mathrm{MET})$

- $\log_{10}(p_{\mathrm{T}}$ or $E_{\mathrm{T}})$: the input value depends on what kind of data delphes output. For example, Track and EFlowTrack use p_{T} since they come from charged particle trajectory. Photon, NeutralHadron, and Tower use E_{T} since calorimeter-like objects are usually energy deposit and photon doesn't have mass.

- $\Delta\phi$ relative to leading ℓ: Low model uses raw ϕ since low model is the constituent-level ParT, we show the detector-level EFlow constituents to the model. But for high-model, we use $\Delta\phi$ relative to leading ℓ. Because inside a collider, events rotation around the beam axes shouldn't affect the physics classification. Therefore, the absolute ϕ value itself isn't an important information, it is more important to know the relative angle to each other.

- $\log_{10}(m)$: Objects without meaningful mass, such as MET, EFlowPhoton, EFlowNeutralHadron, are set to zero and use: log 10(max(mass, 1e-3)).

- EFlow constituents selection rules: There are several EFlow constituents for each event. We merge the three types of EFlow objects into a single list for each event, sort the list by p_{T} or E_{T} in descending order, select the top 128 objects, and apply zero-padding if there are fewer than 128 objects.

A short summary of comparison between our current model and the original model:

Table 21: Comparison between the original Particle Transformer architecture and the modified ParT v4 model used in this work.

Component	Original ParT paper	Our current model
Task	Jet tagging	VBS event classification
Input objects	Jet constituents	High-level physics objects or energy-flow constituents
Class token	Particle-attention blocks → token → class-attention blocks	`CLS` token as the initial input
Class-attention blocks	After the particle-attention blocks	No separate class-attention block is used
Output	Multi-class SoftMax classifier	Binary sigmoid classifier
Edge features	Pairwise particle-interaction features are embedded and added to the attention logits	High-level and optimized modes:explicit edge features; Low-level mode: without edge features
Global features	No event-level global features used	The optimized model: concatenates global features with the final `CLS`-token

As shown in Figure 10, there are three kinds of model trained for different purpose:

- Low-Level: Let the model directly examine constituents that closely resemble the detector output to see if it can learn useful event structures on its own. Input Objects are `EFlowTrack, EFlowPhoton, EFlowNeutralHadron`. These are particle-flow level constituents, closer to the detector reconstructed particles, but still not the initial detector hits.
- High-Level: To test whether a Transformer can outperform a dense neural network when using only traditional physics objects? Input Objects are `lep1, lep2, jet1, jet2, MET`. These are the analysis-level objects, reconstructed by Delphes according to tracker, calorimeter, muon system, jet cluster. Interaction features are incorporated to give more information to the model to improve performance.
- Optimized: Combining two types of information; On one hand, preserving the clear physical meaning of high-level objects; on the other, incorporating EFlow constituents to allow the model to observe finer-grained event activity. Input objects are `lep1, lep2, jet1, jet2, MET + leading EFlow constituents`. In addition, global features are incorporated; thus, the optimized model is a physics-informed Particle Transformer.

20.3 Hyper-parameters

Table 22: Model Hyperparameters and Training Configuration

Hyperparameter	**Value**
optimizer	`AdamW`
learning rate	`5e-4`
weight decay	`1e-4`
batch size	`256`
max epochs	`100`
patience	`8`
embedding dim	`64`
attention heads	`4`
Transformer layers	`4`
dropout	`0.2`
loss	weighted `BCELoss`
gradient clipping	`5.0`
folds	`5`, but current results are fold0 only

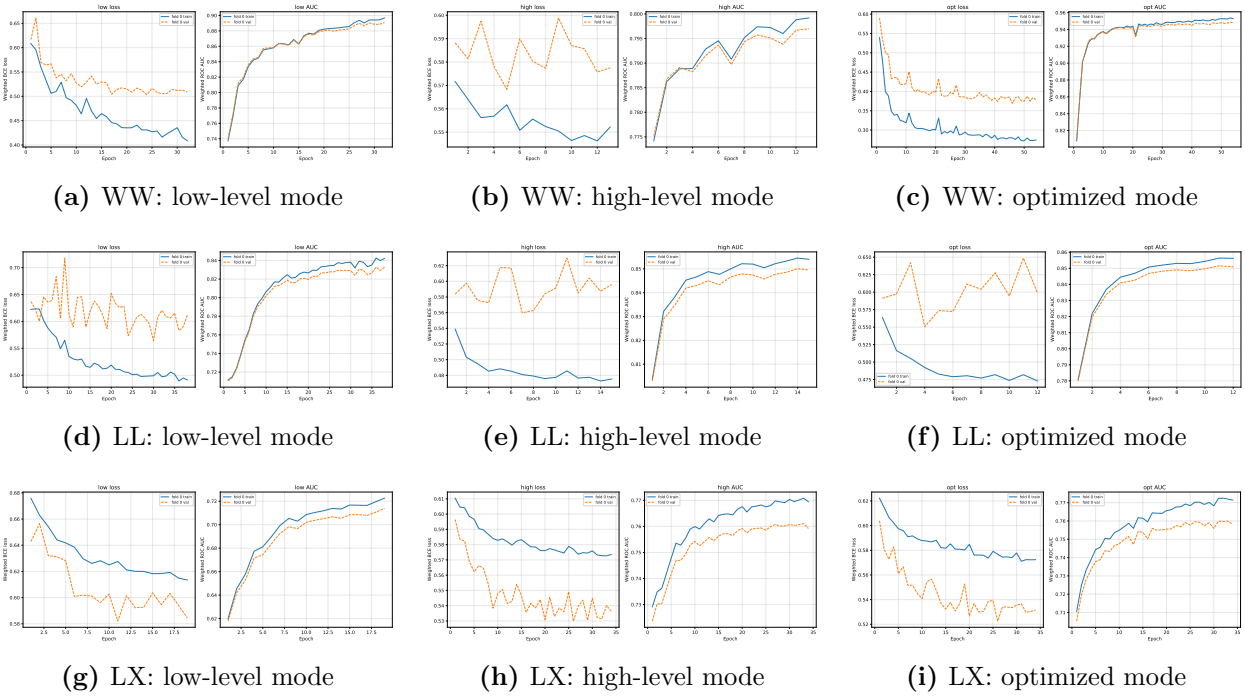
20.4 Results

I summarized the performance, including the AUC value, the expected discovery significance Z_0, the 95% CL upper limit, the training time, and the best epoch for three kinds of modes.

Table 23: Performance, upper-limit results, and training information for the low-, high-, and optimized-level ParT representations.

Task	**Mode**	**AUC**	Z_0	**Upper limits** μ_{95}, σ_{95} [fb]	**Training time**	**Best epoch**
WW	low	0.8757	24.5697	0.0868, 0.0741	52.0 min	24
	high	0.7923	20.4625	0.0885, 0.0756	7.3 min	5
	opt	0.9441	29.3286	0.0845, 0.0722	52.1 min	46
LL	low	0.8308	2.1456	0.9592, 0.4249	112.0 min	30
	high	0.8455	2.2392	0.9164, 0.4060	15.3 min	7
	opt	0.8429	2.2227	0.9223, 0.4086	28.5 min	4
LX	low	0.7064	6.0093	0.3011, 0.5691	58.2 min	11
	high	0.7586	6.9991	0.2503, 0.4731	34.1 min	26
	opt	0.7590	6.9767	0.2510, 0.4743	65.5 min	26

In Figure 23, we can see that for WW models, optimized model has the best performance while low-level model also performed better than previous dense-NN model. Meaning that giving out global features did help the model to classify signal from background pretty well. For LL model, things became different. High-level model has the best performance among the other two. Noticed that LX model performs extremely worse, this might be something wrong in the code that we can find out also in Figure 11 for the unusual loss functions and ROC curves.



(a) WW: low-level mode **(b)** WW: high-level mode **(c)** WW: optimized mode

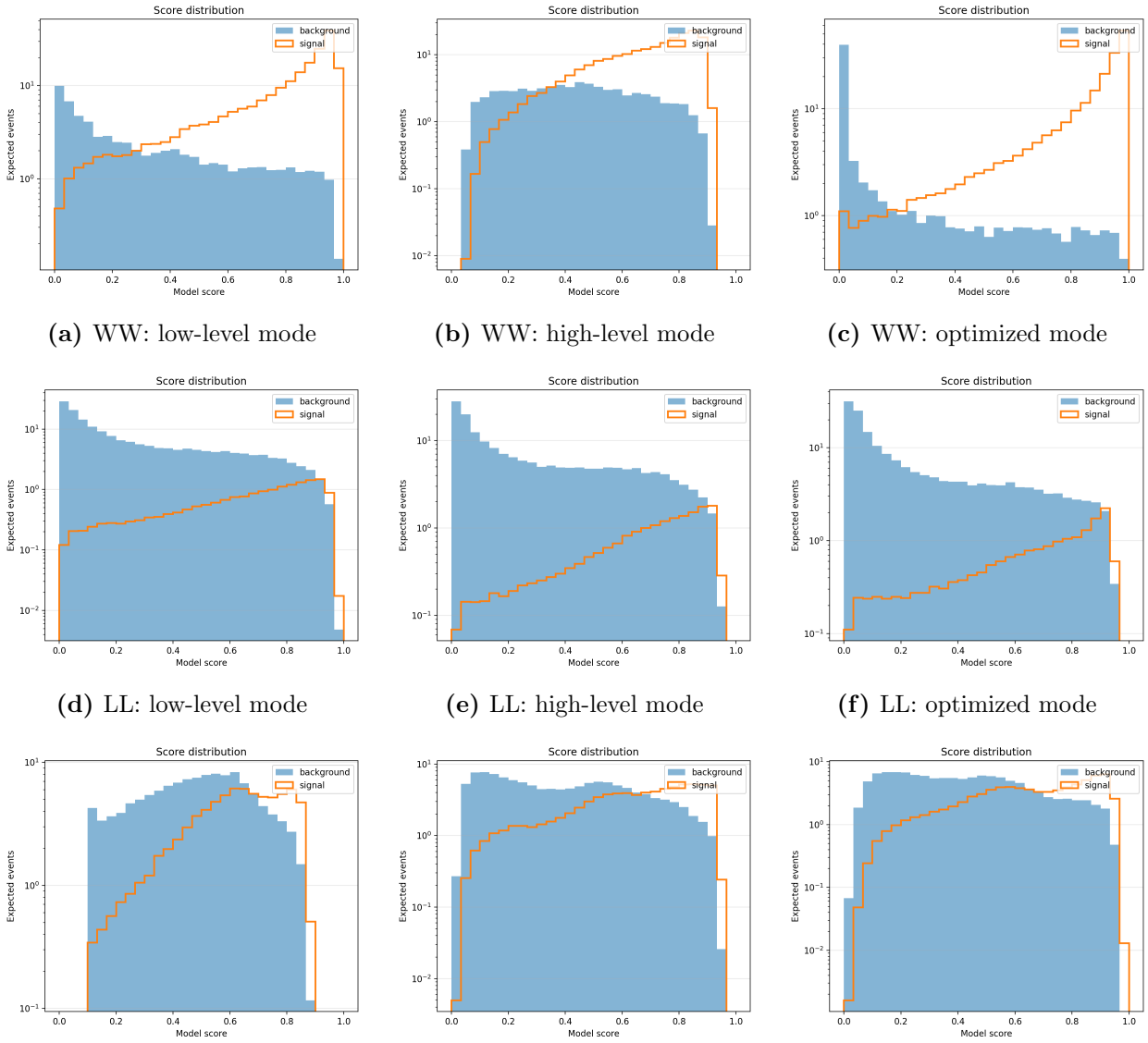
(d) LL: low-level mode **(e)** LL: high-level mode **(f)** LL: optimized mode

(g) LX: low-level mode **(h)** LX: high-level mode **(i)** LX: optimized mode

Figure 11: Training curves for the WW, LL, and LX classification tasks using the low-level, high-level, and optimized ParT representations. Each panel shows the weighted binary cross-entropy loss and weighted ROC AUC for the training and validation samples.

In Figure 11, the loss functions for high-level models seems to oscillate too violent. Meaning that maybe for epochs are needed. While I am only using 8 patience for the training. For LX models, there might be some code error or training problem we need to figure out since the output training curves seem to be weird and abnormal.

See Figure 12 to visualize the classification ability for each model.



(a) WW: low-level mode **(b)** WW: high-level mode **(c)** WW: optimized mode

(d) LL: low-level mode **(e)** LL: high-level mode **(f)** LL: optimized mode

(g) LX: low-level mode **(h)** LX: high-level mode **(i)** LX: optimized mode

Figure 12: Model-score distributions for the WW, LL, and LX classification tasks using the low-level, high-level, and optimized ParT representations. The filled histograms show the expected background distributions, while the outlined histograms show the corresponding signal distributions for fold 0.

Figure 12 shows that signal events are generally shifted toward larger model scores, while background events preferentially populate lower scores. The separation is most pronounced for the WW task, particularly in the optimized representation, whereas the LL and LX tasks exhibit greater signal–background overlap.

7/26

21 Aligned Model

To make comparison with Feng-Yang's model, we set the high-level model structure and hyperparameters the same to see if we can get similar results or performance. But I am using the previous parquet input file, therefore, there is still some input feature difference. For example, I have the mass and the type ID input, while Feng-Yang is using the one-hot coding to identify the input object type. We also tested the "half" model using only two particle attention blocks and one class attention block with half of the feature embedding dimension. Detail settings are listed as follow:

- Aligned Model:
  - Input objects: leading/subleading lepton/jet, MET
  - Particle features: $\log 10(p_{\mathrm{T}}), \eta, \Delta\phi(\ell_1)$, and $\log 10(\mathrm{mass})$ [64, 256, 48]
  - Type ID $\rightarrow$ embedding layer [16]
    Therefore, type ID concatenate with particle features getting dimension = 64.
  - Interaction features: $\Delta\eta, \Delta\phi$, and ΔR (calculated in the model, not stored in the parquet file.)
  - Particle attention blocks: 3; number of heads: 4.
  - Class attention blocks: 1; number of heads: 4.
  - No global features included.
- Half Model:
  - Input objects: leading/subleading lepton/jet, MET
  - Particle features: $\log 10(p_{\mathrm{T}}), \eta, \Delta\phi(\ell_1)$, and $\log 10(\mathrm{mass})$ [32, 128, 24]
  - Type ID $\rightarrow$ embedding layer [8]
    Therefore, type ID concatenate with particle features getting dimension = 32.
  - Interaction features: $\Delta\eta, \Delta\phi$, and ΔR (calculated in the model, not stored in the parquet file.)
  - Particle attention blocks: 2; number of heads: 4.
  - Class attention blocks: 1; number of heads: 4.
  - No global features included.

The hyperparameters used in this model are shown in Table 26.

Hyperparameter	Value
Optimizer	Adam
Learning rate	0.001
Batch size	2048
Maximum epochs	200
Early-stopping patience	10
Random seed	42
Loss function	Weighted binary cross-entropy (BCE)
Weight strategy	Hybrid weighting, same as the previous v4 model
Gradient clipping	Maximum norm of 5.0
Score output	Binary sigmoid
Selection-metric	`val_loss`

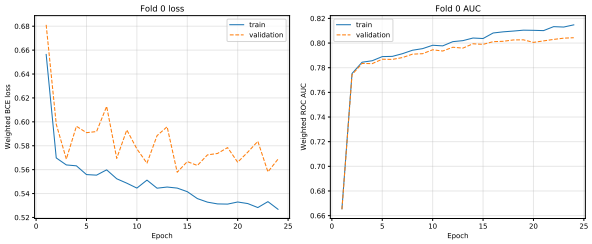
Table 24: Hyperparameters used for model training.

*Gradient clipping: It limits the gradient norm (the sum of all trainable parameters' gradient) to a specific maximum value to prevent excessively large updates that could destabilize training. It is not a model architecture component, but rather a safety setting for the training process.

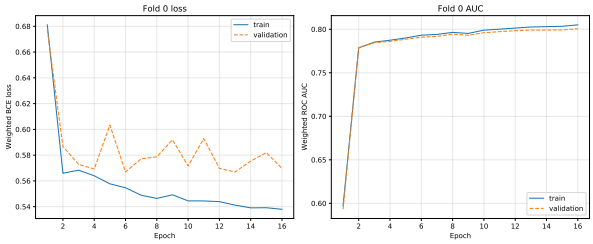
Table 25: Performance comparison of the aligned and half high-level models using validation AUC as the checkpoint selection for the WW, LL, and LX classification tasks.

Task	Model	AUC	Expected Z_0	μ_{95}	σ_{95} [fb]	Epochs	Best epoch	Time
WW	aligned	0.8012	N/A	N/A	N/A	24	14	4.6 min
WW	half	0.7933	N/A	N/A	N/A	16	6	3.0 min
LL	aligned	0.8409	1.743	2.175	0.718	14	4	5.2 min
LL	half	0.8444	1.804	2.150	0.709	17	7	5.1 min
LX	aligned	0.7568	5.645	1.437	1.946	18	8	6.1 min
LX	half	0.7588	5.822	1.432	1.939	22	12	6.7 min

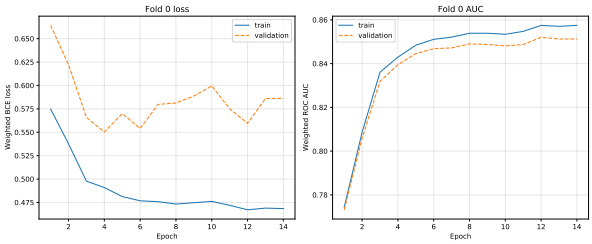
As shown in Table 25, the performance of the models did not improve significantly while we decrease the number of particle attention blocks and the input feature dimensions.



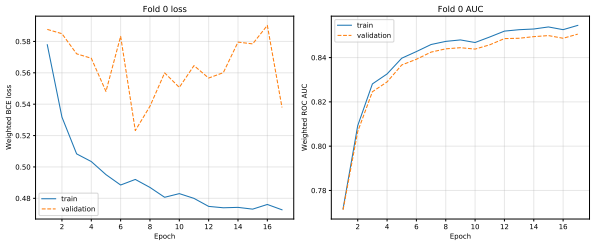
(a) WW: aligned model



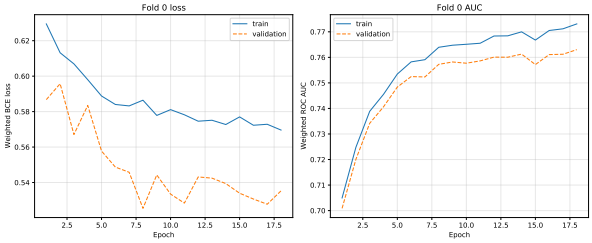
(b) WW: half model



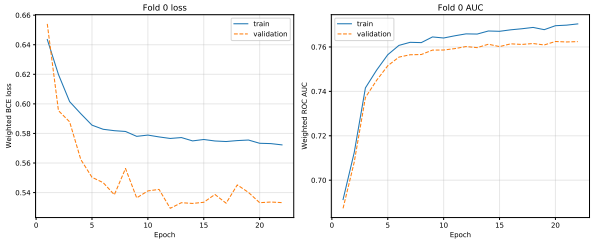
(c) LL: aligned model



(d) LL: half model



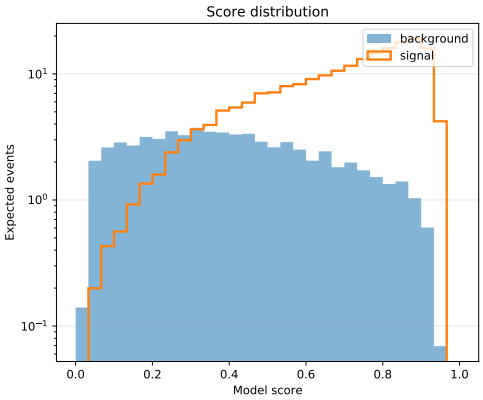
(e) LX: aligned model



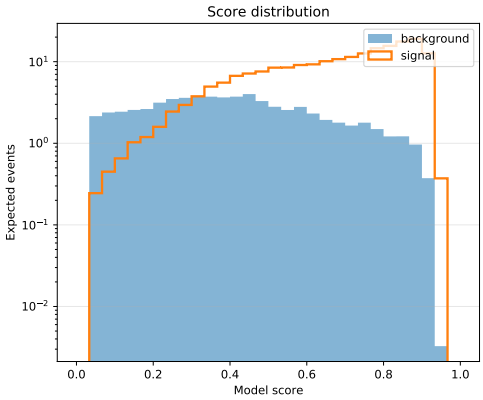
(f) LX: half model

Figure 13: Training curves for the WW, LL, and LX classification tasks. The left column shows the aligned models, while the right column shows the corresponding half models.

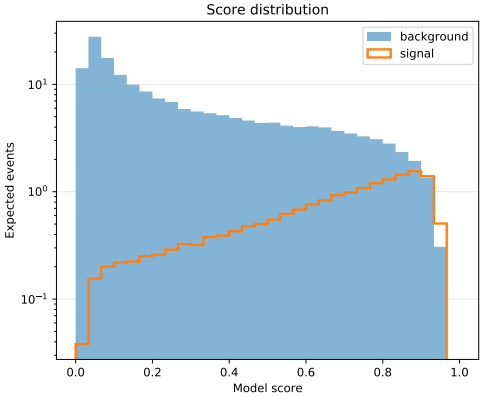
In Figure 13, we noticed that the loss curves for all model seems to be too steep for a reasonable training. Therefore, I adjusted the checkpoint selection and learning rate in Section 22.



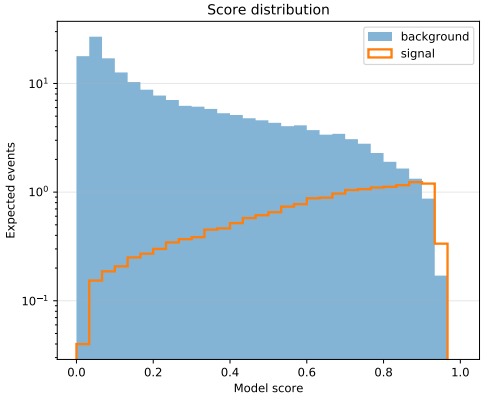
(a) WW: aligned model



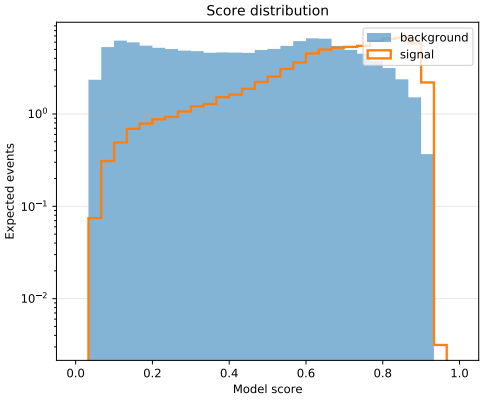
(b) WW: half model



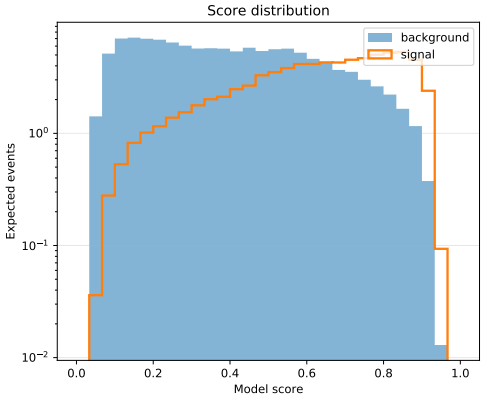
(c) LL: aligned model



(d) LL: half model



(e) LX: aligned model



(f) LX: half model

Figure 14: Score distributions for the WW, LL, and LX classification tasks. The left column shows the aligned models, while the right column shows the corresponding half models.

We can see some steep decline in the extreme output score values for 0 and 1, especially for LL and LX in Figure 14. This problem is also solved in Section 22. The imperfect separation for LX task might be the future direction we should figure out.

22 Training Problem

The original training curve, especially the loss curves, is too steep. The main problems are:

1. The calculation of validation loss is prone to fluctuations caused by batch weight distribution.
2. Checkpoint selection relies on validation loss, whereas our primary concern is AUC or significance.
3. Consequently, we might select an epoch that offers superior probability calibration but suboptimal signal/background separation.

Therefore, instead of using the lowest weighted loss as the checkpoint selection, I change to use the AUC value as the checkpoint selection. We can then find the epoch that has the best signal/background separation ability rather than having the best probability calibration. I also set a floating learning rate such that the model starts by learning rapidly with larger learning rate. As the optimal point is approached, automatically switch to smaller learning for fine-tuning.

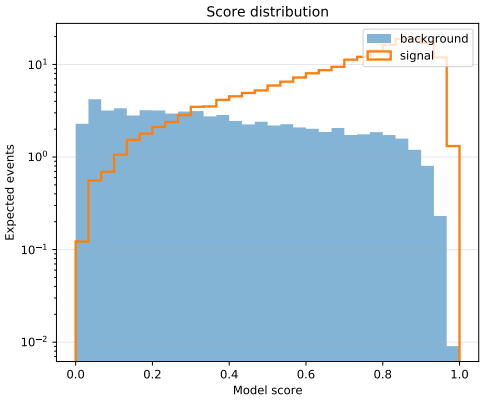
Hyperparameter	**Value**
Optimizer	Adam
Learning rate	start from 0.0005 (patience 3, lr factor 0.5, min-lr 0.00001)
Batch size	2048
Maximum epochs	200
Early-stopping patience	15
Random seed	42
Loss function	Weighted binary cross-entropy (BCE)
Weight strategy	Hybrid weighting, same as the previous v4 model
Gradient clipping	Maximum norm of 5.0
Score output	Binary sigmoid
Selection-metric	`val _ AUC`

Table 26: Hyperparameters used for model training.

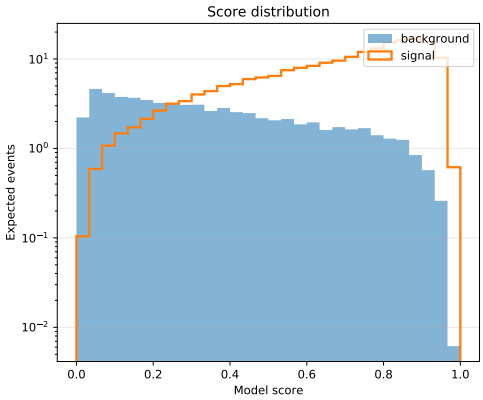
The stable adjusted models are better overall. WW AUC improves from about 0.80 to 0.81, LL AUC improves clearly from about 0.84 to 0.856-0.857, and LX improves from about 0.757-0.759 to 0.765 as shown in Table 27. For the significances, stable LL improves from $Z_0 \sim$ 1.74-1.80 to $\sim$1.83, and stable LX improves from Z_0 $\sim$5.64-5.82 to $\sim$5.98. The price is training time: stable training runs longer because the adjusted code uses `val_AUC` checkpoint selection, smaller learning rate, patience 15, and learning rate scheduling. That is expected and likely healthier than the very short previous run.

Table 27: Performance comparison of the aligned and half high-level models for the WW, LL, and LX classification tasks.

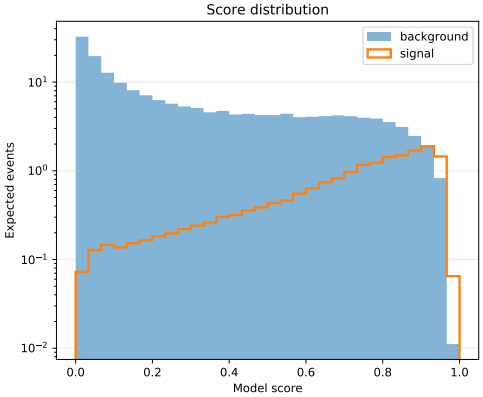
Task	**Model**	**AUC**	**Expected** Z_0	μ_{95}	σ_{95} [**fb**]	**Epochs**	**Best epoch**	**Time**
WW	aligned	0.8107	N/A	N/A	N/A	66	51	12.2 min
WW	half	0.8127	N/A	N/A	N/A	90	75	14.3 min
LL	aligned	0.8574	1.833	2.116	0.698	54	39	17.0 min
LL	half	0.8561	1.832	2.119	0.699	90	75	26.4 min
LX	aligned	0.7647	5.983	1.427	1.932	58	43	19.3 min
LX	half	0.7647	5.978	1.428	1.933	78	63	22.7 min



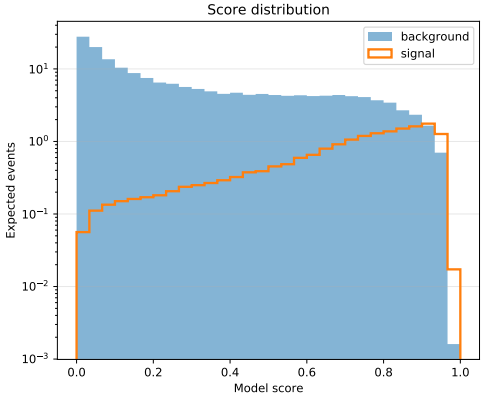
(a) WW: aligned model



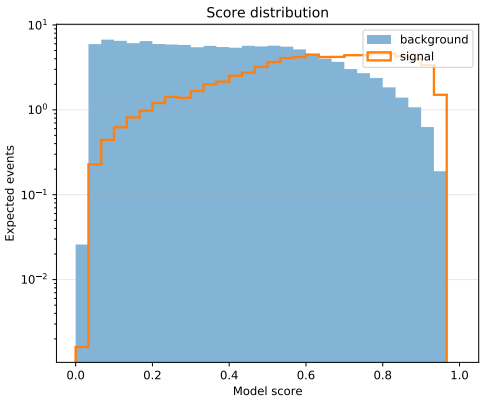
(b) WW: half model



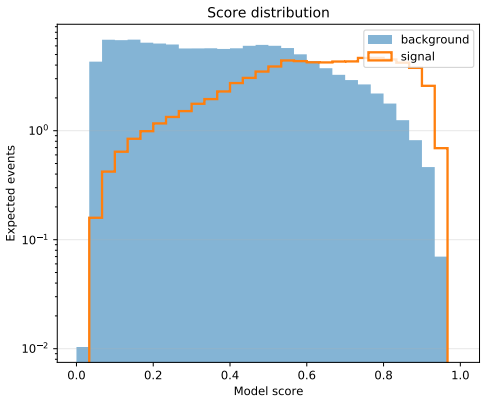
(c) LL: aligned model



(d) LL: half model



(e) LX: aligned model

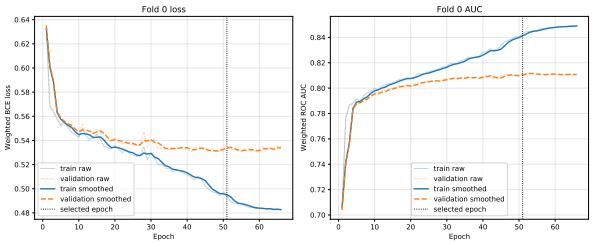


(f) LX: half model

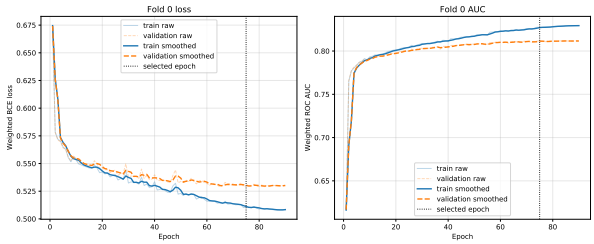
Figure 15: Score distributions trained by using validation AUC as the checkpoint selection for the WW, LL, and LX classification tasks. The left column shows the aligned models, while the right column shows the corresponding half models.

Compare to the previous score distribution in Figure 14, there is less steep decline on the two edges, output score 0 and 1 in Figure 15, indicating that we might have adjusted the model training to a more healthier way so that models are well trained to confidently giving out extreme values

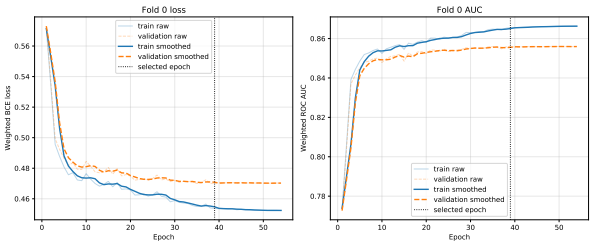
such as 0 and 1.



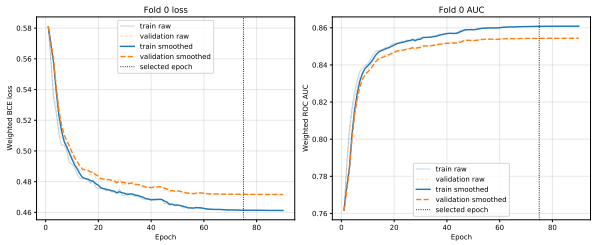
(a) WW: aligned model



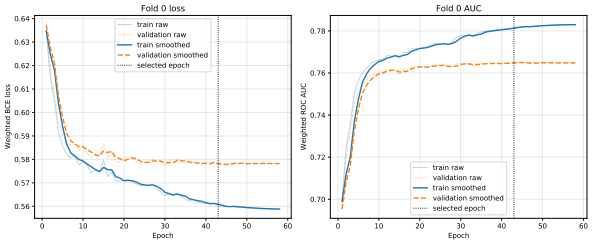
(b) WW: half model



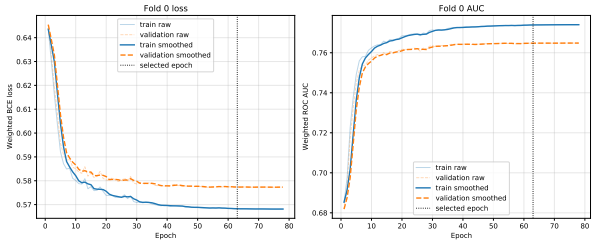
(c) LL: aligned model



(d) LL: half model



(e) LX: aligned model



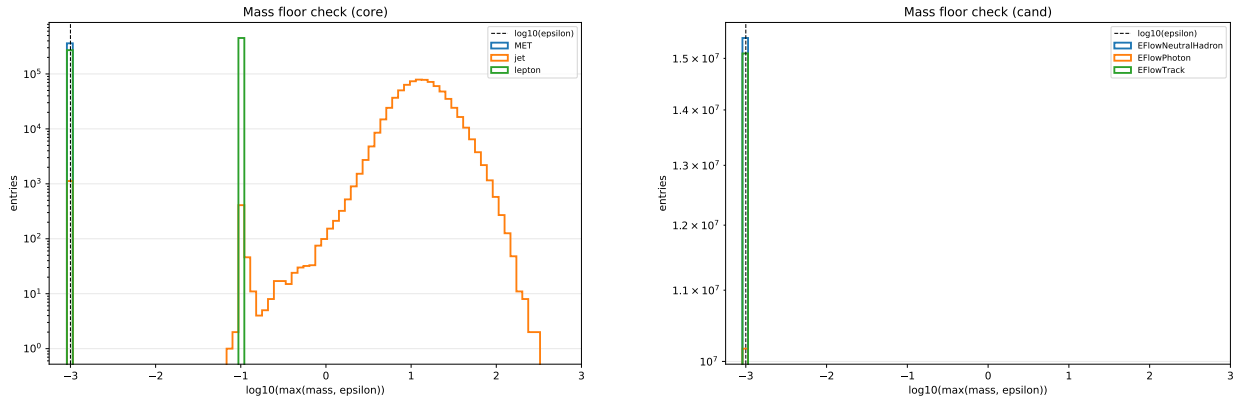
(f) LX: half model

Figure 16: Training curves using validation AUC as the checkpoint selection for the WW, LL, and LX classification tasks. The left column shows the aligned models, while the right column shows the corresponding half models.

After the adjust to use `val_AUC` checkpoint selection, smaller learning rate, patience 15, and learning-rate scheduling, the loss curves became more gradual and was no longer so steep as shown in Figure 16. The smooth curves used moving average method with window size three for better visualizing the trend of the curves.

23 Mass Distribution

For the case some of the input objects are not having meaningful mass, we set `log(max(m,1e-3))`. But we still need to make sure that `1e-3` is a small enough value. Therefore, we plot the mass distribution of the input objects, including the low-level and high-level input objects.



(a) WW: high-level mass distribution

(b) WW: low-level mass distribution

Figure 17: Mass distributions for the WW high-level and low-level tasks.

From Figure 17, we can see that for high-level inputs 17a `1e-3` is a sufficient small value which can be separated from other objects. But for low-level inputs 17b, we should exclude the mass as an particle feature since the mass of low-level objects are set to zero in the code to write the input parquet files. For `EFlowPhoton` this is reasonable since photon is exactly massless. For `EFlowNeutralHadron`, this is an approximation. The EFlow neutral hadrons in Delphes typically provide E_{T}, η, and ϕ, but do not necessarily have a reliable mass branch, so we initially set the mass to zero. For `EFlowTrack`, this is also an approximation. The track itself is a detector track; the specific type of charged particle is not necessarily known. Without particle identification, one cannot safely assign the mass of a pion, kaon, or proton. Therefore, we set it to 0 in the code.

7/29

24 Semi-Completely Aligned Model

There are two kinds of model for this week to make sure and test how object type and edge features embedding dimension may affect model performance.

24.1 One Hot Coding

In this version, the type is represented as a one-hot vector rather than using type embedding; this vector is concatenated directly to the node features, and the combined input is fed into the same node embedding MLP. The edge features remain the original ones.

Table 28: One hot coding aligned: Performance comparison of the aligned and half high-level models using validation AUC as the checkpoint selection for the WW, LL, and LX classification tasks.

Task	**Model**	**AUC**	**Expected Z_0**	μ_{95}	σ_{95} **[fb]**	**Epochs**	**Best epoch**	**Time**
WW	aligned	0.8088	N/A	N/A	N/A	59	44	11.0 min
WW	half	0.8024	N/A	N/A	N/A	77	62	11.8 min
LL	aligned	0.8561	1.835	2.117	0.699	61	46	19.3 min
LL	half	0.8549	1.826	2.123	0.700	81	66	24.0 min
LX	aligned	0.7609	5.890	1.431	1.938	65	50	21.6 min
LX	half	0.7625	5.920	1.430	1.936	101	86	29.6 min

24.2 One Hot Coding + Edge Embedding Dimension

In this version, one-hot encoding is used for the node type as well. The edge branch has been modified to more closely resemble the ParT code you shared: it uses $\Delta\eta$, $\Delta\phi$, and ΔR followed by a Conv2d sequence ($3 \to 64 \to 64 \to$ num_heads) to generate the attention bias.

Table 29: One hot coding + edge embedding dimension align: Performance comparison of the aligned and half high-level models using validation AUC as the checkpoint selection for the WW, LL, and LX classification tasks.

Task	**Model**	**AUC**	**Expected Z_0**	μ_{95}	σ_{95} **[fb]**	**Epochs**	**Best epoch**	**Time**
WW	aligned	0.8041	N/A	N/A	N/A	75	60	15.7 min
WW	half	0.7992	N/A	N/A	N/A	67	52	11.3 min
LL	aligned	0.8551	1.805	2.128	0.702	70	55	24.9 min
LL	half	0.8497	1.792	2.142	0.707	96	81	31.7 min
LX	aligned	0.7604	5.864	1.432	1.939	64	49	21.1 min
LX	half	0.7598	5.797	1.433	1.941	94	79	29.6 min

Overall, one hot coding only version is performing better than one hot coding along with corrected edge embedding dimension. Especially for WW and LL model. Compared to the previous "stable adjusted model," the one-hot version shows no significant improvement.

25 Completely Aligned Model

After the previous model comparison, we found out that our weighting methods were not exactly identical. Therefore, I adjusted my weighting method to get aligned with the thesis, which is also the same method of Feng-Yang's model, and trained our model with four kinds of model structure: type embedding (object types concatenate with node features after separately going through different embedding layers) or one hot coding (object types concatenate with node features and go through the same embedding layers) and linear edge features [3-32-4] or convolution 2D edge embedding

method [3-64-64-4]. The performance for each kinds of task and model structures are listed in the following tables:

Table 30: Performance comparison of type-coding and edge-embedding configurations for the WW, LL, and LX classification tasks.

Task	Type coding	Edge method	Model	Weighted AUC	Best validation AUC	Epochs	Best epoch	Time
WW	one-hot	linear `3--32--4`	aligned	0.7828	0.7793	68	58	8.7 min
			half	**0.7846**	**0.7822**	95	85	12.0 min
		Conv2D `3--64--64--4`	aligned	0.7816	0.7791	53	43	7.5 min
			half	0.7786	0.7757	41	31	5.8 min
	type embedding	linear `3--32--4`	aligned	0.7713	0.7680	47	37	6.3 min
			half	0.7731	0.7715	53	43	6.7 min
		Conv2D `3--64--64--4`	aligned	0.7741	0.7713	36	26	5.0 min
			half	0.7742	0.7716	65	55	8.7 min
LL	one-hot	linear `3--32--4`	aligned	0.8326	0.8355	69	59	16.1 min
			half	0.8336	0.8369	110	100	25.6 min
		Conv2D `3--64--64--4`	aligned	0.8326	0.8353	68	58	16.7 min
			half	**0.8341**	**0.8372**	113	103	27.1 min
	type embedding	linear `3--32--4`	aligned	0.8226	0.8246	56	46	12.8 min
			half	0.8225	0.8251	111	101	24.1 min
		Conv2D `3--64--64--4`	aligned	0.8219	0.8245	58	48	14.2 min
			half	0.8209	0.8240	99	89	22.9 min
LX	one-hot	linear `3--32--4`	aligned	0.7790	0.7824	56	46	14.7 min
			half	0.7776	0.7806	65	55	15.8 min
		Conv2D `3--64--64--4`	aligned	0.7788	0.7822	56	46	14.7 min
			half	**0.7803**	**0.7835**	105	95	25.7 min
	type embedding	linear `3--32--4`	aligned	0.7674	0.7705	57	47	13.8 min
			half	0.7697	0.7728	113	103	24.6 min
		Conv2D `3--64--64--4`	aligned	0.7654	0.7691	48	38	12.4 min
			half	0.7657	0.7686	83	73	19.9 min

In Table 32, concatenating one-hot vectors before embedding is clearly superior to using separate type embeddings. This holds true for all three tasks, with a performance gap of approximately:

- *WW*: 0.008 to 0.011
- LL: 0.010 to 0.013
- LX: 0.010 to 0.014

Conv2D edge embedding does not significantly outperform the original linear edge [3-32-4] configuration. The differences are usually minimal, often around 0.001, suggesting that the type encoding method is what truly matters, rather than the depth of the edge embedding. Current best combination is listed in Table 33.

Table 31: Best-performing configuration for each classification task.

Task	**Best setting**	**AUC**
WW	one-hot + linear edge + half	**0.7846**
LL	one-hot + Conv2D edge + half	**0.8341**
LX	one-hot + Conv2D edge + half	**0.7803**

8/13

26 Low-Level Model Training

After testing the model training using high-level input objects, we turn to low-level input objects to see if we can get better model performance.

Table 32: Performance comparison of one hot coding/type embedding, linear/Conv2d, and aligned/half model structure for the WW, LL, and LX classification tasks using low-level input objects. Weighting method is aligned with the thesis, using the two step weighting.

Task	Type coding	Edge method	Model	Weighted AUC	Epochs	Best epoch	Time (min)
WW	one-hot	linear `3--32--4`	aligned	0.9374	44	34	69.2
			half	0.9440	76	66	119.3
		Conv2D `3--64--64--4`	aligned	**0.9446**	56	46	123.8
			half	0.9435	86	76	174.9
	type embedding	linear `3--32--4`	aligned	0.9420	67	57	107.5
			half	0.9409	96	86	141.2
		Conv2D `3--64--64--4`	aligned	0.9413	52	42	116.3
			half	0.9400	59	49	121.7
LL	one-hot	linear `3--32--4`	aligned	0.8074	33	23	91.0
			half	0.8091	52	42	147.9
		Conv2D `3--64--64--4`	aligned	0.8147	33	23	131.7
			half	**0.8224**	71	61	254.9
	type embedding	linear `3--32--4`	aligned	0.7882	49	39	143.3
			half	0.8116	98	88	267.8
		Conv2D `3--64--64--4`	aligned	0.8049	46	36	184.4
			half	0.7995	85	75	310.2
LX	one-hot	linear `3--32--4`	aligned	0.7565	36	26	100.4
			half	0.7531	62	52	167.2
		Conv2D `3--64--64--4`	aligned	0.7588	26	16	105.0
			half	**0.7643**	56	46	202.2
	type embedding	linear `3--32--4`	aligned	0.7345	59	49	168.8
			half	0.7523	90	80	254.8
		Conv2D `3--64--64--4`	aligned	0.7501	48	38	193.2
			half	0.7353	73	63	295.2

In Table 32, concatenating one-hot vectors before embedding is generally superior to using separate type embeddings. This holds true across most variations, with a performance gap (when one-hot outperforms type embedding) of approximately:

- WW: up to 0.003
- LL: 0.010 to 0.023

- LX: 0.008 to 0.022

Conv2D edge embedding shows measurable improvements over the original linear edge `3--32--4` configuration in most half-precision and high-complexity setups, though the type encoding method remains a highly influential factor. The current best combination for each task is listed in Table 33.

Table 33: Best-performing configuration for each classification task.

Task	**Best setting**	**AUC**
WW	one-hot + Conv2D edge + aligned	**0.9446**
LL	one-hot + Conv2D edge + half	**0.8224**
LX	one-hot + Conv2D edge + half	**0.7643**

27 High-Level 5 Fold Performance

To determine whether the difference in AUC values between Feng-Yang and myself stemmed from a failure to train on all folds of the 5-fold cross-validation and average the results, we trained models using high-level input objects, incorporating one-hot encoding, aligned, and a Conv2D architecture. Subsequently, we calculated the average AUC from the five folds and reported the corresponding uncertainty in the following table:

Table 34: Performance summary for the high-level LL aligned model with one-hot type encoding and Conv2D edge embedding.

Task	**Weighted AUC**	**Best epoch mean**	**Epochs mean**	**Total time (min)**
LL	0.8335 ± 0.0011	53.4	63.4	73.4

The high-level LL one-hot Conv2D model (Table 34) was evaluated with 5-fold cross validation and achieved a senior-weighted AUC of 0.8335 ± 0.0011. For low-level EFlow-only model (Table 32), fold-0 diagnostic tests show very strong WW discrimination, with the best AUC reaching 0.9446. However, for polarization classification, the best low-level LL and LX AUC values are 0.8224 and 0.7643, respectively, slightly below the corresponding high-level performance. This suggests that EFlow constituents are highly informative for separating EW WW from backgrounds, while polarization classification still benefits from reconstructed high-level VBS topology.

28 Larger Dataset

We also wish to investigate whether larger datasets can help improve model performance. Therefore, we generated samples for various types of events, including WW-EW, WW-QCD, WZ-EW, WZ-QCD, W_LW_L, W_LW_T, and W_TW_T.

28.1 High-Level Results

First of all, we re-train the model using the larger dataset as summarized in table 35. The perfor-

Model Training	Signal Events	Background Events
WW	480,825	236,891
LL	425,457	970,439
LX	888,083	507,813

Table 35: Summary of signal and background training events for WW, LL, and LX models.

mance for the high-level inputs are summarized in the following table:

Table 36: Performance comparison of one-hot coding/type embedding, linear/Conv2D edge embedding, and aligned/half model structures for the WW, LL, and LX classification tasks using high-level input objects. The event weighting follows the two-step weighting procedure used in the thesis.

Task	Type coding	Edge method	Model	Weighted AUC	Epochs	Best epoch	Time (min)
WW	one-hot	linear `3--32--4`	aligned	0.7985	83	73	20.0
			half	**0.8030**	147	137	35.0
		Conv2D `3--64--64--4`	aligned	0.7983	69	59	16.8
			half	0.8022	130	120	31.2
	type embedding	linear `3--32--4`	aligned	0.7813	40	30	10.0
			half	0.7843	77	67	18.8
		Conv2D `3--64--64--4`	aligned	0.7861	45	35	11.2
			half	0.7849	61	51	15.0
LL	one-hot	linear `3--32--4`	aligned	0.8363	59	49	27.7
			half	**0.8373**	117	107	53.6
		Conv2D `3--64--64--4`	aligned	0.8362	51	41	24.8
			half	0.8371	105	95	47.9
	type embedding	linear `3--32--4`	aligned	0.8279	48	38	22.8
			half	0.8274	90	80	40.6
		Conv2D `3--64--64--4`	aligned	0.8282	67	57	32.2
			half	0.8279	152	142	69.2
LX	one-hot	linear `3--32--4`	aligned	0.7838	58	48	27.7
			half	**0.7848**	142	132	64.6
		Conv2D `3--64--64--4`	aligned	0.7839	56	46	27.0
			half	0.7844	98	88	45.1
	type embedding	linear `3--32--4`	aligned	0.7744	64	54	30.7
			half	0.7746	82	72	38.3
		Conv2D `3--64--64--4`	aligned	0.7730	45	35	21.7
			half	0.7720	105	95	49.3

Compared to the baseline performance in Table 32, the best AUC values for various task types across different model architectures show slight improvements, yet training time has increased to two or three times the original duration.

28.2 Low-Level Results

Based on the results in Table 37, the best AUC values for the *WW* task did not improve across any of the eight model architectures. Furthermore, due to longer-than-expected processing times, it is currently not possible to list all results, as some models are still undergoing training.

Table 37: Performance comparison of one-hot coding/type embedding, linear/Conv2D edge embedding, and aligned/half model structures using low-level input objects. The event weighting follows the two-step weighting procedure used in the thesis. Training time is reported in hours.

Task	Type coding	Edge method	Model	Weighted AUC	Epochs	Best epoch	Time (h)
WW	one-hot	linear `3--32--4`	aligned	0.9321	60	50	3.1
			half	0.9238	82	72	4.2
		Conv2D `3--64--64--4`	aligned	**0.9353**	62	52	4.4
			half	0.9331	54	44	3.5
	type embedding	linear `3--32--4`	aligned	0.9291	58	48	3.0
			half	0.9261	44	34	2.2
		Conv2D `3--64--64--4`	aligned	0.9306	55	45	4.0
			half	0.9293	70	60	4.7
LL	one-hot	linear `3--32--4`	aligned	0.8177	39	29	3.9
		Conv2D `3--64--64--4`	aligned	0.8264	34	24	4.7
	type embedding	Conv2D `3--64--64--4`	aligned	0.8184	55	45	7.8

29 Aligned, One hot, Conv2D

This section examines the differences between using AUC and loss values when selecting checkpoints. We evaluated the aligned, one-hot coding, and Conv2D configuration specifically for the WW, LL, and LX tasks. The inputs encompassed both low-level and high-level objects; for low-level inputs, we excluded the mass term from the node features, as we observed that the mass terms for EFlow components were set to zero.

Table 38: Performance comparison across different tasks and datasets. Training time is reported in hours.

Task	Dataset	Weighted AUC	Checkpoint Selection	Best Val. AUC	Val. Loss	Epochs	Best Epochs	Time (h)
WW	1M	0.9247	auc	0.9260	0.3442	47	37	1.8
		0.9247	loss	0.9250	0.3403	43	33	1.7
	2.11M	0.9310	auc	**0.9319**	0.3274	55	45	3.9
		0.9312	loss	0.9308	**0.3266**	36	26	2.6
LL	1M	0.8218	auc	0.8209	0.5177	37	27	2.7
		0.8218	loss	0.8203	0.5176	39	29	2.9
	2.11M	0.8246	auc	**0.8251**	0.5152	31	21	4.3
		0.8243	loss	0.8245	**0.5127**	41	31	5.7
LX	1M	0.7642	auc	0.7658	0.5778	40	30	3.0
		0.7642	loss	0.7658	0.5778	40	30	3.0
	2.11M	0.7743	auc	**0.7768**	0.5680	38	28	5.3
		0.7728	loss	0.7762	**0.5666**	44	34	6.1

29.1 Low-Level without Mass

Only the 2.11M-dataset, AUC-checkpoint, WW and LL configurations have directly matching mass-included results. For the available comparisons, excluding mass term did not improve fold-0 per-

Task	With mass AUC	Without mass AUC
WW	0.935251	0.931037
LL	0.826420	0.824634

Table 39: Comparison of AUC with and without mass.

formance:

- *WW* weighted AUC decreased by 0.42 percentage points.
- LL weighted AUC decreased by 0.18 percentage points.

However, this does not necessarily mean the zero mass variable contains useful physics. A constant input can normally be absorbed by network biases, but changing the input dimension changes model initialization and optimization. Because these are only fold-0 results, there is no fold-to-fold uncertainty from which to judge whether these small changes are significant. There is no matching mass-included result for LX, so those comparisons cannot yet be made fairly.

29.2 Loss Checkpoint vs. AUC Checkpoint

where Δ weighted AUC = loss checkpoint -AUC checkpoint. *WW* and LL tasks differences are extremely small. For the results using 2.11M dataset to train, LX favors checkpoint selection by

Dataset	Task	Δ weighted AUC	Best-epoch change
1M	WW	+0.000018	−4
1M	LL	+0.000090	+2
1M	LX	0.000000	0
2.11M	WW	+0.000195	−19
2.11M	LL	−0.000322	+10
2.11M	LX	−0.001498	+6

Table 40: Summary of changes in weighted AUC and best epoch for 1M and 2.11M datasets.

about 0.15 percentage points. Loss selection stopped WW (2.11M) training considerably earlier: epoch 26 rather than 45. There is no universal test-AUC winner, but loss checkpoint selection is reasonable and matches the requested thesis style convention.

29.3 Larger vs. Smaller Dataset

Checkpoint	Task	Old AUC	Combined AUC	Improvement
AUC	WW	0.924693	0.931037	+0.006344
AUC	LL	0.821757	0.824634	+0.002877
AUC	LX	0.764194	0.774342	+0.010148
Loss	WW	0.924711	0.931232	+0.006521
Loss	LL	0.821847	0.824311	+0.002464
Loss	LX	0.764194	0.772844	+0.008649

Table 41: Comparison of Old AUC and Combined AUC across different checkpoints and tasks.

The larger dataset improves all six matched comparisons:

- Largest improvement: LX, approximately 0.86–1.01 percentage points.
- WW improves approximately 0.63–0.65 percentage points.
- LL improves approximately 0.25–0.29 percentage points.

29.4 Low-Level vs. High-Level

where ΔAUC = low-level- high-level. The main findings are shown as follow:

- For WW, low-level improves AUC by approximately 0.133–0.143, which is a very large improvement.
- For LL, high-level is better by approximately 0.011.
- For LX, high-level is better by approximately 0.010–0.015.

The same pattern appears with both dataset sizes, so the conclusion is reasonably stable. Furthermore, low-level inputs are computationally much more expensive. The additional cost is strongly justified for WW, but not currently justified by the LL or LX AUC results.

Dataset	Task	High-level AUC	Low-level no-mass AUC	ΔAUC	Better input
1M	WW	0.781600	0.924693	+0.143093	Low-level
1M	LL	0.832600	0.821757	−0.010843	High-level
1M	LX	0.778800	0.764194	−0.014606	High-level
2.11M	WW	0.798300	0.931037	+0.132737	Low-level
2.11M	LL	0.836200	0.824634	−0.011566	High-level
2.11M	LX	0.783900	0.774342	−0.009558	High-level

Table 42: Comparison of high-level AUC and low-level no-mass AUC across datasets and tasks.

29.5 High-Level Checkpoint Selection Test

I also used the loss value as the checkpoint selection and compare the difference between loss and AUC. The average performance for high-level input, LL task, one-hot coding, Conv2D, and all 5-fold trained is:

$$\mathrm{AUC}_{\mathrm{loss}} = 0.83398 \pm 0.00073$$

Quantity	AUC checkpoint	Loss checkpoint	Loss − AUC
Weighted AUC mean	0.8335	0.833979	+0.000479
Best epoch mean	53.4	48.4	−5.0 epochs
Total epochs mean	63.4	58.4	−5.0 epochs
Total training time	73.4 min	74.32 min	+0.92 min

Table 43: Comparison of metrics between AUC and Loss checkpoints.

Checkpoint selection by loss gives a slightly higher mean test AUC, which is only a 0.048 percentage-point improvement and is smaller than the fold-to-fold variation. Therefore, these is no evidence of a meaningful performance difference between loss and AUC checkpoint selection.

8/31

Last time, we trained models with low-level objects, and trained aligned, one-hot coding, and Conv2D model with larger dataset. Low-level input improved the performance only for WW task. The performance for increasing the number of training dataset didn't improve much. Therefore, for this week, we trained:

- Smaller dataset to test model saturation.
- Enlarge model structure also to test saturation.
- Edge features effect for both low and high level input.

We are only training the model structure: Aligned, one-hot coding, Conv2D for all task.

30 Complete Saturation Comparison

Since we don't have significant improvement while enlarging the dataset to 2.11M. We want to figure whether the model is saturated. We can try by two ways: enlarge the model structure or decrease the number of dataset. Decreasing the number of dataset is more computational efficiency, therefore, we will try to use 100k and 500k raw events per process, capped before the signal-region cut.

Raw events per process	**WW (Signal / Background)**	**LL (Signal / Background)**	**LX (Signal / Background)**
100k	22,951 / 11,248	20,005 / 46,045	42,113 / 23,937
500k	113,690 / 56,084	100,572 / 229,910	210,363 / 120,119
1M	227,645 / 111,868	201,498 / 459,361	420,596 / 240,263
2.11M	480,825 / 236,891	425,457 / 970,439	888,083 / 507,813

Table 44: Signal and background events after SR for WW, LL, and LX processes.

30.1 Training Using Different Dataset

Task	**Input configuration**	**100k (min, epoch)**	**500k**	**1M**	**2.11M**
WW	High / geometry / mass	0.5847 (0.5, 14)	0.7849 (4.4, 64)	0.7895 (11.9, 71)	0.7989 (23.5, 79)
WW	Low / no edge / no mass	0.7458 (5.8, 47)	0.8069 (24.5, 50)	0.8544 (52.5, 54)	0.9006 (127.9, 63)
WW	Low / Geometry / no mass	0.7848 (13.1, 58)	0.9189 (48.3, 52)	0.9277 (111.7, 43)	**0.9314 (262.6, 49)**
LL	High / geometry / mass	0.8060 (5.2, 67)	0.8271 (19.9, 74)	0.8315 (16.1, 62)	**0.8364 (32.0, 58)**
LL	Low / no edge / no mass	0.7063 (18.1, 87)	0.7021 (40.8, 44)	0.7944 (93.8, 49)	0.8141 (194.5, 49)
LL	Low / Geometry / no mass	0.7718 (21.0, 54)	0.8048 (59.9, 33)	0.8194 (187.8, 40)	0.8245 (394.9, 37)
LX	High / geometry / mass	0.7499 (2.2, 73)	0.7717 (3.6, 46)	0.7790 (15.3, 65)	**0.7832 (27.8, 48)**
LX	Low / no edge / no mass	0.5212 (3.4, 16)	0.6409 (24.5, 26)	0.7510 (99.9, 52)	0.7660 (158.8, 40)
LX	Low / Geometry / no mass	0.7146 (23.4, 61)	0.7467 (88.9, 49)	0.7596 (122.9, 35)	0.7727 (430.8, 41)

Table 45: Complete saturation test for high-level and low-level using 100k, 500k, 1M, and 2.11M dataset. Data for "Low / Geometry / no mass" rows integrated from provided reference figures.

The training results are shown in Table 45 and plotted in Figure 18

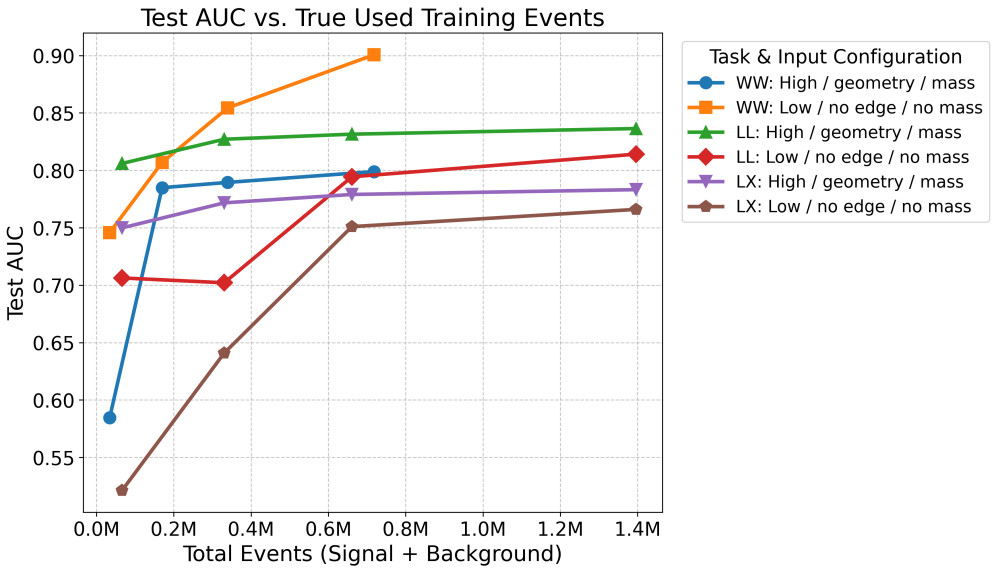


Figure 18: Test AUC value with different size of dataset for WW, LL, and LX tasks.

The high-level models approach a plateau much earlier. Their improvement beyond 500k is comparatively small.

- For WW:
  - Low-level no-edge WW is clearly not saturated at 500k: $0.8069 \to 0.8544 \to 0.9006$.
  - Low-level geometry WW is approaching saturation, as the increase from 1M to 2.11M is only 0.0036; the 1M mark has already reached approximately 99.2% of the AUC-above-random value of 2.11M.
- For LL:
  - Low-level no-edge LL is unstable at smaller sizes: 500k is slightly worse than 100k. However, 1M and 2.11M give large improvements.
  - Low-level geometry started saturating but not yet complete, as the gain gradually decreases: $+0.0330 \to +0.0146 \rightharpoonup +0.0052$.
- For LX:
  - Low-level no-edge LX also continues improving: $0.6409 \to 0.7510 \to 0.7660$.
  - Low-level geometry saturation has not yet been reached, as the value increases by 0.0131 when moving from 1M to 2.11M. Almost the same as the +0.0129 increase observed when moving from 500k to 1M.

In conclusion, saturation depends on the representation: high-level are approximately saturated around 500k-1M, while low-level no-edge generally are not saturated at 500k.

30.2 Paper-Scale ParT for WW no-edge low-level

Since there may have improvements for WW no-edge low-level training in the previous section after adding more dataset, we tried to use another method: enlarge the model structure. Our original aligned ParT model:

- 3 Particle attention blocks
- 1 Class attention block
- dimension 64
- 4 heads
- hidden dimension 256

And the original ParT model mentioned in the paper [10] is using the following settings: **8 particle attention blocks, 2 class attention blocks, embedding dimension 128, 8 heads, and hidden dimension 512.**

Task	Aligned no-edge (min, epoch)	Paper-scale (min, epoch)
WW	**0.9006** (127.9, 63)	0.8771 (385.2, 66)
LL	0.8141 (194.5, 49)	**0.8193** (466.8, 41)
LX	**0.7660** (158.8, 40)	0.5167* (125.4*, 11)

Table 46: Comparison of aligned no-edge and original paper-scale ParT models for low-level training.

- For WW: model overfits as it gets larger.
  - Best epoch = 56; Train AUC = 0.9332; Validation AUC = 0.8766; Test AUC = 0.8771.
  - Increasing the Transformer capacity did not improve the AUC. Instead, it led to increased overfitting, and the training time was approximately three times that of the original.
- For LL: slight improvement, but poor efficiency.
  - Spending 2.4 times the time yielded an AUC increase of only +0.0051.
- For LX: undergoes optimization collapse.
  - Epoch-by-epoch checks revealed that the loss remained fixed at: $0.693147 \approx \ln 2$. This indicates that the model has degenerated to outputting a constant signal probability, rather than learning normally.

31 Edge Feature Comparison

We also are curious about how edge features affect the model performance since edge features (interaction features) are included in the attention, the bias, which is the special part component for ParT model. Therefore, we try to train the model by the following settings:

- low-level / high-level
- with edge features (geometry: $\Delta\eta$, $\Delta\phi$, and ΔR / ParT interactions: $\ln\Delta, \ln k_T, \ln z, \ln m^2$) / without edge features

All models use 2.11M raw events per process before the signal region cut. The trained architectures are summarized as follow:

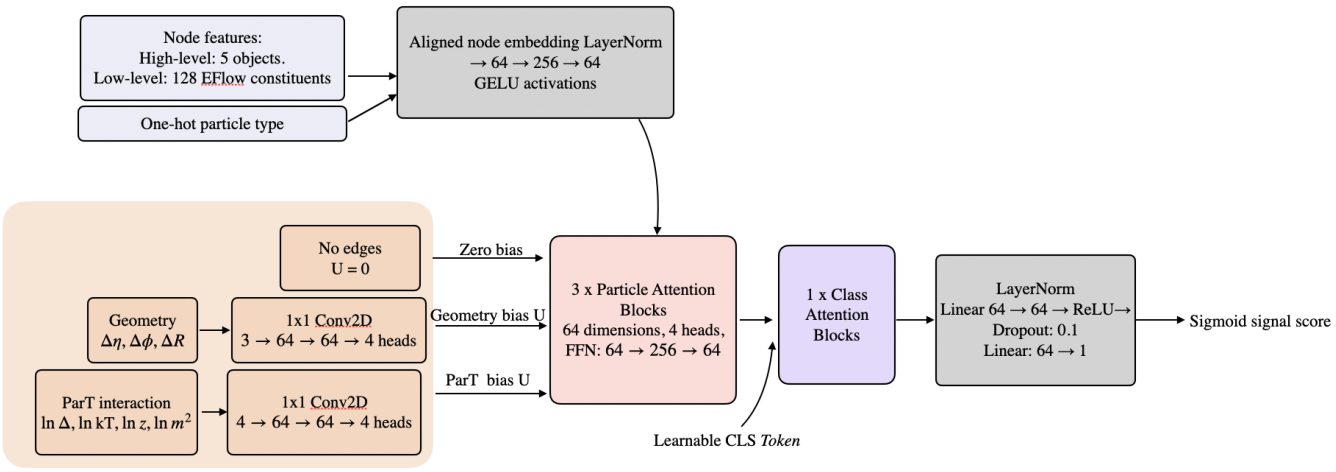


Figure 19: Three different kinds of edge features training model architecture.

Inside each particle-attention block:

$$A_{ij}^{(h)} = \text{Softmax}_j \left[\frac{Q_i^{(h)} K_j^{(h)T}}{\sqrt{16}} + U_{ij}^{(h)} + M_{ij} \right],$$

where $h - 1, ..., 4$ is the attention head and M is the padding mask.

Level	Mass	Interaction	WW (min, epoch)	LL	LX
High	Included	None	0.7975 (48.2, 94)	0.8347 (58.7, 60)	0.7795 (52.9, 54)
High	Included	Geometry	0.7989 (23.5, 79)	**0.8364 (32.0, 58)**	**0.7832 (27.8, 48)**
High	Included	ParT	0.8013 (18.1, 74)	0.8361 (26.2, 58)	0.7816 (39.7, 75)
Low	Excluded	None	0.9006 (127.9, 63)	0.8141 (194.5, 49)	0.7660 (158.8, 40)
Low	Excluded	Geometry	0.9310 (236.4, 55)	0.8246 (257.1, 31)	0.7743 (315.7, 38)
Low	Excluded	ParT	**0.9345 (543.6, 47)**	0.8314 (809.7, 38)	0.7751 (424.8, 38)

Table 47: Performance for three different kinds of interaction input: without edge features, with geometry interaction features, and the original ParT interaction features.

- Edge features have little effect on high-level models. All improvements over on edges are below 0.004 AUC.
- Edge features consistently improve low-level models:
  - WW: $+0.030 \sim 0.034$

  - LL: $+0.011 \sim 0.017$
  - LX: approximately +0.009
- ParT gives the highest low-level AUC for all three tasks.
- Geometry provides a substantially better performance-cost compromise. ParT training took approximately 544, 810, and 425 minutes for WW, LL, and LX.

9/29

32 Hybrid Model

Given that different types of tasks benefit from either low-level or high-level models, we attempt to employ a hybrid model as shown in the list below:

- high-level + low-level input + original ParT interaction features. (aligned + one hot coding + Conv2D)
- high-level input + original ParT interaction features + geometry edge features. (aligned + one hot coding + Conv2D)
- low-level input + original ParT interaction features +geometry edge features. (aligned + one hot coding + Conv2D)
- high-level + low-level input + original ParT interaction features + geometry edge features. (aligned + one hot coding + Conv2D)

Input	**Interactions**	**WW AUC**	**LL AUC**	**LX AUC**
High+Low	ParT	**0.93115** (248.3; 51)	0.83639 (360.7; 38)	0.78147 (478.0; 50)
High	Geometry+ParT	0.79853 (160.7; 83)	0.83618 (135.4; 36)	0.78318 (50.0; 50)
Low	Geometry+ParT	0.92523 (532.2; 54)	0.83098 (562.1; 29)	0.77684 (461.3; 39)
High+Low	Geometry+ParT	0.92034 (325.2; 48)	**0.83704** (409.7; 31)	**0.78428** (423.6; 33)

Table 48: Hybrid model performance for aligned, one hot coding, and Conv2D model structure. Training time are written in the unit minutes.

32.1 Geometry Edge vs. ParT Edge

Should we combine geometry edge and ParT original edge and plug them into the model? By comparing the results for high+low for the interaction features "ParT" only or "Geometry+ParT", the results are very task dependent.

- WW: Geometry channels decrease the performance obviously.
- LL: Small improvement after the combination.
- LX: Larger improvement but still subtle.

There is partial overlap between Geometry and Original ParT features:

- ΔR and $\ln \Delta$ highly related.
- $\Delta\eta$ and $\Delta\phi$ are already included in Δ.

For WW, additional features might represent redundancy or noise. For polarization tasks, explicit angular and directional information (signed $\Delta\eta$ and $\Delta\phi$) can be more beneficial.

32.2 High Level Tokens

Does adding high-level tokens help improve the performance? Using the same edge setting: geometry + ParT edges, we compare the performance for low-level input object only and low-level with high-level together.

- WW: High-level tokens decreases 0.005 AUC.
- LL and LX: both improved 0.00606 and 0.00744 AUC, respectively.

But compare to the high-level only model performance, there is only subtle improvement for LL and LX, 0.00086 and 0.00110, respectively. In short, LL and LX improvement highly related to high-level information, low-level constituents only improve the performance about 0.001. While WW signal/background discrimination mainly comes from low-level constituent patterns. High level objects are reconstructed from constituents, causing information duplication.

33 Two-Sigmoid Multitask ParT

Compare to the original dense-NN in the thesis, the model they trained used a two step classifier due to the reason that LL or LX signals are relatively small compare to the background. However, if we can somehow train our model and let it output two kinds of score simultaneously, it should become a more effective model.

33.1 Model Structure

For each event, instead of outputting one signal score, multitask particle transformer output two scores: signal head p(EW signal) and polarization head p(LL or LX | EW). There are two kinds of targets: detection target (Polarized EW vs. external background) and polarization target (LL vs. LT+TT or LL+LT vs. TT). They both share the same particle transformer block and CLS representation. This test uses low-level particle inputs: EFlowTrack, EFlowPhoton, and EFlowNeutralHadron.

- Signal: `WW-EW-LL`, `WW-EW-LT`, and `WW-EW-TT`.
- Background: `WW-QCD`, `WZ-EW`, and `WZ-QCD`.

Non-polarized WW-EW are not included into the training. The multitask model employs a shared ParT encoder connected to two sigmoid heads:

$$x \longrightarrow \text{shared ParT encoder} \begin{cases} s_{\text{det}} & \text{Detection head} \\ s_{\text{pol}} & \text{Polarization head} \end{cases}$$

Detection head

$$y^{\text{det}} = \begin{cases} 1, & LL,\ LT,\ TT \\ 0, & WW_{\text{QCD}},\ WZ_{\text{EW}},\ WZ_{\text{QCD}} \end{cases}$$

Polarization head

- LL model:

$$y^{\text{pol}} = \begin{cases} 1, & LL \\ 0, & LT,\ TT \end{cases}$$

- LX model:

$$y^{\text{pol}} = \begin{cases} 1, & LL,\ LT \\ 0, & TT \end{cases}$$

There is no polarization label for external background. Therefore, they only train the detection head, and the weight assigned to the polarization loss is zero.

Loss for the multitask model is defined by:

$$\mathcal{L} = \mathcal{L}_{\text{detection}} + \mathcal{L}^{\text{masked}}_{\text{polarization}}.$$

where

$$\mathcal{L}_{\text{detection}} = \text{BCE}(y_{\text{EW}}, s_{\text{det}})$$

is calculated for all signal and background events.

Polarization loss

$$\mathcal{L}_{\text{pol}} = \frac{\sum_i M_i w_i \,\text{BCE}(y_{\text{pol},i}, s_{\text{pol},i})}{\sum_i M_i w_i},$$

where

$$M_i = \begin{cases} 1, & \text{polarized EW signal} \\ 0, & \text{external background.} \end{cases}$$

33.2 Weighting Method

If the actual event yields are used directly:

$$N_{\text{background}} \gg N_{\text{LX}} \gg N_{\text{LL}},$$

the total loss is almost entirely dominated by the background. Consequently, even if the model predicts "background" for every event, it could still achieve a very low average loss.

The current model is using a three level weighting:

1. For each process, the total weight retains the original Senior/thesis process weight.
2. Within each train/validation/test split, the weight is first divided by the process's event count.
3. The two binary categories for each head are then separately normalized to the same total weight.

Currently, the two heads each uses the same weighting method mentioned in the thesis.

33.3 Checkpoint Selection

For the multitask checkpoint, the following is used as the selection metric:

$$\frac{\mathrm{AUC}_{\mathrm{det,val}} + \mathrm{AUC}_{\mathrm{pol,val}}}{2}$$

In addition to the two individual AUCs, a one-step score is also calculated:

$$s_{\mathrm{one-step}} = (s_{\mathrm{det}}) \times (s_{\mathrm{pol}}).$$

This represents:

$$P(\text{longitudinal target}) = P(\text{EW})P(\text{longitudinal} \mid \text{EW}).$$

33.4 Result

Table 49: Average scores : LL model

Process	Detection	Polarization	Product
LL	0.844	0.690	0.585
LT	0.828	0.463	0.386
TT	0.847	0.276	0.235
WWjj QCD	0.439	0.416	0.173
WZjj EW	0.289	0.365	0.107
WZjj QCD	0.171	0.385	0.066

Table 50: Average scores : LX model

Process	Detection	Polarization	Product
LL	0.825	0.720	0.588
LT	0.816	0.614	0.494
TT	0.847	0.399	0.331
WWjj QCD	0.437	0.629	0.252
WZjj EW	0.280	0.532	0.143
WZjj QCD	0.176	0.553	0.087

Both output one-step scores show that the model are learning the right thing. Scores for signals are way higher than external backgrounds, despite external backgrounds still having a score for the polarization head.

Target	Edges	Detection AUC	Polarization AUC	Time	Epochs
LL	ParT	0.91573	0.82772	517.3 min	74
LL	Geometry+ParT	**0.92426**	**0.82967**	871.1 min	68
LX	ParT	0.92503	0.77195	355.6 min	51
LX	Geometry+ParT	**0.92524**	**0.77504**	927.7 min	63

Table 51: Performance for multitask ParT with ParT edges or geometry and ParT edges. Both for LL and LX task testing ParT original interactions only or geometry+ParT interactions.

33.4.1 The Effect of Geometry on Multitasking

After adding geometry edge into the training, both LL and LX task improved.

- LL detection head improved obviously.
- LL and LX polarization head improved subtly.
- Training time also increased after adding geometry edges.

33.4.2 Multitask Diagnostic

Does multitasking improve the performance for polarization? **Multitasking did not improve the polarization AUC in isolation.** The discriminating features for detection head and polarization head are not the same. Shared encoder has to serve for both objectives, therefore, decrease a little bit of the polarization performance.

33.4.3 2D Binning Results

I also try to calculate the 95% CL upper limit and expected discovery significance Z_0 to compare with the previous dense-NN to see if the significance becomes larger while the upper limit becomes smaller.

Target	**Edges**	**Expected Z_0**	**μ_{95}**	**SM fiducial σ (fb)**	**95% upper σ (fb)**
LL	ParT	1.6560σ	2.1787	0.138912	0.302649
LL	Geometry + ParT	$\mathbf{1.6806\sigma}$	**2.1657**	0.138912	**0.300847**
LX	ParT	$\mathbf{5.2081\sigma}$	1.4185	0.579456	0.821949
LX	Geometry + ParT	5.1708σ	**1.4172**	0.579456	**0.821228**

Table 52: Expected discovery significance and 95% CL upper limit for multitask model.

Compared to the original dense-NN results: 1.512σ for LL and 4.557σ for LX, particle transformer truly shows higher expected discovery significance.

34 Multitask Model ParT

34.1 Flat Four-Class ParT

The output scores:

$$p_{\text{bkg}} + p_{\text{LL}} + p_{\text{LT}} + p_{\text{TT}} = 1.$$

For the 2D binning figure, we used the following variables:

$$D = 1 - p_{\text{bkg}},$$

$$P_{\text{LL}} = \frac{p_{\text{LL}}}{p_{\text{LL}} + p_{\text{LT}} + p_{\text{TT}}},$$

$$P_{\text{LX}} = \frac{p_{\text{LL}} + p_{\text{LT}}}{p_{\text{LL}} + p_{\text{LT}} + p_{\text{TT}}}.$$

Using the weighted cross entropy to let four class, external background, LL, LT, and TT, to have the same contribution in the training loss.

34.2 Sigmoid-Softmax ParT

There are two heads: detection sigmoid and polarization softmax, where detection sigmoid output score to classify WW-EW vs. external background and polarization softmax output scores to identify LL vs. LT vs. TT events. External background only participate in the detection loss. After each of the two loss (detection loss and polarization loss) terms is normalized, they are summed in a 1:1 ratio.

2D score uses:

$$D = \text{sigmoid}(z_{\text{det}}),$$

$$P_{\text{LL}} = p_{\text{LL}}, \qquad P_{\text{LX}} = p_{\text{LL}} + p_{\text{LT}}.$$

Model	WW AUC	LL AUC	LX AUC	epochs	Time
Flat four-class	0.9005	0.8345	0.7802	50	646.4 min
Hierarchical multiclass	**0.9162**	**0.8347**	**0.7807**	52	277.3 min

Table 53: Performance and training detail for flat four class and hierarchical multiclass ParT.

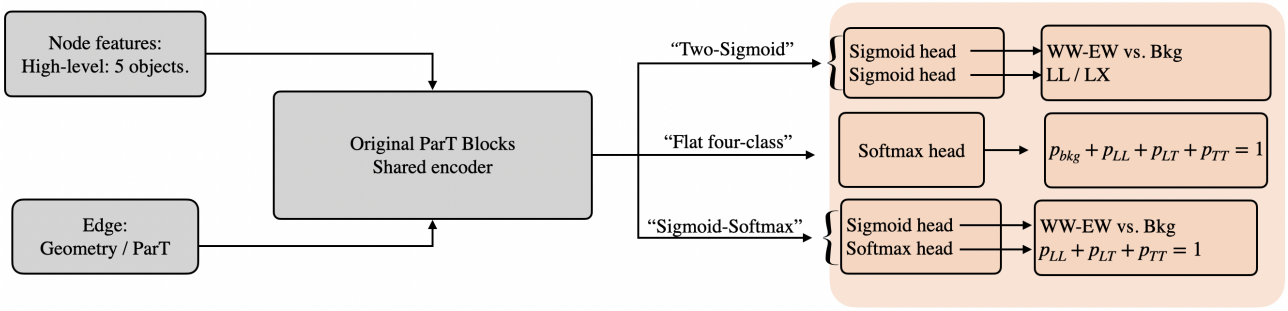


Figure 20: Model structure for the multitask ParT, including two-Sigmoid, flat four-class, and Sigmoid-Softmax settings.

Model	LL Z	LL μ_{95}	LX Z	LX μ_{95}
Dense-NN	1.5521	2.2013	4.8408	1.4000
ParT high, geometry	1.6703	2.1328	5.0183	1.3905
ParT low, geometry	1.6812	2.1178	5.2121	1.3775
ParT hybrid, geometry	1.7153	2.0922	Invalid	Invalid
ParT hybrid, geometry+ParT	**1.7778**	**2.0714**	**5.4562**	**1.3673**
Two-Sigmoid, ParT edges	1.6863	2.1140	5.2164	1.3798
Two-Sigmoid, geometry+ParT	1.7118	2.1011	5.1830	1.3779
Flat four-class	1.7566	2.1124	4.8227	1.3845
Sigmoid-Softmax	1.7726	2.1218	4.9480	1.3794

Table 54: LL and LX expected discovery significance and 95% CL upper limit using fixed thesis binning method and setting systematics=0.

According to Table 54, the best results are using the model two-step hybrid model with geometry and ParT edges.

35 Polarization vs. Background

In this section, we will discuss why the classification improvement for WW way easier than the polarization classification and try to figure out if we are able to adjust the model to improve the polarization tasks.

35.1 Difficulties

The difference between WW-EW and external backgrounds include: VBS tagging jets, forward topology, color/radiation pattern, central hadronic activity, leptons relative to tagging jets, and the overall event structure between WW-QCD, WZ and WW-EW. These information are mostly stored in the reconstructed particles. Therefore, ParT and low-level object input models can easily improve the AUC for WW task.

For polarization, the main information comes from $W \to \ell\nu$ decay angle.

$$W_L : \quad \frac{d\Gamma}{d\cos\theta^*} \propto \sin^2\theta^*,$$

$$W_T : \quad \frac{d\Gamma}{d\cos\theta^*} \propto (1 \pm \cos\theta^*)^2.$$

But the problem is, there are two neutrinos for the final state. For the true experimental data, we are only able to measure combined transverse momentum for the two neutrinos. It is unknown which W boson corresponds to each neutrino and the longitudinal momentum of the neutrino. The label for polarization even depend on the reference frame of the polarization.

35.2 Oracle Test

Oracle means a deliberately impossible real data input: generator-truth neutrinos or W bosons are supplied at inference. It tests the information ceiling for that particular classifier and truth variables, not an attainable bound for reconstructed data. Before sending jobs for large time training, we first conduct an information-ceiling study.

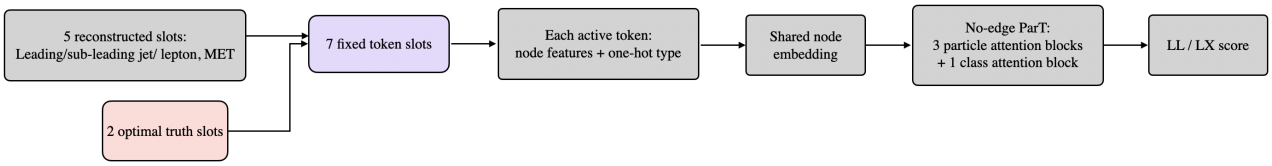


Figure 21: Model structure for oracle tests. Two optimal truth slots are added.

- Reco baseline: Reconstructed leptons, jets and MET.
- Truth-neutrino oracle: Reco inputs + truth ν_1, ν_2 four-vectors.
  To test missing neutrino information.
- Truth-W oracle: Reco inputs + truth W_1, W_2 four-vectors.
  To test the theoretical polarization ceiling.

There are 2.11M raw events for each process before signal region cut. Node features are using $\ln p_T, \eta, \Delta\phi, \ln\text{mass}$ + object-type one-hot. Model with 3 particle blocks and 1 class attention block. For Truth W, the two final W copies with Status=62 from the generator record are used; for the truth neutrino, the direct neutrino daughter of each W is selected, preserving the W–neutrino pairing.

	LL			**LX**		
Input condition	**AUC**	**Epoch**	**Time**	**AUC**	**Epoch**	**Time**
Reco only	0.8339	57	55.7 min	0.7809	58	56.4 min
Reco + truth neutrinos	0.9102	60	58.8 min	0.8676	74	72.3 min
Reco + truth W bosons	0.9129	83	77.5 min	0.8609	63	58.6 min

Table 55: Performance and training detail for the oracle test.

According to Table 55, once the charged leptons are known, the truth neutrinos already capture virtually the entire improvement provided by the truth W bosons. Simply testing deeper ParT or other models is unlikely to recover the completely unobservable longitudinal momentum on its own.

The significant improvements achieved by Truth Oracle indicate that polarization-sensitive information is likely concentrated in:

- W rest-frame decay angles
- lepton–neutrino angular correlations
- individual W transverse momenta
- neutrino longitudinal momenta
- correct W reconstruction

However, the reconstruction model only has access to the total transverse imbalance of the two neutrinos. This constitutes an underconstrained inverse problem, where a single set of leptons and MET can correspond to multiple possible sets of neutrino momenta. Consequently, a single deterministic regression head tends to learn the average of all possible solutions—an average solution that does not necessarily preserve polarization information.

35.3 Oracle information decomposition

In this subsection, we try to figure out which specific component of the "truth neutrino" is responsible for the Oracle improvement, and can that component be recovered from the detector data? All six Oracle rows below uses: 2.11M raw/process, fold 0, aligned high-level no-edge ParT, identical weighting, checkpoint chosen by validation AUC.

Additional input information	**LL / LX AUC**	**LL / LX epoch**	**LL/LX time**
Reconstructed only	0.8339 / 0.7809	57 / 58	55.7 / 56.4 min
Perfect total truth-neutrino p_x, p_y	0.8417(10.2%) / 0.7858(5.7%)	69 / 43	67.3 / 42.2 min
Two truth-neutrino η	0.8626(37.6%) / 0.8121(49.4%)	57 / 58	56.0 / 56.8 min
Individual truth-neutrino p_T, ϕ	0.8716(49.4%) / 0.8194(44.5%)	69 / 52	55.0 / 41.5 min
Full truth-neutrino four-vectors	0.9102(100%) / **0.8676**(100%)	60 / 74	58.8 / 72.3 min
Truth W four-vectors	**0.9129** / 0.8609	83 / 63	77.5 / 58.6 min

Table 56: Performance for the oracle information decomposition test. The percentage written next to the AUC is the oracle gap recovered relative to full-neutrinos test.

According to Table 56, we have the following conclusions about the oracle information we might need to be aware of:

- MET resolution is not the main bottleneck.

  Replacing reconstructed MET with the perfect generator-level total neutrino transverse momentum improves only 0.0078 and 0.0050 for LL and LX task, respectively.

- **It is important to split the MET into two neutrinos.**

  Providing the (p_T, ϕ) of the two individual neutrinos increases the AUC by approximately 0.038, recovering nearly half of the "Oracle gap." In real data, only the following is measured:

  $$\vec{p}_T^{\,\text{miss}} \approx \vec{p}_{T,\nu_1} + \vec{p}_{T,\nu_2},$$

  yet the individual momenta of the neutrinos are unknown. Therefore, the crucial information is not the total MET, but rather the possible distribution of:

  $$\vec{p}_T^{\,\text{miss}} \longrightarrow (\vec{p}_{T,\nu_1}, \vec{p}_{T,\nu_2})\,.$$

- **Longitudinal information is also very important.**

  Even when only the neutrino pseudorapidity η is included (without providing the true p_T or ϕ), this improves the LL and LX task 0.0287 and 0.0312, respectively. This indicates that polarization discrimination relies heavily on the longitudinal direction of the neutrino.

  However, the detector cannot directly measure the neutrino's p_z or η. This information can only be inferred statistically using visible leptons, jets, measured MET, event topology, or on-shell constraints.

- Transverse and longitudinal information are complementary.

  The combined improvements from the transverse component alone and the η component alone still fall short of the full-neutrino result. The remaining gaps are approximately:

  - LL: about 13% of the full Oracle gap.
  - LX: about 20% of the full Oracle gap.

This indicates that polarization information is still present in: p_T–η correlations, neutrino–lepton pairing, two-neutrino correlations, and reconstructed W rest-frame geometry.

- Truth W does not significantly outperform full neutrinos.

  The results for "truth W" and "full-neutrino" are very close; the slight differences likely stem from training fluctuations. Therefore, "directly predicting the truth W" need not be viewed as a completely different approach; the primary useful information is already contained within the lepton and neutrino kinematics.

In conclusion, we should try to figure out how to let the model to predict multiple possible individual-neutrino configurations from reconstructed objects.

35.4 Neutrino Reconstruction Difficulties

For our W vector boson scattering processes, it is a two-neutrino problem, which leads out that there is no unique solution. Each neutrino contains three dimension of freedom:

$$(p_x, p_y, p_z).$$

Therefore, there are six DoF for two-neutrino process. MET gives out two constraints:

$$p_{x,\nu_1} + p_{x,\nu_2} = p_x^{\text{miss}},$$

$$p_{y,\nu_1} + p_{y,\nu_2} = p_y^{\text{miss}}.$$

W boson mass also gives out two constraints:

$$(p_{\ell_1} + p_{\nu_1})^2 = m_W^2,$$

$$(p_{\ell_2} + p_{\nu_2})^2 = m_W^2.$$

There are still 2 DoF, which lead to the difficulty of the neutrino reconstruction.

The task of reconstructing neutrinos is an inverse problem. The detector gives out the measured events:

$$x = \{\ell_1, \ell_2, j_1, j_2, \text{MET}, \text{EFlow objects}, \ldots\}.$$

What we really want to know are the hidden / latent quantities

$$z = \{p_{\nu_1}, p_{\nu_2}\},$$

which is:

$$x \longrightarrow z.$$

It would be ideal if each x corresponded to a unique z:

$$z = f(x).$$

Deterministic regression can succeed in this scenario. However, the two-neutrino final state is closer:

$$x \longrightarrow \boxed{p(z|x)}$$

35.5 Helicity Angle

Besides reconstructing a specific set of four vectors for the two neutrinos, we can also choose to construct the helicity angle information since it is an variable that is highly related to the particle polarization. To construct the helicity angle, we need to first find out the parent particle:

$$W \to l\nu$$

1. Reconstruct $p_W = p_\ell + p_\nu$
2. Boost the event to the W-rest frame.
3. Calculate the helicity angle by:
$$\theta^* = \angle\left(\vec{p}_\ell^{\,*}, \hat{n}_W\right),$$
where $\hat{n}_W$ is the W boson flight direction in WW-cmf and $\vec{p}_\ell^{\,*}$ is for the charged lepton in W-rest frame.

10/7

36 Neutrino Model Testings for Last Week

Since for polarization tasks, we are having better performance using high-level object inputs. We then first try to use these high-level objects as the input for neutrino reconstruction and helicity angle learning. The following two subsections are all only using high-level objects as input.

36.1 Learned Neutrino Query

Query is a learned vector that requests information from object representations via attention, it does not contain a truth neutrino. In the original class-attention stage, in addition to the CLS token, two learnable neutrino query tokens are added:

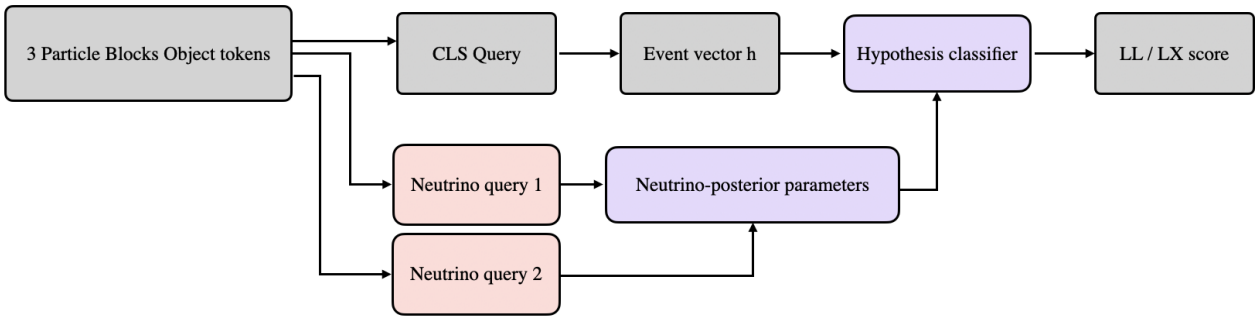


Figure 22: Model structure for the Neutrino Series testings.

*Neutrino posterior parameters: For each event, the two query outputs are concatenated into a 128-dimensional vector. The model then outputs:

$$\{\pi_k, \mu_k, \sigma_k\}, \qquad k = 1, \ldots, 8.$$

where:

- π_k: the mixture weight of the k-th neutrino configuration.
- μ_k: the center of the k-th component.
- σ_k: the width or uncertainty range of the k-th component.

Each μ_k consists of four latent quantities:

$$\mu_k = \left(\frac{p_{x,\nu_1}}{|\text{MET}|}, \frac{p_{y,\nu_1}}{|\text{MET}|}, \eta_{\nu_1}, \eta_{\nu_2} \right)_k .$$

Then, the second neutrino is defined using the measured MET:

$$\vec{p}_{T,\nu_2} = \vec{E}_T^{\text{miss}} - \vec{p}_{T,\nu_1}.$$

Model	**Task**	**Weighted AUC**	**Trained epoch**	**Time (min)**	**Classification loss**	**Neutrino NLL**
Matched baseline	LL	0.8267	109	68.2	0.5111	—
Joint classification query	LL	0.8255	118	82.0	0.5131	2.0447
Truth-pretrained query	LL	**0.8284**	91	14.6	**0.5081**	**1.6307**
Matched baseline	LX	0.7742	84	43.8	0.5722	—
Joint classification query	LX	0.7732	101	60.9	0.5728	2.0119
Truth-pretrained query	LX	**0.7749**	75	12.1	**0.5696**	**1.5609**

Table 57: Performance for learned neutrino query series ParT model. For joint classification query, query branch and classifier trained together by classification loss; no truth NLL in this phase. $L = L_{\text{classification}}$. For truth-pretrained query: first 10 epochs of truth-neutrino NLL only; $L = L_{\nu,\text{NLL}}$.. Then classification plus truth coefficient λ_ν 0.02 $\rightarrow$ 0 over 30 epochs. $L = L_{\text{classification}} + \lambda_\nu L_{\nu,\text{NLL}}$,

*NLL: negative log-likelihood.

Let x be the reconstructed event and t be the MC truth-neutrino configuration. The model outputs:

$$q_\theta(t \mid x).$$

This represents the density reflecting how plausible the truth configuration t is, according to the model, given the reconstructed event x. NLL is defined as:

$$\text{NLL} = -\ln q_\theta(t \mid x).$$

Small gains from truth pretraining missed the +0.005 pilot threshold. The next step changed where the branch is attached and explicitly constructed candidate W bosons.

36.2 Helicity Angle Model

In this reconstructed model, we try to let the model learn the most related information to the W polarization, the decay angle.

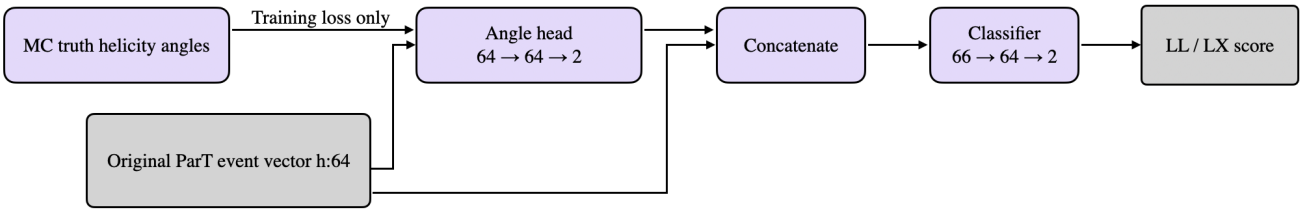


Figure 23: Model structure for helicity angle series testings.

There are four types trained: matched angle-head control, P_2 supervised, signed-$\cos\theta^*$-supervised, and matched full-data control. The difference lies in whether the angle head is trained using the physical "truth target" and the amount of data utilized.

1. Matched angle-head control

   Only 500k / process dataset are used. It has an angle head that outputs two numbers. These two numbers are concatenated with h and fed into the classifier. But there is no truth angle pretraining or loss, there is only the LL/LX classification loss. Therefore, these two numbers are merely arbitrary hidden features and cannot be termed true predicted helicity angles.

2. P_2-supervised: primarily used for LL tasks.

   First, calculate the following for the MC truth W boson and truth neutrino:

   $$c_i = \cos\theta_i^*.$$

   Then, convert this to:

   $$P_2(c_i) = \frac{3c_i^2 - 1}{2}.$$

   Since squaring eliminates the sign:

   $$P_2(+c) = P_2(-c).$$

   This approach focuses on the longitudinal versus transverse decay angular shapes, without requiring the model to distinguish the sign of the angle. The model outputs two P_2 predictions; it is first pre-trained using the truth P_2 values and subsequently fine-tuned in conjunction with the LL classification task.

3. Signed-$\cos\theta^*$-supervised: primarily used for LX tasks.

   The target is simply:

   $$(c_1, c_2) = (\cos\theta_1^*, \cos\theta_2^*).$$

   It preserves the sign:

   $$+0.7 \neq -0.7.$$

4. Matched full-data control: this training uses the full dataset, 2.11M / process.

Dataset / Task	Model	AUC
500k LL, 5-fold mean	Matched angle-head control	0.825260
500k LL, 5-fold mean	P_2-supervised	0.830438
500k LX, 5-fold mean	Matched angle-head control	0.772932
500k LX, 5-fold mean	Signed-cos θ^*-supervised	0.777961
2.11M LL	Matched full-data control	0.834319
2.11M LL	P_2-supervised	0.837245
2.11M LX	Matched full-data control	0.781510
2.11M LX	Signed-cos θ^*-supervised	0.784629

Table 58: Performance comparison across different datasets, tasks, and supervision models for the helicity angle testings.

It gains appeared in every tested 500k fold and persisted at 2.11M, but became smaller.

37 Reconstruct Neutrinos by Neural Network

Since the oracle test in the previous section indicates that truth neutrino information improve the performance for the classification task significantly, we try to figure out if there is any published paper dealing with the neutrino reconstruction problem using neural network.

In the paper: ν^2-Flows: Fast and improved neutrino reconstruction in multi-neutrino final states with conditional normalizing flows [11], they also deal with two neutrino problem:

$$t\bar{t} \to b\ell\nu\,\bar{b}\ell\nu$$

Instead of having the neural network simply guess a single pair

$$\hat{\mathbf{p}}_{\nu_1}, \hat{\mathbf{p}}_{\nu_2},$$

it learns

$$\boxed{p(\mathbf{p}_{\nu_1}, \mathbf{p}_{\nu_2} \mid X_{\text{measured}})}$$

In other words: given a measured event, which neutrino configurations are more likely?

37.1 Model Structure

The model structure can be divided into two part: event encoder and conditional normalizing flow

$$\boxed{\underbrace{\text{Event Encoder}}_{\text{Understand measured event}} \quad + \quad \underbrace{\text{Conditional Normalizing Flow}}_{\text{Learn } p(\nu_1,\nu_2|x)}}$$

37.1.1 Event Encoder

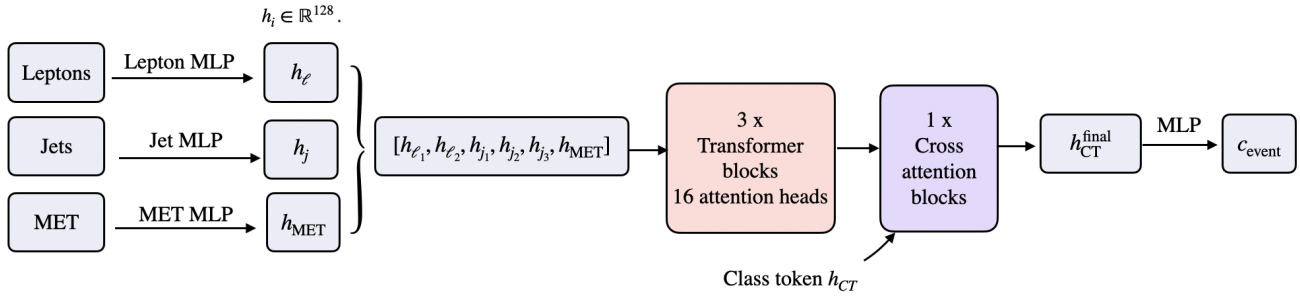


Figure 24: The event encoder model structure for ν^2 Flow

Event encoder only transform the measured event information into a 128-dimensional vector.

$$\boxed{x_{\text{measured}} \rightarrow c_{\text{event}}}$$

$$\boxed{c_{\text{event}} \in \mathbb{R}^{128}}$$

37.1.2 Conditional Normalizing Flow

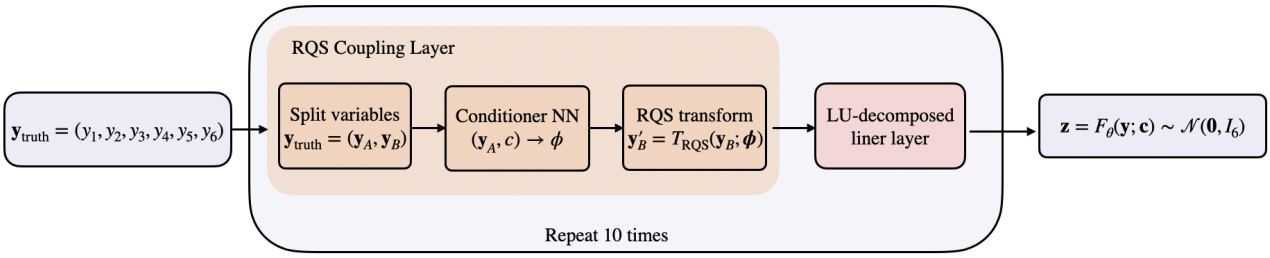


Figure 25: The conditional normalizing flow model structure for ν^2 Flow

During training, the model tries to learn the conditional probability density. And the workflow looks like this:

$$\mathbf{x} \rightarrow \text{Object MLPs} \rightarrow \text{Transformer} \rightarrow \mathbf{c},$$

with

$$\mathbf{y}_{\text{truth}} \xrightarrow[\mathbf{c}]{F_\theta} \mathbf{z}.$$

The training loss is using the negative log-likelihood (NLL):

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^{N} \log p_\theta(\mathbf{y}_i^{\text{truth}} \mid \mathbf{x}_i),$$

while we get our conditional probability for each candidate neutrino configuration by:

$$\log p_\theta(\mathbf{y} \mid \mathbf{x}) = \log p_Z(\mathbf{z}) + \sum_{k=1}^{K} \log \left| \det J_k \right|.$$

During testing, the workflow is just the inverse version. And since we required the transformation to be invertible, we can get our inverse transform:

$$\mathbf{x} \to \text{Event Encoder} \to \mathbf{c},$$

with

$$\mathbf{z} \xrightarrow{F_\theta^{-1}} \mathbf{y}_{\text{truth}}.$$

Sample from standard Gaussian:

$$\mathbf{z}^{(1)} \sim \mathcal{N}(\mathbf{0}, I_6).$$

And get the first candidate neutrino:

$$\mathbf{y}^{(1)} = F_\theta^{-1}(\mathbf{z}^{(1)}; \mathbf{c}).$$

We can then get several configurations of candidate neutrino by repeatedly sampling from the standard Gaussian. Finally, we get several sets of candidate neutrino configurations and the corresponding conditional probability density:

$$\mathbf{y}^{(k)} \sim p_\theta(\mathbf{y} \mid \mathbf{x}).$$

37.2 Claim

Claim	**Their Results**
Multiple neutrino reconstruction	Capable of joint reconstruction of multiple neutrinos
Ambiguous solution	Learns joint conditional probability density $p_\theta(\nu_1, \nu_2 \mid x)$
Correlations	Able to reproduce neutrino-pair correlations
Bias	Smaller bias compared to ν-Weighting / Ellipse
Hard mass constraints	Does not require hard imposing m_W, m_t
Failed events	Flow produces solutions for all events; their Ellipse implementation has $\sim 22\%$ without a solution
Direct regression	Significantly worse than Normalizing Flow
Speed	~ 71 ms/event on CPU; GPU batching can be extremely fast
Downstream physics	Unfolding statistical precision improved by ~ 1.5–$2\times$ over ν-Weighting, and up to $\sim 4\times$ over Ellipse
Robustness	Maintains good performance when changing generators or including ISR

Table 59: Summary of results from Raine et al., "ν^2-Flows"

10/19

38 ν^2 Flow with Classifier

We apply the ν^2 flow as our neutrino reconstruction model and then connect it to the classifier. ν^2 flow gives out a conditional probability distribution, we can sample from the distribution and give out potential solutions to the neutrino pairs. But the interface of the ν^2 flow and the classifier varies, we tried three kinds of interface:

1. Single-draw baseline uses one sampled neutrino pair.

   $$x_{\text{measured}} \to p_\theta(\nu_1, \nu_2 \mid x)$$

   Although the Flow learns a whole conditional distribution, but we only do posterior draw once:

   $$z^{(1)} \sim \mathcal{N}(0, I_6)$$

   then,

   $$(\nu_1^{(1)}, \nu_2^{(1)}) = F_\theta^{-1}(z^{(1)}; x).$$

2. Highest-density selects one neutrino pair according to the largest conditional flow density.

   From several samples:

   $$(\nu_1^{(1)}, \nu_2^{(1)}), \ldots, (\nu_1^{(N)}, \nu_2^{(N)})$$

   choose the highest conditional density:

   $$(\nu_1, \nu_2) = \arg\max_k p(\nu_1^{(k)}, \nu_2^{(k)}|x)$$

3. Score average averages the classifier probabilities from all the peaks in the conditional distribution. The number of peaks varies with the event.

   Sample from the conditional distribution:

   $$(\nu_1^{(k)}, \nu_2^{(k)}) \sim p(\nu_1, \nu_2|x)$$

   Every set of neutrino go through the classifier once:

   $$P(C|x, \nu_1^{(k)}, \nu_2^{(k)}).$$

   And finally average against all potential samples:

   $$P(C|x) \approx \frac{1}{N} \sum_{k=1}^{N} P(C|x, \nu_1^{(k)}, \nu_2^{(k)})$$

Flow inputs	Task	Single-draw baseline	Highest-density	Score average
5 objects	LL	0.833047	0.835461	–
5 objects	LX	0.777850	0.780548	–
3 objects	LL	0.829559	0.833029	–
3 objects	LX	0.778478	0.781957	–

Table 60: Comparison of baseline and density-based sampling across tasks and object counts.

References

[1] ATLAS Collaboration. "Evidence for longitudinally polarized W bosons in the electroweak production of same-sign W boson pairs in association with two jets in pp collisions at $\sqrt{s} = 13$ TeV with the ATLAS detector". In: *Phys. Rev. Lett.* 135 (2025), p. 111802. DOI: 10.1103/bpln-ccql.

[2] The ATLAS Collaboration. "Measurement and interpretation of same-sign W boson pair production in association with two jets in pp collisions at $\sqrt{s} = 13$ TeV with the ATLAS detector". In: *JHEP* 04 (2024), p. 026. DOI: 10.1007/JHEP04(2024)026. arXiv: 2312.00420 [hep-ex].

[3] Max V. Stange. "Longitudinal $W^{\pm}$ Boson Polarization in Same-Charged $W^{\pm}W^{\pm}$ Scattering in the ATLAS Experiment". Doctoral thesis. Technische Universität Dresden, Jan. 2025.

[4] J. Alwall et al. "The automated computation of tree-level and next-to-leading order differential cross sections, and their matching to parton shower simulations". In: *JHEP* 07 (2014), p. 079. DOI: 10.1007/JHEP07(2014)079. arXiv: 1405.0301 [hep-ph].

[5] Enrico Bothmann et al. "Event generation with Sherpa 3". In: (Oct. 2024). arXiv: 2410.22148 [hep-ph].

[6] Barry M. Dillon and Michael Spannowsky. "Theory-informed neural networks for particle physics". In: *arXiv* (2025). DOI: 10.48550/arxiv.2507.13447.

[7] Alexander Shmakov et al. "SPANet: Generalized Permutationless Set Assignment for Particle Physics using Symmetry Preserving Attention". In: *SciPost Physics* 12 (2022), p. 178. DOI: 10.21468/SciPostPhys.12.5.178. arXiv: 2106.03898 [hep-ex].

[8] M. J. Fenton, A. Shmakov, T.-W. Ho, et al. "Permutationless many-jet event reconstruction with symmetry preserving attention networks". In: *Physical Review D* 105.11 (2022), p. 112008. DOI: 10.1103/physrevd.105.112008.

[9] Miao Hu. "Precision Measurements of Vector Boson Scattering Production and Searches for Charged Higgs Bosons at the Large Hadron Collider". Thesis Advisor: Markus Klute. PhD thesis. Massachusetts Institute of Technology, Sept. 2022. URL: https://dspace.mit.edu/handle/1721.1/146059.

[10] Huilin Qu, Congqiao Li, and Sitian Qian. "Particle Transformer for Jet Tagging". In: *Proceedings of the 39th International Conference on Machine Learning, PMLR 162:18281-18292, 2022* (2022). DOI: 10.48550/arxiv.2202.03772.

[11] Johnny Raine et al. "ν^2-Flows: Fast and improved neutrino reconstruction in multi-neutrino final states with conditional normalizing flows". In: *SciPost Phys.* 16.5 (2024), p. 127. DOI: 10.21468/SciPostPhys.16.5.127. arXiv: 2312.02278 [hep-ph].